Java Interview Questions

497 questions · 41 topics · 8 parts

Core Java · Collections · JDBC · Java 8 features · Multithreading · Java 21 concurrency

Corrected / Updated (47 answers)
Where the original site's answer was wrong, outdated or garbled, the answer is fixed and a shaded note says what changed and which official source it was checked against (Javadoc, JLS, JEPs, vendor docs, or a run on JDK 21).

Use the **bookmarks panel** of your PDF viewer for the Part → Topic → Question tree. Every index line is a link.

Index

Core Java - Part 1

Core Java: Basics of Java Interview Questions ..... 22 Q 4
Core Java - OOP Concepts: Initial OOP Interview Questions ..... 4 Q 14
Core Java - OOPs Concepts: Constructor Interview Questions ..... 12 Q 16
Core Java - OOPs Concepts: static Keyword Interview Questions ..... 12 Q 22
Core Java - OOPs Concepts: Inheritance Interview Questions ..... 21 Q 26
Core Java - OOP Concepts: Method Overloading Interview Questions ..... 6 Q 37
Core Java - OOP Concepts: Method Overriding Interview Questions ..... 12 Q 40
Core Java - OOPs Concepts: final keyword Interview Questions ..... 11 Q 45

Core Java - Part 2

Core Java - OOPs: Polymorphism Interview Questions ..... 6 Q 49
Core Java - OOPs Concepts: Abstraction Interview Questions ..... 17 Q 52
Core Java - OOPs Concepts: Package Interview Questions ..... 7 Q 57
Java: Exception Handling Interview Questions ..... 18 Q 60
Java: String Handling Interview Questions ..... 19 Q 69
Core Java: Nested classes and Interfaces Interview Questions ..... 11 Q 77
Garbage Collection Interview Questions ..... 10 Q 81
I/O Interview Questions ..... 10 Q 85
Serialization Interview Questions ..... 8 Q 90
Networking Interview Questions ..... 5 Q 95
Reflection Interview Questions ..... 6 Q 97
Miscellaneous Interview Questions ..... 19 Q 99
Internationalization Interview Questions ..... 2 Q 105
Java Bean Interview Questions ..... 3 Q 106
RMI Interview Questions ..... 6 Q 107
Core Java: Data Structure Interview Questions ..... 9 Q 109

Multithreading & Concurrency

Multithreading Interview Questions and Answers ..... 50 Q 123
Concurrency Interview Questions ..... 35 Q 134

Collections

Java Collections Interview Questions and Answers ..... 34 Q 146

JDBC

JDBC Interview Questions and Answers ..... 30 Q 161

Java 8 Features

Overview & General ..... 5 Q 176
Lambdas & Functional Interfaces ..... 10 Q 178
Default & Static Methods ..... 5 Q 181
Streams ..... 19 Q 183

Collectors 4 Q 190
Optional 1 Q 192
Date & Time API 2 Q 193
JVM & Tools 4 Q 194

Java 21 Concurrency

Virtual Threads (Java 21) 12 Q 195
Structured Concurrency & Scoped Values 4 Q 199
Modern Concurrency Essentials 15 Q 201

Modern JVM & JDBC

Garbage Collection & JVM Memory (Modern) 6 Q 206
JDBC Best Practices (Modern) 5 Q 209

Core Java - Part 1

Core Java: Basics of Java Interview Questions

Questions in this topic (22)

1) What is Java?
2) What are the differences between C++ and Java?
3) List the features of Java Programming language.
4) What do you understand by Java Virtual Machine (JVM)?
5) What are the differences between JDK, JRE, and JVM?
6) How many types of memory areas are allocated by the JVM?
7) What is JIT compiler?
8) What is the platform?
9) What are the main differences between the Java platform and other platforms?
10) What gives Java its 'write once and run anywhere' nature?
11) What is a ClassLoader?
12) Is an empty .java file name a valid source file name?
13) Is delete, next, main, exit, or null keyword in Java?
14) If I do not provide any arguments on the command line, then what will the value stored in the String array passed into the main() method, empty or NULL?
15) What if we write static public void instead of public static void?
16) What is the default value of the local variables?
17) What are the various access specifiers in Java?
18) What is the purpose of static methods and variables?
19) What are the advantages of packages in Java?
20) What is the output of the following Java program?
21) What is the output of the following Java program?
22) What is the output of the following Java program?

1) What is Java?

Java is a high-level, object-oriented, robust, secure programming language, platform-independent, high-performance, Multithreaded, and portable programming language. It was developed by James Gosling and his team at Sun Microsystems in June 1991. It can also be known as the platform, as it provides its own JRE and API. Java is designed to be versatile, secure, and portable, allowing developers to write code that can run on various platforms without modification.

To read more: Java Tutorial

2) What are the differences between C++ and Java?

The differences between C++ and Java are given in the following table.

Aspect	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in Windows, web-based, enterprise, and mobile applications.
Design Goal	C++ was designed for systems and applications programming. It was an extension of the C programming language.	Java was designed and created as an interpreter for printing systems, but later extended as a support network for computing. It was designed with a goal of being easy to use and accessible to a broader audience.

Goto Statement	C++ supports the goto statement.	Java does not support the goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java does not support multiple inheritance through a class. Interfaces in Java can achieve it.
Operator Overloading	C++ supports operator overloading.	Java does not support operator overloading.
Pointers	C++ supports pointers. We can write a pointer program in C++.	Java supports pointers internally. However, we cannot write the pointer program in Java. It means Java has restricted pointer support in Java.
Compiler and Interpreter	C++ uses a compiler only. C++ is compiled and run using the compiler, which converts source code into machine code, so C++ is platform-dependent.	Java uses a compiler and an interpreter. Java source code is converted into bytecode at compilation time. The interpreter executes this bytecode at runtime and produces output. Java is interpreted, which is why it is platform independent.
Call by Value and Call by Reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in Java.
Structure and Union	C++ supports structures and unions.	Java does not support structures and unions.
Thread Support	C++ does not have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in thread support.
Documentation comment	C++ does not support documentation comments.	Java supports documentation comments (/** ... */) to create documentation for Java source code.
virtual Keyword	C++ supports the virtual keyword so that we can decide whether or not to override a function.	Java has no virtual keyword. We can override all non-static methods by default. In other words, non-static methods are virtual by default.
Unsigned Right shift >>>	C++ does not support the >>> operator.	Java supports the unsigned right shift >>> operator that fills zero at the top for the negative numbers. For positive numbers, it works the same as the>> operator.
Inheritance Tree	C++ creates a new inheritance tree every time.	Java uses a single inheritance tree always because all classes are the child of the Object class in Java. The object class is the root of the inheritance tree in Java.
Hardware	C++ is nearer to hardware.	Java is not so interactive with hardware.
Object-Oriented	C++ is an object-oriented language. However, in the C language, a single root hierarchy is not possible.	Java is also an object-oriented language. However, everything (except fundamental types) is an object in Java. It is a single root hierarchy as everything gets derived from java.lang.Object.
Memory Management	Manual memory management using new and delete keywords.	Automatic memory management through garbage collection.
Exception Handling	Supports try, throw, and catch for exception handling.	Exception handling using try, throw, and catch as well.
Standard Library	Standard Template Library (STL) for data structures and algorithms.	Java Standard Edition (SE) library for various utilities and functionality.
App Development	Commonly used for desktop applications.	Widely used for web, enterprise, and mobile applications.
Runtime Environment	Closer to the machine may have better performance.	Java Virtual Machine (JVM) provides a layer of abstraction, sacrificing some performance for portability.
Garbage Collection	Manual memory management, no built-in garbage collection.	Automatic garbage collection to manage memory.

To read more: Difference between C++ and Java

3) List the features of Java Programming language.

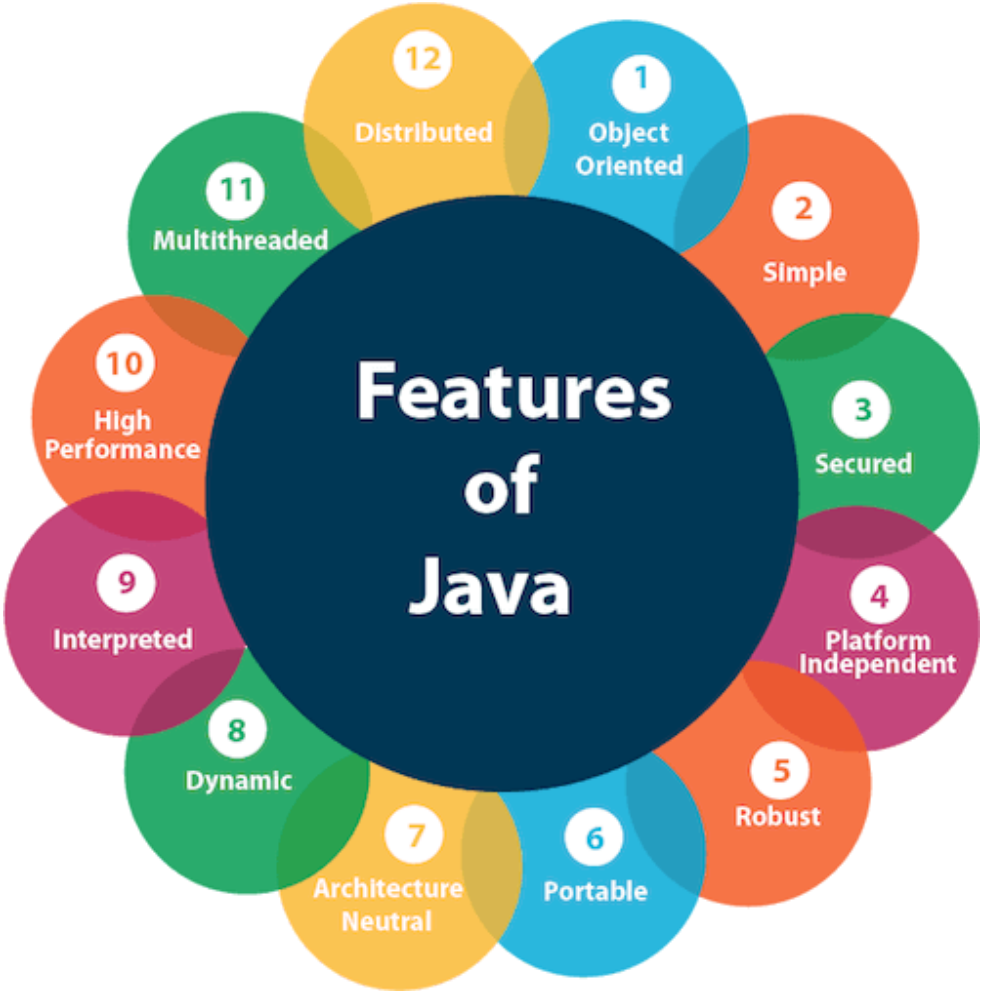
Corrected

CORRECTED The source said 'Everything in Java is treated as an object'. Java has eight primitive types (int, double, boolean ...) that are not objects. It also misquoted 'write once, run anywhere'.

Checked against: JLS chapter 4: types (primitive values are not objects).

- There are the following features in the Java Programming Language.
- **Simple:** Java is easy to learn. The syntax of Java is based on C++, which makes it easier to write programs in it.
- **Object-Oriented Programming (OOP):** Java follows the object-oriented paradigm, which allows us to maintain our code as a combination of different types of objects that incorporate both data and behavior. Java is object-oriented, but not purely: the eight primitive types are not objects. Organizing code into classes and objects promotes modularity and reuse.
- **Portable:** Java supports a write-once-run-anywhere approach. We can execute the Java program on every machine. A Java program (.java) is converted to bytecode (.class), which can be easily run on every machine.
- **Platform Independent:** Java is a platform-independent programming language. It is different from other programming languages like C and C++, which need a platform to be executed. Java comes with its own platform on which its code is executed. Java doesn't depend upon the operating system to be executed.
- **Secured:** Java is secured because it does not use explicit pointers. Java also provides the concept of ByteCode and Exception handling, which makes it more secured. The Java Runtime Environment (JRE) includes a security manager and a set of APIs to control access to resources and prevent malicious activities.
- **Robust:** Java is a strong programming language as it uses strong memory management. The concepts like Automatic garbage collection, Exception handling, etc., make it more robust.
- **Architecture Neutral:** Java is architecture-neutral as it is not dependent on the architecture. In C, the size of data types may vary according to the architecture (32-bit or 64-bit), which does not exist in Java.
- **Interpreted:** Java uses the Just-in-time (JIT) interpreter along with the compiler for the program execution.
- **High Performance:** Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (for example, C++). While interpreted, Java's Just-In-Time (JIT) compilation translates bytecode into machine code for improved performance.
- **Multithreaded:** We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it does not occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc. Java's bytecode can be executed on any device with a JVM, ensuring portability across different platforms.
- **Distributed:** Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.
- **Dynamic:** Java is a dynamic language. It supports dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

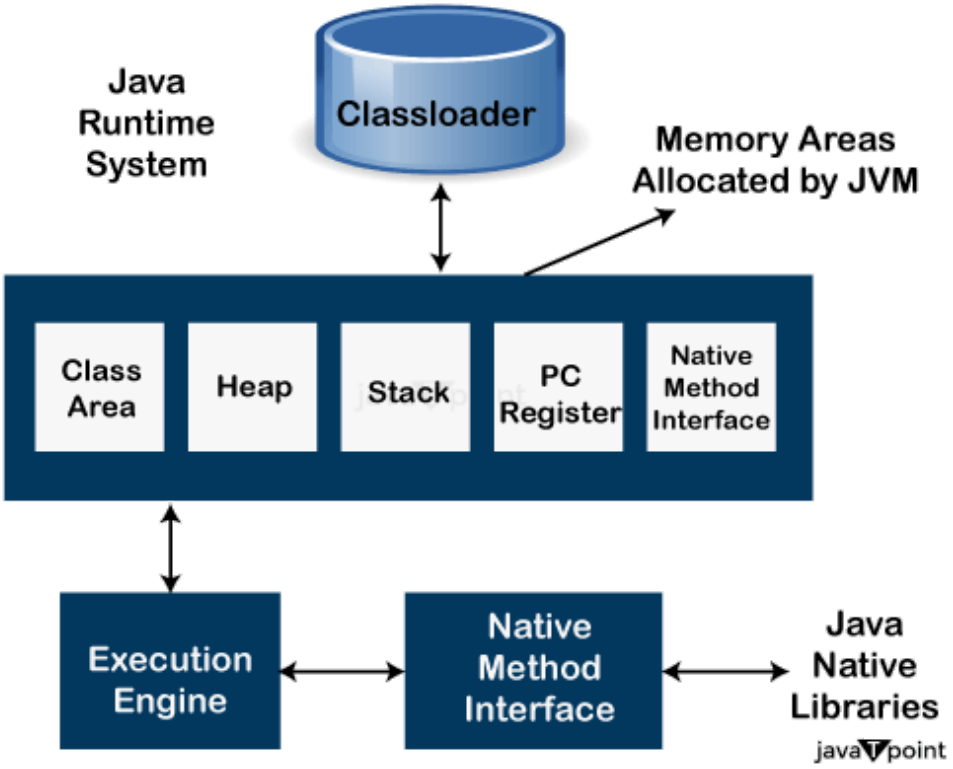
To read more: Features of Java



4) What do you understand by Java Virtual Machine (JVM)?

Java Virtual Machine is a virtual machine that enables a computer to run Java programs. It is a virtualized execution environment that allows Java programs to run on any device with a compatible JVM. JVM is the specification that must be implemented in the computer system. It interprets Java bytecode and translates it into machine code for the underlying hardware.

To read more: Java Virtual Machine (JVM)



5) What are the differences between JDK, JRE, and JVM?

Feature	JDK	JRE	JVM
Definition	Java Development Kit (JDK) is a software development kit used to develop Java applications.	Java Runtime Environment (JRE) is a software package that provides Java Virtual Machine (JVM), class libraries and other components to run applications in Java.	Java Virtual Machine (JVM) is an abstract machine that provides an environment for the execution of Java ByteCodes.
Functions	Provides tools for developing Java applications. It includes compiler (javac), JRE, and debugging tools.	It provides an environment for executing Java applications. It includes JVM and libraries required to run Java programs, but does not include development tools.	The virtual machine that runs the bytecode generated by the compiler. It's a part of JRE and is platform-dependent but provides a platform-independent execution environment.
Use Cases	Used by developers to write, compile, and debug Java programs.	Used by end-users to execute Java applications without needing development tools.	Converts bytecode into machine code and executes it on the underlying hardware.
Platform Dependency	It is platform dependent.	It is platform dependent.	It is platform dependent.
Implementation	JDK = JRE + Development tools	JRE = JVM + Class libraries	JVM = provides a runtime environment.

To read more: Difference between JDK, JRE, and JVM

6) How many types of memory areas are allocated by the JVM?

Class (Method) Area: Class Area stores per-class structures such as the runtime constant pool, field, method data, and the code for methods.

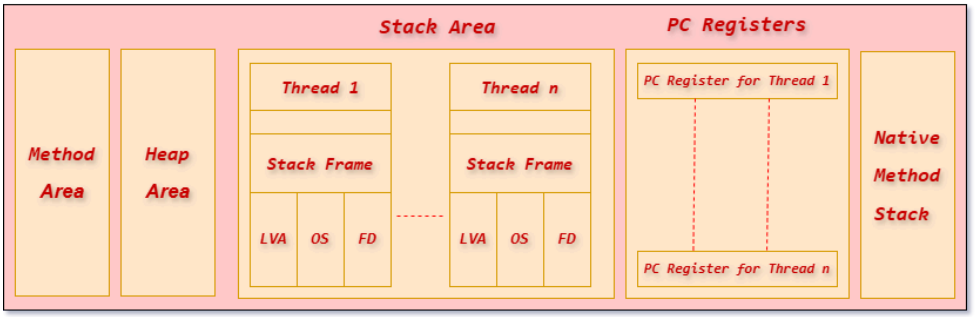
Heap: It is the runtime data area in which memory is allocated to the objects

Stack: Java Stack stores frames. Holds local variables and partial results, supporting method execution. Each thread has a private JVM stack, created at the same time as the thread. A new frame is created each time a method is invoked. A frame is

destroyed when its method invocation completes.

PC Register (Non-Heap): Keeps track of the current execution position for each thread.

Native Method Stack (Non-Heap): It contains native method information and supports native code execution.



JVM Memory Struture

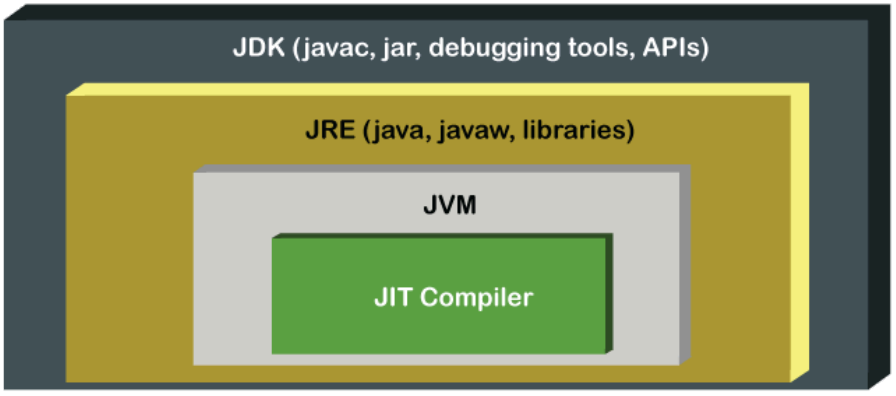
To read more: Memory Management in Java

7) What is JIT compiler?

Just-In-Time (JIT) Compiler: It is used to improve the performance. The compiler is part of the Java Runtime Environment (JRE).

It translates Java bytecode into native machine code at runtime. Therefore, reduces the amount of time needed for compilation.

Here, the term "compiler" refers to a translator from the instruction set of a Java Virtual Machine (JVM) to the instruction set of a specific CPU.



To read more: JIT in Java

8) What is the platform?

A platform refers to the combination of hardware and software on which programs run. In the context of Java, the platform includes the operating system and the Java Virtual Machine (JVM).

There are two types of platforms:

1. Software-Based
2. Hardware-Based

9) What are the main differences between the Java platform and other platforms?

There are the following differences between the Java platform and other platforms.

1. Java is platform-independent due to its "Write Once, Run Anywhere" nature.

2. Other platforms may require platform-specific code and compilation for each target environment.

10) What gives Java its 'write once and run anywhere' nature?

Java compiler converts the Java programs into a class file (bytecode), which is the intermediate language between source code and machine code. Code is compiled to bytecode, which is platform-independent, and the JVM executes this bytecode on any device with a compatible JVM.

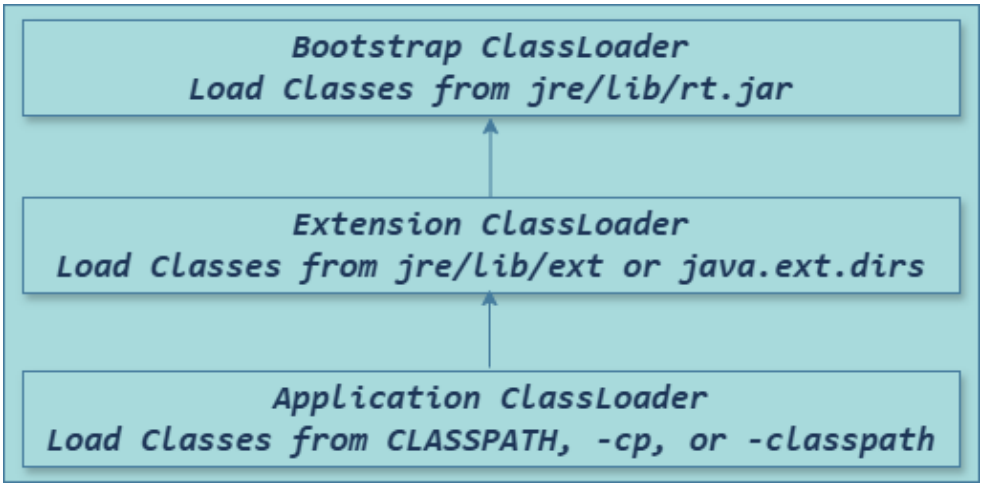
To read more: Java Compiler

11) What is a ClassLoader?

Classloader is a subsystem of the JVM that is used to load class files. It loads Java classes into memory during runtime. Responsible for finding and loading class files needed by a Java program.

There are three built-in classloaders in Java.

1. **Bootstrap ClassLoader:**This is the first classloader, which is the superclass of Extension classloader. It loads the *jar* file, which contains all class files of Java Standard Edition, like java.lang package classes, java.net package classes, java.util package classes, java.io package classes, java.sql package classes, etc.
2. **Extension ClassLoader:**This is the child classloader of Bootstrap and the parent classloader of the System classloader. It loads the jar files located inside *$JAVA_HOME/jre/lib/ext*
3. **System/Application ClassLoader:**This is the child classloader of the Extension classloader. It loads the class files from the classpath. By default, the classpath is set to the current directory. You can change the classpath using the "-cp" or "-classpath" switch. It is also known as the Application classloader.



To read more: ClassLoader in Java

12) Is an empty .java file name a valid source file name?

Corrected

> **CORRECTED** The source's text and its commands disagreed. The file really can be named just `.java`; its example then used `javac Main.java`.

Yes. A file whose name is only the extension, `.java`, is a valid source file name as long as it has no `public` class (a public class must live in a file named after it).

```
// save as ".java"  (empty file name)
class Main {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Compile with `javac .java`. That creates `Main.class` (the class name decides the output file), and you run it with `java Main`. Confirmed on JDK 21.

13) Is delete, next, main, exit, or null keyword in Java?

No, in Java, delete, next, main, exit, and null are not keywords.

14) If I do not provide any arguments on the command line, then what will the value stored in the String array passed into the main() method, empty or NULL?

It is empty, but not null. If we do not provide any command-line arguments when running a Java program, the String array passed into the main() method will be an empty array, not null. The args parameter of the main() method is an array of strings, and it is always initialized by Java, even if no command-line arguments are provided.

15) What if we write static public void instead of public static void?

The program compiles and runs correctly because the order of specifiers does not matter in Java.

In Java, the placement of access modifiers (public, private, protected) and other specifiers (static, final, etc.) before the return type (void in this case) is flexible. Both public static void and static public void are accepted and considered correct syntax.

Here are the two equivalent valid declarations for the main() method:

```
public static void main(String[] args) {
    // Code goes here
}
static public void main(String[] args) {
    // Code goes here
}
```

To read more Java main() Method

16) What is the default value of the local variables?

In Java, local variables are not assigned default values by the compiler. They must be explicitly initialized before they are used, or a compilation error will occur.

```
public class Main {
    public static void main(String[] args) {
        int x;
        System.out.println(x); // Compilation error: The local variable x may not have been initialized
    }
}
```

To read more: Java Variables

17) What are the various access specifiers in Java?

In Java, access specifiers are the keywords that are used to define the access scope of a method, class, or variable. There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.

Access Specifiers

public: public classes, methods, or variables that are defined as public can be accessed by any class or method.

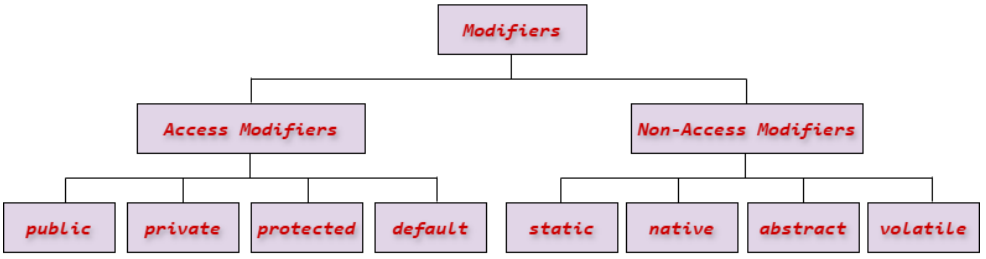
protected: The protected classes, methods, or variables can be accessed by the class of the same package, or by a subclass of this class, or within the same class.

private: The private class, methods, or variables defined as private can be accessed within the class only.

default (package-private): If no access specifier is used, it is considered package-private. Members with default access are accessible only within the same package.

Non-Access Specifier

It includes static, synchronized, abstract, native, volatile, transient.



Java Access Modifiers

To read more: Java Access Specifiers

18) What is the purpose of static methods and variables?

The methods or variables defined as static are shared among all the objects of the class. Static is the part of the class and not of the object. Static methods and variables in Java are associated with the class rather than with an instance of the class. Therefore, static is used in the case where we need to define variables or methods that are common to all the objects of the class.

For example, in the class simulating the collection of the students in a college, the name of the college is a common attribute to all the students. Therefore, the college name will be defined as static.

Class-level Access: Static members belong to the class itself rather than to instances of the class. It means they can be accessed using the class name without creating an instance of the class.

Constants: Static variables are commonly used to define constants in a class. These values are shared among all instances of the class and can be accessed using the class name.

To read more: Static Method in Java

19) What are the advantages of packages in Java?

Packages in Java provide a way to organize and structure classes and interfaces into a hierarchical structure. They offer several advantages:

- Packages avoid the name clashes.
- The Package provides easier access control.
- We can also have the hidden classes that are not visible outside and used by the package.
- It is easier to locate the related classes.

To read more: Advantages of Packages in Java

20) What is the output of the following Java program?

Example

```
class Main
{
        public static void main (String args[])
        {
                System.out.println(10 + 20 + "Tpointtech");
                System.out.println("Tpointtech" + 10 + 20);
        }
}
```

Output:

```
30Tpointtech
Tpointtech1020
```

Explanation

In the first case, 10 and 20 are treated as numbers and added to be 30. Now, their sum 30 is treated as a string and concatenated with the string **Tpointtech**. Therefore, the output will be **30Tpointtech**.

In the second case, the string Tpointtech is concatenated with 10 to be the string **Tpointtech10**, which will then be concatenated with 20 to be **Tpointtech1020**.

To read more: Java Programs

21) What is the output of the following Java program?

Example

```
class Main
{
        public static void main (String args[])
        {
                System.out.println(10 * 20 + "Tpointtech");
                System.out.println("Tpointtech" + 10 * 20);
        }
}
```

Output:

```
200Tpointtech
Tpointtech200
```

Explanation

In the first case, the numbers 10 and 20 will be multiplied first, and then the result 200 is treated as a string and concatenated with the string **Tpointtech** to produce the output **200Tpointtech**.

In the second case, the numbers 10 and 20 will be multiplied first to be 200 because the precedence of multiplication is higher than addition. The result 200 will be treated as a string and concatenated with the string Tpointtech to produce the output as Tpointtech200.

22) What is the output of the following Java program?

```
class Test
{
        public static void main (String args[])
        {
                for(int i=0; 0; i++)
                {
                        System.out.println("Hello Tpointtech");
                }
        }
}
```

The above code will result in a compilation error due to the non-boolean expression in the second part of the for loop, and given is an integer value, i.e., 0.

Core Java - OOP Concepts: Initial OOP Interview Questions

Questions in this topic (4)

23) What is an object-oriented paradigm?
24) What is an object?
25) What is the difference between an object-oriented programming language and an object-based programming language?
26) What will be the initial value of an object reference that is defined as an instance variable?

23) What is an object-oriented paradigm?

It is a programming paradigm based on objects having data and methods defined in the class to which it belongs. Key principles of the object-oriented paradigm include encapsulation, inheritance, and polymorphism, providing a framework for designing and structuring software in a way that models real-world entities and promotes code reusability and maintainability. Objects are the instances of classes that interact with one another to design applications and programs.

There are the following features of the object-oriented paradigm.

- OOPs follows the bottom-up approach in program design.
- Focus on data with methods to operate upon the object's data
- Includes concepts like Encapsulation and abstraction, which hide the complexities from the user and show only functionality.
- Implements the real-time approach like inheritance, abstraction, etc.

The examples of the object-oriented paradigm are C++, Simula, Smalltalk, Python, C#, etc.

To read more: OOPs (Object-Oriented Programming) Concepts in Java

24) What is an object?

The Object is a real-time entity having some state and behavior. In Java, an Object is an instance of a class that has instance variables as the state of the object and methods as the behavior of the object. The object of a class can be created by using the new keyword.

For example, if we have a class called Car, an object of that class could represent a specific car with attributes like color, model, year, and methods like startEngine() or drive(). Each instance of the Car class would be a distinct object with its own unique state.

To read more: Java Classes & Objects

25) What is the difference between an object-oriented programming language and an object-based programming language?

There are the following basic differences between object-oriented languages and object-based languages.

Object-oriented languages follow all the concepts of OOP, whereas object-based languages do not follow all the concepts of OOP, like inheritance and polymorphism. It allows the creation of classes and objects, and it supports the defining and implementation of methods within those classes.

Object-oriented languages do not have the inbuilt objects, whereas Object-based languages have the inbuilt objects; for example, JavaScript has the window object. It allows the creation of objects and encapsulation of data, but it might not provide the complete set of features associated with full OOP.

Examples of object-oriented programming are Java, C#, Smalltalk, etc., whereas the examples of object-based languages are JavaScript, VBScript, etc.

26) What will be the initial value of an object reference that is defined as an instance variable?

In Java, when an object reference is declared as an instance variable (a member variable of a class). It is automatically initialized to null by default. It is true for all object references, regardless of the class type.

Core Java - OOPs Concepts: Constructor Interview Questions

Questions in this topic (12)

27) What is the constructor?
28) How many types of constructors are used in Java?
29) What is the purpose of a default constructor?
30) Does the constructor return any value?
31) Is the constructor inherited?
32) Can you make a constructor final?
33) Can we overload the constructors?
34) What do you understand by copy constructor in Java?
35) What are the differences between the constructors and methods?
36) What is the output of the following Java program?
37) What is the output of the following Java program?
38) What is the output of the following Java program?

27) What is the constructor?

The constructor can be defined as a special type of method that is used to initialize the state of an object. It is invoked when the class is instantiated, and the memory is allocated for the object. Every time an object is created using the new keyword, the default constructor of the class is called. The name of the constructor must be similar to the class name. The constructor must not have an explicit return type.

To read more: Constructors in Java

28) How many types of constructors are used in Java?

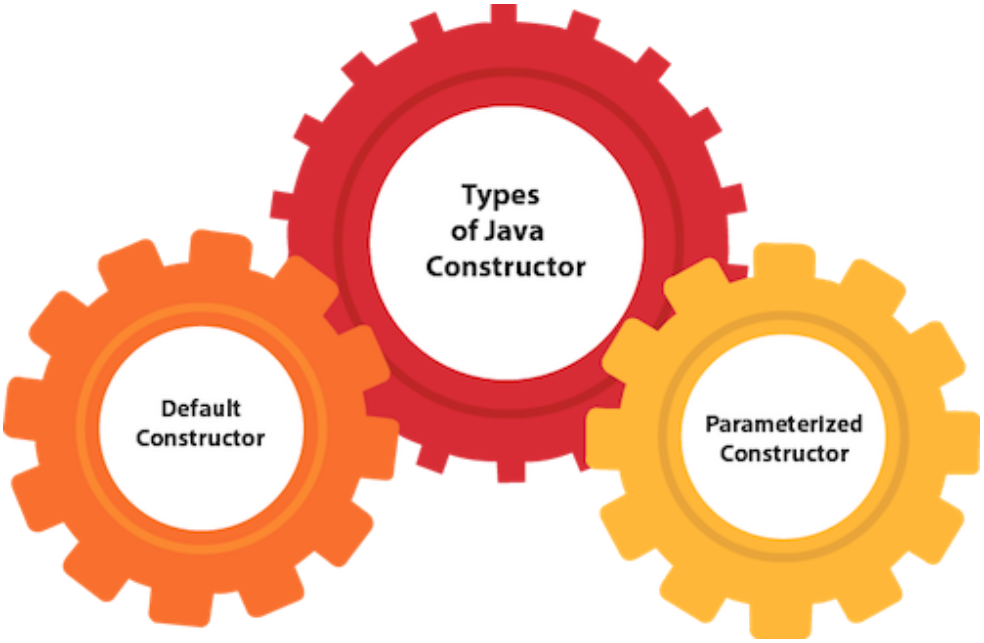
Based on the parameters passed in the constructors, there are two types of constructors in Java.

Default Constructor: The default constructor is the one that does not accept any value. The default constructor is mainly used to initialize the instance variable with the default values. The default constructor initializes instance variables to their default values (numeric types to 0, booleans to false, and object references to null). A default constructor is invoked implicitly by the compiler if there is no constructor defined in the class.

```
public class Main {
    // Default constructor (automatically provided if not defined explicitly)
}
```

Parameterized Constructor: The parameterized constructor is the one that can initialize the instance variables with the given values. It allows us to initialize the object with specific values during the object creation process. In other words, we can say that the constructors that can accept the arguments are called parameterized constructors.

```
public class Main {
    // Parameterized constructor
    public Example(int value) {
        // Initialization logic here
    }
}
```



29) What is the purpose of a default constructor?

The default constructor in Java is a constructor with no parameters. The purpose of the default constructor is to assign the default value to the objects. The Java compiler automatically provides it if a class does not have any explicit constructor defined.

Example

```
class Main {
int id;
String name;
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
Main m1=new Main ();
Main m2=new Main();
m1.display();
m2.display();
}
}
```

Output:

```
0 null
0 null
```

Explanation: In the above class, you are not creating any constructor, so the compiler provides you a default constructor. Here, 0 and null values are provided by the default constructor.



30) Does the constructor return any value?

Yes, in Java, a constructor does not have a return type, not even void. The purpose of a constructor is to initialize the object's state and prepare it for use. The constructor implicitly returns the current instance of the class (we cannot use an explicit return type with the constructor).

31) Is the constructor inherited?

No, Constructors are not inherited. The superclass constructor can be called from the first line of a subclass constructor by using the keyword super and passing appropriate parameters to set the private instance variables of the superclass.

32) Can you make a constructor final?

No, in Java, constructors cannot be declared as final. The final keyword is used to indicate that a class, method, or variable cannot be further subclassed, overridden, or modified. However, it does not apply to constructors.

33) Can we overload the constructors?

Yes, the constructors can be overloaded by changing the number of arguments accepted by the constructor or by changing the data type of the parameters.

Each constructor provides a different way to initialize an object of the class. The choice of which constructor to invoke is based on the arguments passed during object creation.

Consider the following example.

Example

```
class Test
{
        int i;
        public Test(int k)
        {
            i=k;
        }
        public Test(int k, int m)
        {
            System.out.println("Hi I am assigning the value max(k, m) to i");
            if(k>m)
            {
                i=k;
            }
            else
            {
                i=m;
            }
        }
}
public class Main
{
        public static void main (String args[])
        {
                Test test1 = new Test(10);
                Test test2 = new Test(12, 15);
                System.out.println(test1.i);
                System.out.println(test2.i);
        }
}
```

Output:

```
Hi I am assigning the value max(k, m) to i
10
15
```

In the above program, **the constructor** Test is overloaded with another constructor. In the first call to the constructor, the constructor with one argument is called, and i will be initialized with the value 10. However, in the second call to the constructor, the constructor with the 2 arguments is called, and i will be initialized with the value 15.

34) What do you understand by copy constructor in Java?

There is no copy constructor in Java. However, we can copy the values from one object to another, like a copy constructor in C++.

There are many ways to copy the values of one object into another in Java. They are:

- By using a constructor
- By assigning the values of one object to another
- By using Object.clone() Method

In this example, we are going to copy the values of one object into another using a Java constructor.

Example

```
public class Main {
    int id;
    String name;
    // Constructor to initialize integer and string
    Main(int id, String name) {
        this.id = id;
        this.name = name;
    }
    // Copy constructor
    Main(Main s) {
        this.id = s.id;
        this.name = s.name;
    }
    void display() {
        System.out.println(id + " " + name);
    }
    public static void main(String[] args) {
        Main obj1 = new Main(111, "Karan");
        Main obj2 = new Main(obj1);
        obj1.display();
        obj2.display();
    }
}
```

Output:

```
111 Karan
111 Karan
```

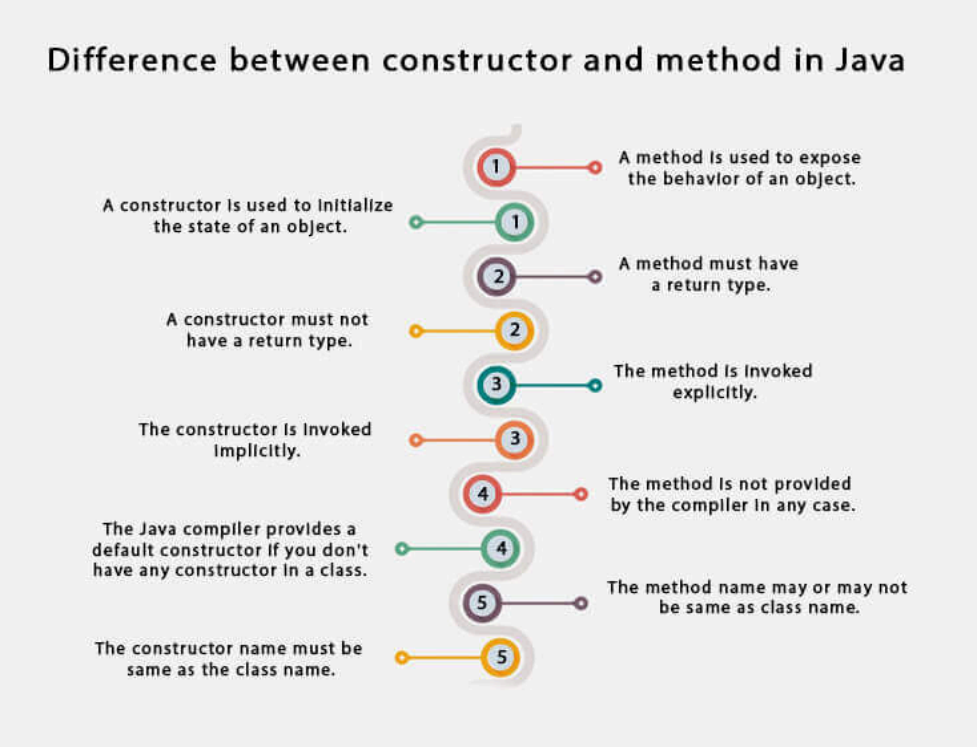
Copy constructors can be useful in scenarios where you need to duplicate the state of an existing object without directly referencing the same data. They are not built into the Java language like some other languages, but we can define them in our classes as needed.

To read more: Copy Constructor in Java

35) What are the differences between the constructors and methods?

There are many differences between constructors and methods.

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if we do not have any constructor in a class.	The compiler does not provide the method in any case.
The constructor's name must be the same as the class name.	The method name may or may not be the same as the class name.



To read more: [Difference Between Constructor and Method in Java]

36) What is the output of the following Java program?

Example

```
public class Main
{
    Main(int a, int b)
    {
        System.out.println("a = "+a+" b = "+b);
    }
    Main(int a, float b)
    {
        System.out.println("a = "+a+" b = "+b);
    }
    public static void main (String args[])
    {
        byte a = 10;
        byte b = 15;
        Main obj = new Main(a,b);
    }
}
```

Output:

```
a = 10 b = 15
```

Here, the data type of the variables a and b, i.e., byte, gets promoted to int, and the first parameterized constructor with the two integer parameters is called.

37) What is the output of the following Java program?

Example

```
class Test
{
        int i;
}
public class Main
{
        public static void main (String args[])
        {
                Test test = new Test();
                System.out.println(test.i);
        }
}
```

Output:

```
0
```

The output of the program is 0 because the variable i is initialized to 0 internally. As we know that a default constructor is invoked implicitly if there is no constructor in the class, the variable i is initialized to 0 since there is no constructor in the class.

38) What is the output of the following Java program?

```
class Test
{
        int test_a, test_b;
        Test(int a, int b)
        {
        test_a = a;
        test_b = b;
        }
        public static void main (String args[])
        {
                Test test = new Test();
                System.out.println(test.test_a+" "+test.test_b);
        }
}
```

Output:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        The constructor Test() is undefined
```

There is a compiler error in the program because there is a call to the default constructor in the main method that is not present in the class. However, there is only one parameterized constructor in the class Test. Therefore, no default constructor is invoked by the constructor implicitly.

Core Java - OOPs Concepts: static Keyword Interview Questions

Questions in this topic (12)

39) What is the static variable?
40) What is the static method?
41) What are the restrictions that are applied to the Java static methods?
42) Why the main() method is static?
43) Can we override the static methods?
44) What is the static block?
45) Can we execute a program without a main() method?
46) What if the static modifier is removed from the signature of the main method?
47) What is the difference between a static (class) method and an instance method?
48) Can we make constructors static?
49) Can we make the abstract methods static in Java?
50) Can we declare the static variables and methods in an abstract class?

39) What is the static variable?

The static variable is used to refer to the common property of all objects (that is not unique for each object), for example, the company name of employees, the college name of students, etc. Static variable gets memory only once in the class area at the time of class loading. Using a static variable makes your program more memory efficient (it saves memory). A static variable belongs to the class rather than the object. The following Java program depicts the use of a static variable.

Example

```
public class Main {
    int rollno;
    String name;
    static final String COLLEGE = "ITS";
    Main(int r, String n) {
        rollno = r;
        name = n;
    }
    void display() {
        System.out.println(rollno + " " + name + " " + COLLEGE);
    }
    public static void main(String[] args) {
        Main obj1 = new Main (111, "Karan");
        Main obj2 = new Main(222, "Aryan");
        obj1.display();
        obj2.display();
    }
}
```

Output:

```
111 Karan ITS
222 Aryan ITS
```

40) What is the static method?

In Java, a static variable is a class-level variable that belongs to the class rather than to instances of the class. It is shared among all instances of the class and is initialized only once when the class is loaded into memory.

- A static method belongs to the class rather than the object.
- There is no need to create the object to call the static methods.
- A static method can access and change the value of the static variable.

Static Keyword in Java

41) What are the restrictions that are applied to the Java static methods?

Java static methods have certain restrictions and characteristics that differentiate them from instance methods. Here are the key restrictions and considerations for Java static methods:

The two main restrictions applied to the static methods are as follows:

- The static method cannot use a non-static data member or call a non-static method directly.
- This and super keyword cannot be used in a static context as they are non-static.
- Static methods in Java cannot be overridden. Even if a subclass declares a static method with the same signature as a static method in its superclass, it is considered method hiding, not overriding.

To read more: Static Method in Java

42) Why the main() method is static?

Because the object is not required to call the static method, if we make the main() method non-static, the JVM will have to create its object first and then call the main() method, which will lead to extra memory allocation.

Here are the main reasons:

The main() method is the entry point for the execution of a Java program. When we run a Java program, the Java Virtual Machine (JVM) looks for the main() method with the following signature:

```
public static void main(String[] args)
```

When a Java program is executed, the JVM loads the class into memory and looks for the main() method to start the program.

To read more: Why main() method is always static in Java

43) Can we override the static methods?

No, static methods cannot be overridden in Java. While a subclass can declare a static method with the same signature as a static method in its superclass, it is considered method hiding, not method overriding.

44) What is the static block?

A static block is used to initialize the static data member. It is executed before the execution of the main() method, at the time of class loading.

Example

```
class Main {
static { System.out.println("static block is invoked"); }
public static void main(String args[]) {
System.out.println("Hello main");
}
}
```

Output:

```
static block is invoked
       Hello main
```

To read more: Static Block in Java

45) Can we execute a program without a main() method?

No, in Java, we cannot execute a program without the main() method. The main() method serves as the entry point for the execution of a Java program, and it must be defined in every Java class that serves as the starting point for an application. The JVM looks for the main() method with the following signature to begin program execution:

```
public static void main(String[] args)
```

To read more: Java main() Method

46) What if the static modifier is removed from the signature of the main method?

Corrected

CORRECTED The source said the program 'throws NoSuchMethodError'. Up to Java 24 the launcher prints a different message, and Java 25 changed the rule.

Checked against: JEP 512.

The program **compiles**. What happens when you run it depends on the Java version:

- **Java 24 and earlier:** the launcher refuses with `Error: Main method is not static in class X, please define the main method as: public static void main(String[] args)` (confirmed on JDK 21).
- **Java 21 to 24:** instance `main` methods are a preview feature (`--enable-preview`); with it the program runs.
- **Java 25 and later (JEP 512):** `main` may be non-static, and the `public` modifier and the `String[]` parameter are optional. The launcher creates an instance of the class and calls `main` .

The classic interview answer is still: the JVM wants a `static` entry point so it can start the program without creating an object.

47) What is the difference between a static (class) method and an instance method?

A method that is declared as static is known as a static method.	A method that is not declared as static is known as an instance method.
We do not need to create the objects to call the static methods.	The object is required to call the instance methods.
Non-static (instance) members cannot be accessed in the static context (static method, static block, and static nested class) directly.	Static and non-static variables can both be accessed in instance methods.
For example: public static int cube(int n){ return n*n*n;}	For example: public void msg(){...}.

To read more: Difference Between Static and Instance Methods in Java

48) Can we make constructors static?

No, constructors cannot be declared as static in Java, as we know that the static context (method, block, or variable) belongs to the class, not the object. Since Constructors are invoked only when the object is created, there is no sense in making the constructors static. Making a constructor static would imply that it belongs to the class and not to instances, which contradicts the fundamental purpose of a constructor. However, if we try to do so, the compiler will show the compiler error..

49) Can we make the abstract methods static in Java?

No, abstract methods cannot be declared as static in Java. In Java, if we make the abstract methods static, they will become part of the class, and we can directly call them, which is unnecessary. Since abstract methods are expected to be overridden by subclasses, making them static would be contradictory to their purpose. Calling an undefined method is completely useless; therefore, it is not allowed.

50) Can we declare the static variables and methods in an abstract class?

Yes, it is possible to declare static variables and methods in an abstract class in Java. As we know that there is no requirement to make the object access the static context; therefore, we can access the static context declared inside the abstract class by using the name of the abstract class.

Consider the following example.

Example

```
abstract class Test
{
    static int i = 102;
    static void display()
    {
        System.out.println("hi !! I am good !!");
    }
}
public class Main extends Test
{
    public static void main (String args[])
    {
        Test.display();
        System.out.println("i = "+Test.i);
    }
}
```

Output

```
hi !! I am good !!
i = 102
```

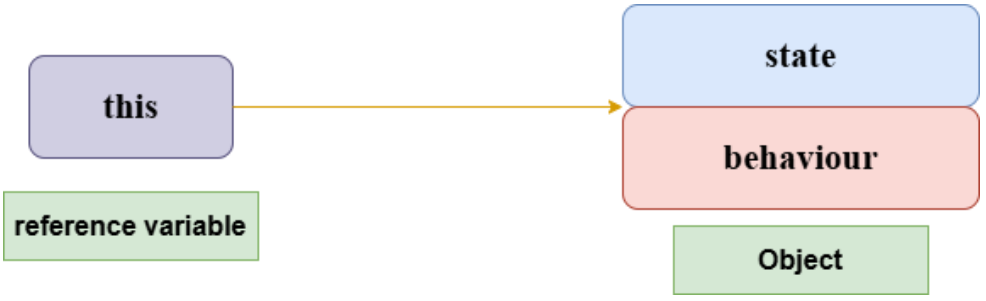
Core Java - OOPs Concepts: Inheritance Interview Questions

Questions in this topic (21)

51) What is this keyword in Java?
52) What are the uses of this keyword?
53) Can we assign the reference to this variable?
54) Can this keyword be used to refer to static members?
55) How can constructor chaining be done using this keyword?
56) What are the advantages of passing this into a method instead of the current class object itself?
57) What is the Inheritance?
58) Why is inheritance used in Java?
59) Which class is the superclass for all the classes?
60) Why is multiple inheritance not supported in Java?
61) What is aggregation?
62) What is composition?
63) What is the difference between aggregation and composition?
64) Why does Java not support pointers?
65) What is super in Java?
66) How can constructor chaining be done by using the super keyword?
67) What are the main uses of the super keyword?
68) What are the differences between this and super keyword?
69) What is the output of the following Java program?
70) Can we use this() and super() in a constructor?
71) What is object cloning?

51) What is this keyword in Java?

In Java, this keyword is a reference variable that refers to the current object. There are various uses of this keyword in Java. It is primarily used within the instance methods of a class to refer to the object on which the method is invoked. It can be used to refer to current class properties such as instance methods, variables, constructors, etc. It can also be passed as an argument to the methods or constructors. It can also be returned from the method as the current class instance.



To read more: this keyword in Java

52) What are the uses of this keyword?

There are the following uses of this keyword:

- It can be used to refer to the current class instance variable.
- It can be used to invoke the current class method (implicitly).
- The constructor this() can be used to invoke the current class constructor.
- It can be passed as an argument in the method call.
- It can be passed as an argument in the constructor call.
- It can be used to return the current class instance from the method.

Let's understand through a Java program.

```
public class Main {
    private int x;
    public void setX(int x) {
        this.x = x; // Using 'this' to refer to the instance variable
    }
}
```

To read more this Keyword in Java

53) Can we assign the reference to this variable?

No, it is not possible to assign a new value to this variable in Java. this keyword is an implicit reference to the current instance of the class and is automatically set by the Java runtime system when a method is invoked. However, if we try to do so, the compiler error will be shown. Consider the following example.

Example

```
public class Main
{
    public Main()
    {
        this = null;
        System.out.println("Main class constructor called");
    }
    public static void main (String args[])
    {
        Main obj = new Main();
    }
}
```

Output

```
10
```

54) Can this keyword be used to refer to static members?

Yes, it is possible to use this keyword to refer to static members because it is just a reference variable that refers to the current class object. However, as we know, it is unnecessary to access static variables through objects. Therefore, it is not the best practice to use this to refer to static members. Consider the following example.

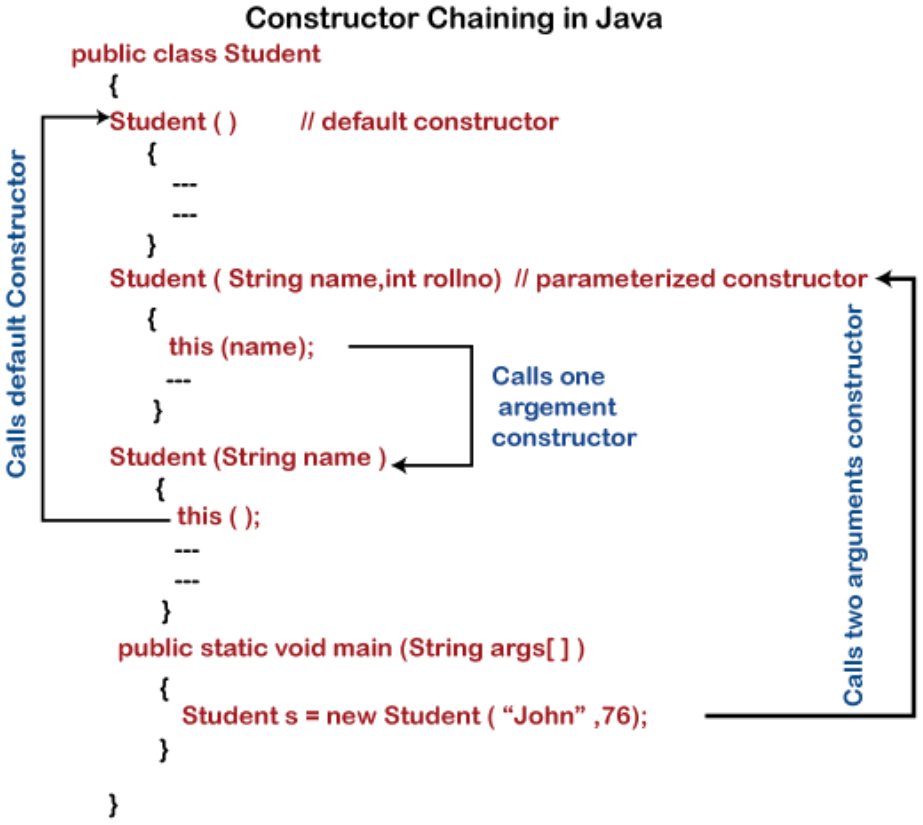
```
public class Main
{
    static int i = 10;
    public Main()
    {
        System.out.println(this.i);
    }
    public static void main (String args[])
    {
        Main obj = new Main();
    }
}
```

Output

```
10
```

55) How can constructor chaining be done using this keyword?

Constructor chaining enables us to call one constructor from another constructor of the class with respect to the current class object. Constructor chaining in Java can be achieved using the this keyword. Consider the following example that illustrates how we can use this keyword to achieve constructor chaining.



```
class Employee
{
    int id, age;
    String name, address;

    public Employee(int age)
    {
        this.age = age;
    }

    public Employee(int id, int age)
    {
        this(age);
        this.id = id;
    }

    public Employee(int id, int age, String name, String address)
    {
        this(id, age);
        this.name = name;
        this.address = address;
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Employee emp = new Employee(105, 22, "Peter", "Delhi");

        System.out.println("ID: " + emp.id +
                           " Name: " + emp.name +
                           " Age: " + emp.age +
                           " Address: " + emp.address);
    }
}
```

Output

```
ID: 105 Name: Peter age: 22 address: Delhi
```

To read more: Constructor Chaining in Java

56) What are the advantages of passing this into a method instead of the current class object itself?

As we know, this refers to the current class object; therefore, it must be similar to the current class object. However, there can be two main advantages of passing this into a method instead of the current class object.

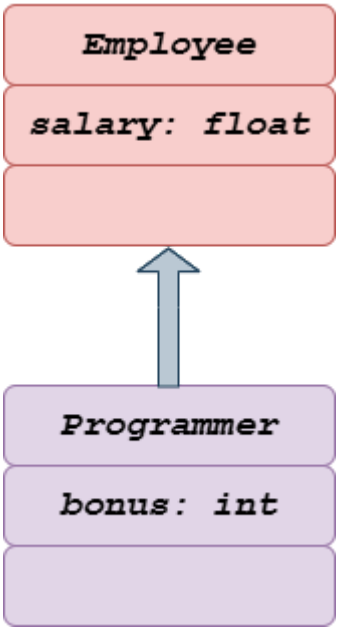
- this is a final variable. Therefore, this cannot be assigned to any new value, whereas the current class object might not be final and can be changed.
- this keyword can be used in the synchronized block.

Passing this into a method instead of the current class object itself provides several advantages:

- Flexibility
- Encapsulation
- Consistency

57) What is the Inheritance?

Inheritance is a mechanism by which one object acquires all the properties and behavior of another object of another class. It is used for Code Reusability and Method Overriding. Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (subclass or derived class) to inherit attributes and behaviors from another class (superclass or base class). Moreover, we can add new methods and fields in your current class also. Inheritance represents the IS-A relationship that is also known as a parent-child relationship.



To read more Inheritance in Java

There are five types of inheritance in Java:

- Single-level inheritance
- Multi-level inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Note: Multiple inheritance is not supported in Java through a class.

To read more: Types of Inheritance in Java

58) Why is inheritance used in Java?

Various advantages of using inheritance in Java are given below.

- Inheritance provides code reusability. Inheritance allows subclasses to inherit attributes and behaviors from a superclass, promoting code reuse. Common functionality can be defined in a superclass, and subclasses can extend or specialize it.
- Runtime polymorphism cannot be achieved without using inheritance.
- We can simulate the inheritance of classes with real-time objects, which makes OOPs more realistic.
- Inheritance provides data hiding. The base class can hide some data from the derived class by making it private.
- Method overriding cannot be achieved without inheritance. Inheritance facilitates polymorphism, allowing objects of a subclass to be treated as objects of the superclass. By method overriding, we can give a specific implementation of some basic method contained by the base class.

To read more: Inheritance in Java

59) Which class is the superclass for all the classes?

In Java, the Object class is the superclass of all other classes. Every class in Java directly or indirectly extends the Object class. The Object class provides fundamental methods that are inherited by all classes, such as toString(), equals().

To read more: Object Class in Java

60) Why is multiple inheritance not supported in Java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in Java. Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and we call it from a child class object, there will be ambiguity in calling the method of A or B class.

The Diamond Problem occurs when a class inherits from two classes that have a common ancestor, leading to ambiguity in method resolution.

Since the compile-time errors are better than runtime errors, Java renders a compile-time error if we inherit 2 classes. So, whether we have the same method or a different one, there will be a compile-time error.

```
class A{
void msg(){System.out.println("Hello");}
}
class B{
void msg(){System.out.println("Welcome");}
}
class Main extends A,B{//suppose if it were
 public Static void main(String args[]){
   Main obj=new Main();
   obj.msg();//Now which msg() method would be invoked?
}
}
```

The above program gives a compile-time error.

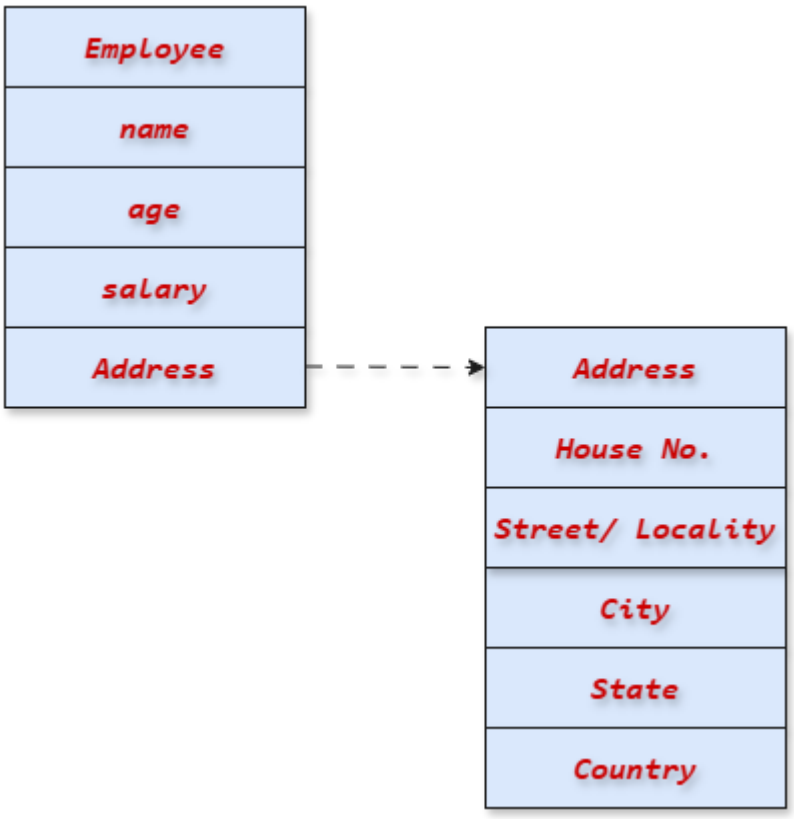
To read more: Multiple Inheritance in Java

61) What is aggregation?

Aggregation can be defined as the relationship between two classes where the aggregate class contains a reference to the class it owns.

Aggregation is a type of association in object-oriented programming where one class contains an object of another class, forming a relationship between them. It represents a "has-a" relationship, indicating that a class has an entity as a part of its structure.

For example, the aggregate class Employee, having various fields such as age, name, and salary, also contains an object of the Address class, having various fields such as Address-Line 1, City, State, and pin-code.



In other words, we can say that Employee (class) has an object of Address class. Consider the following example.

Address.java

```
public class Address {
String city, state, country;
public Address(String city, String state, String country) {
    this.city = city;
    this.state = state;
    this.country = country;
}
}
```

Employee.java

```
public class Emp {
int id;
String name;
Address address;
public Emp(int id, String name,Address address) {
    this.id = id;
    this.name = name;
    this.address=address;
}
void display(){
System.out.println(id+" "+name);
System.out.println(address.city+" "+address.state+" "+address.country);
}
public static void main(String[] args) {
Address address1 = new Address("gzb", "UP", "India");
Address address2 = new Address("gno", "UP", "India");
Emp e=new Emp(111,"varun",address1);
Emp e2=new Emp(112,"arun",address2);
e.display();
e2.display();
}
}
```

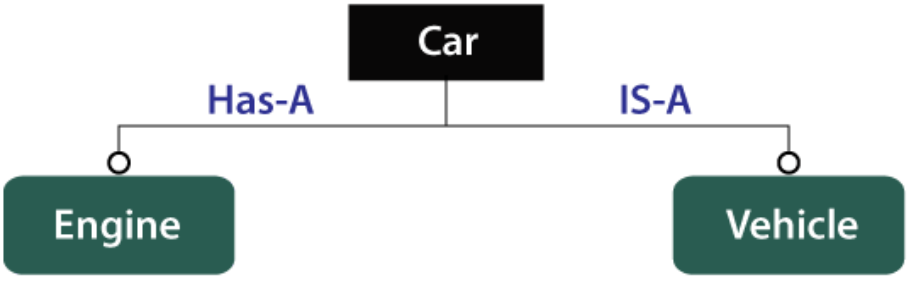
Output

```
111 varun
gzb UP India
112 arun
gno UP India
```

To read more: Aggregation in Java

62) What is composition?

Composition is a stronger form of aggregation in object-oriented programming. Holding the reference of a class within another class is known as composition. When an object contains another object, if the contained object cannot exist without the existence of the container object, then it is called composition. In other words, we can say that composition is a particular case of aggregation, which represents a stronger relationship between two objects. Example: A class contains students. A student cannot exist without a class. There exists a composition between the class and the students.



Composition is a way to create more robust and tightly integrated structures by combining objects to form a larger, more complex system.

To read more: Composition in Java

63) What is the difference between aggregation and composition?

Aggregation represents a weak relationship, whereas composition represents a strong relationship. Aggregation represents a "has-a" relationship, where one class has a reference to another class, but the referenced class can exist independently. Composition also represents a "has-a" relationship, but it is a stronger form of association. For example, the bike has an indicator (aggregation), but the bike has an engine (composition).

To read more: Difference Between Aggregation and Composition in Java

64) Why does Java not support pointers?

Java does not support explicit pointers like those found in languages such as C or C++, as the pointer is a variable that refers to the memory address. They are not used in Java because they are unsafe(unsecured) and complex to understand.

65) What is super in Java?

In Java, super is a keyword that is used to refer to the immediate parent class object. Whenever you create an instance of the subclass, an instance of the parent class is created implicitly, which is referred to by the super reference variable. The super() is called in the class constructor implicitly by the compiler if there is no super or this.

The super keyword is commonly used in scenarios where a class extends another class, and there is a need to differentiate between the members of the subclass and the superclass.

```
class Animal {
Animal() { System.out.println("An animal is created"); }
}
class Dog extends Animal {
Dog(){
System.out.println(" A dog is created");
}
}
class Main {
public static void main(String args[]) {
Dog d=new Dog();
}
}
```

Output:

```
An animal is created
A dog is created
```

To read more: [super Keyword in Java]

66) How can constructor chaining be done by using the super keyword?

Constructor chaining in Java refers to the process of calling one constructor from another within the same class or in the parent class. The super keyword is used to invoke the constructor of the parent class.

Example

```
class Person
{
    String name, address;
    int age;
    public Person(int age, String name, String address)
    {
        this.age = age;
        this.name = name;
        this.address = address;
    }
}
class Employee extends Person
{
    float salary;
    public Employee(int age, String name, String address, float salary)
    {
        super(age,name,address);
        this.salary = salary;
    }
}
public class Main
{
    public static void main (String args[])
    {
        Employee e = new Employee(22, "Mukesh", "Delhi", 90000);
        System.out.println("Name: "+e.name+" Salary: "+e.salary+" Age: "+e.age+" Address: "+e.address);
    }
}
```

Output

```
Name: Mukesh Salary: 90000.0 Age: 22 Address: Delhi
```

67) What are the main uses of the super keyword?

There are the following uses of the super keyword:

- It is used to refer to the immediate parent class instance variable.
- It is used to invoke the immediate parent class method.
- The constructor super() can be used to invoke the immediate parent class constructor.

To read more: super keyword in Java

68) What are the differences between this and super keyword?

These are the following differences between this and super keyword:

The super keyword always points to the parent class contexts whereas this keyword always points to the current instance of the class.

The super keyword is primarily used for initializing the base class variables within the derived class constructor whereas this keyword primarily used to differentiate between local and instance variables when passed in the class constructor.

The super and this must be the first statement inside constructor otherwise the compiler will throw an error.

To read more: Difference between this and super in Java

69) What is the output of the following Java program?

```
class Person
{
    public Person()
    {
        System.out.println("Person class constructor called");
    }
}

class Employee extends Person
{
    public Employee()
    {
        System.out.println("Employee class constructor called");
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Employee e = new Employee();
    }
}
```

Output

```
Person class constructor called
Employee class constructor called
```

Explanation

The super() is implicitly invoked by the compiler if no super() or this() is included explicitly within the derived class constructor. Therefore, in this case, The Person class constructor is called first and then the Employee class constructor is called.

70) Can we use this() and super() in a constructor?

No, you cannot use both this() and super() in the same constructor because this() and super() must be the first statement in the class constructor. Using both in the same constructor would result in a compilation error.

Example:

```
public class Main {
    Main()
     {
         super();
         this();
         System.out.println("Main class object is created");
     }
     public static void main(String []args){
     Main t = new Main();
     }
}
```

Output:

```
Main.java:5: error: call to this must be first statement in constructor
```

To read more: Difference Between this and super in Java

71) What is object cloning?

Object cloning in Java refers to the process of creating an exact copy of an object. The clone() method of the Object class is used to clone an object. The java.lang.Cloneable interface must be implemented by the class whose object clone we want to create. If we do not implement Cloneable interface, clone() method generates CloneNotSupportedException. The clone() method creates a new object with the same state as the original object.

```
protected Object clone() throws CloneNotSupportedException
```

To read more: Object Cloning in Java

Core Java - OOP Concepts: Method Overloading Interview Questions

Questions in this topic (6)

72) What is method overloading?
73) Why is method overloading not possible by changing the return type in Java?
74) Can we overload the methods by making them static?
75) Can we overload the main() method?
76) What is method overloading with type promotion?
77) What is the output of the following Java program?

72) What is method overloading?

Method overloading is the polymorphism technique that allows us to create multiple methods with the same name but different signatures. Method overloading in Java occurs when a class has multiple methods with the same name but different parameter lists (number, type, or order of parameters). We can achieve method overloading in the following two ways:

- Different number of arguments
- Different data types of arguments

Method overloading increases the readability of the program. Method overloading is performed to figure out the program quickly.

To read more: Method Overloading in Java

73) Why is method overloading not possible by changing the return type in Java?

In Java, method overloading is not possible by changing the return type of the program to avoid ambiguity. The method name and the parameter types determine the method signature. Changing only the return type does not provide enough information for the compiler to distinguish between overloaded methods.

```
class Adder{
static int add(int a,int b){return a+b;}
static double add(int a,int b){return a+b;}
}
class Main {
public static void main(String[] args){
System.out.println(Adder.add(11,11));//ambiguity
}}
```

Output:

```
Compile Time Error: method add(int, int) is already defined in class Adder
```

74) Can we overload the methods by making them static?

No, we cannot overload the methods by just applying the static keyword to them (number of parameters and types are the same). Consider the following example.

```
public class Animal
{
    void consume(int a)
    {
        System.out.println(a+" consumed!!");
    }
    static void consume(int a)
    {
        System.out.println("consumed static "+a);
    }
    public static void main (String args[])
    {
        Animal a = new Animal();
        a.consume(10);
        Animal.consume(20);
    }
}
```

Output

```
Animal.java:7: error: method consume(int) is already defined in class Animal
    static void consume(int a)
                ^
Animal.java:15: error: non-static method consume(int) cannot be referenced from a static context
        Animal.consume(20);
              ^
2 errors
```

75) Can we overload the main() method?

Yes, we can have any number of main() methods in a Java program by using method overloading. However, the JVM looks for the standard public static void main(String[] args) signature when starting a Java program. Overloaded main() methods can be called from within the standard main() method, but the program execution begins from the standard main() method.

To read more: Can we overload the main() method in Java

76) What is method overloading with type promotion?

By type promotion is method overloading, we mean that one data type can be promoted to another implicitly if no exact matching is found.

Java Method Overloading with Type Promotion

As displayed in the above diagram, the byte can be promoted to short, int, long, float, or double. The short datatype can be promoted to int, long, float, or double. The char datatype can be promoted to int, long, float, or double, and so on. Consider the following example.

```
class Main {
  void sum(int a,long b){System.out.println(a+b);}
  void sum(int a,int b,int c){System.out.println(a+b+c);}
  public static void main(String args[]){
Main obj=new Main();
  obj.sum(20,20);//now second int literal will be promoted to long
  obj.sum(20,20,20);
  }
}
```

Output

```
40
60
```

77) What is the output of the following Java program?

```
class Main
{
    void sum(int a, long b)
    {
        System.out.println("a method invoked");
    }

    void sum(long a, int b)
    {
        System.out.println("b method invoked");
    }

    public static void main(String args[])
    {
        Main obj = new Main();

        obj.sum(20, 20L);
    }
}
```

Output

```
a method invoked
```

Explanation

There are two methods defined with the same name, i.e., sum. The first method accepts the integer and long types, whereas the second method accepts long and the integer types. The parameters passed are a = 20, b = 20. We cannot tell which method will be called, as there is no clear differentiation mentioned between integer literal and long literal. This is the case of ambiguity. Therefore, the compiler will throw an error.

Core Java - OOP Concepts: Method Overriding Interview Questions

Questions in this topic (12)

78) What is method overriding?
79) Can we override the static method?
80) Why can we not override a static method?
81) Can we override the overloaded method?
82) What are the differences between method overloading and overriding?
83) Can we override the private methods?
84) Can we change the scope of the overridden method in the subclass?
85) Can we modify the throws clause of the superclass method while overriding it in the subclass?
86) What is the output of the following Java program?
87) Can you have virtual functions in Java?
88) What is a covariant return type?
89) What is the output of the following Java program?

78) What is method overriding?

If a subclass provides a specific implementation of a method that is already provided by its parent class, it is known as Method Overriding. Method overriding in Java occurs when a subclass provides a specific implementation for a method that is already defined in its superclass. It is used for runtime polymorphism and to implement the interface methods.

Rules for Method Overriding

- The method must have the same name as in the parent class.
- The method must have the same signature as in the parent class.
- Two classes must have an IS-A relationship between them.

To read more: Method Overriding in Java

79) Can we override the static method?

No, we cannot override the static method because they are a part of the class, not the object.

To read more: Static method in Java

80) Why can we not override a static method?

Static methods cannot be overridden in the traditional sense of dynamic method dispatch that is associated with polymorphism. It is because the static method is a part of the class, and it is bound with the class, whereas the instance method is bound with the object, and static gets memory in the class area, and instance gets memory in the heap.

81) Can we override the overloaded method?

Yes, we can override the overloaded method in Java. Overriding: Overriding is a feature of OOP languages like Java that is related to run-time polymorphism. A subclass (or derived class) provides a specific implementation of a method in a superclass (or base class).

82) What are the differences between method overloading and overriding?

Method Overloading	Method Overriding
Method overloading increases the readability of the program.	Method overriding provides the specific implementation of the method that is already provided by its superclass.

Method overloading occurs within the class.	Method overriding occurs in two classes that have IS-A relationship between them.
In this case, the parameters must be different.	In this case, the parameters must be the same.

To read more: Difference Between Method Overloading and Method Overriding in Java

83) Can we override the private methods?

No, we cannot override the private methods in Java because the scope of private methods is limited to the class, and we cannot access them outside of the class. But private methods are not accessible outside their class. Therefore, private methods cannot be overridden because subclasses do not inherit them.

84) Can we change the scope of the overridden method in the subclass?

Yes, we can change the scope of the overridden method in the subclass. However, we must notice that we cannot decrease the accessibility of the method. The following point must be taken care of while changing the accessibility of the method.

- The private access specifier can be changed to protected, public, or default.
- The protected access specifier can be changed to public or default.
- The default access specifier can be changed to public.
- The public access specifier will always remain public.

85) Can we modify the throws clause of the superclass method while overriding it in the subclass?

Yes, we can modify the throws clause of the superclass method while overriding it in the subclass. However, there are some rules that are to be followed while overriding in case of exception handling.

- If the superclass method does not declare an exception, the subclass's overridden method cannot declare a checked exception, but it can declare an unchecked exception.
- If the superclass method declares an exception, the subclass overridden method can declare the same, a subclass exception, or no exception, but cannot declare a parent exception.

86) What is the output of the following Java program?

Example

```
class Base
{
    void method(int a)
    {
        System.out.println("Base class method called with integer a = "+a);
    }
    void method(double d)
    {
        System.out.println("Base class method called with double d ="+d);
    }
}
class Derived extends Base
{
    @Override
    void method(double d)
    {
        System.out.println("Derived class method called with double d ="+d);
    }
}
public class Main
{
    public static void main(String[] args)
    {
        new Derived().method(10);
    }
}
}
```

Output

```
Base class method called with integer a = 10
```

Explanation

The method() is overloaded in class Base, whereas it is derived in class Derived with the double type as the parameter. In the method call, the integer is passed.

87) Can you have virtual functions in Java?

Yes, all functions in Java are virtual by default. It means that method invocation is determined at runtime based on the actual type of the object on which the method is called. So, while the term "virtual" is not explicitly used in Java, the concept is present through the mechanism of dynamic method dispatch and method overriding.

To read more: Virtual Function in Java

88) What is a covariant return type?

Since Java 5, it is possible to override any method by changing the return type if the return type of the subclass overriding method is a subclass type. It is known as a covariant return type. The covariant return type specifies that the return type may vary in the same direction as the subclass.

Example

```
class A {
A get(){return this;}
}
class Main extends A {
Main get() {
return this; }
void message() {
System.out.println("Welcome to the covariant return type");}
public static void main(String args[]){
new Main().get().message();
}
}
```

Output

```
Welcome to the covariant return type
```

To read more: Covariant Return Type in Java

89) What is the output of the following Java program?

Example

```
class Base
{
    public void baseMethod()
    {
        System.out.println("BaseMethod called ...");
    }
}
class Derived extends Base
{
    public void baseMethod()
    {
        System.out.println("Derived method called ...");
    }
}
public class Main
{
    public static void main (String args[])
    {
        Base b = new Derived();
        b.baseMethod();
    }
}
```

Output

```
Derived method called ...
```

Explanation

The method of a Base class, i.e., baseMethod(), is overridden in the derived class. In the Main class, the reference variable b (of type Base class) refers to the instance of the Derived class. Here, Runtime polymorphism is achieved between the class Base and Derived.

At compile time, the presence of method baseMethod() is checked in the Base class; if its presence is then the program is compiled, otherwise a compiler error will be shown. In this case, baseMethod() is present in the Base class; therefore, it is compiled successfully.

However, at runtime, it checks whether the baseMethod() has been overridden by the Derived class; if so, then the Derived class method is called, otherwise the Base class method is called. In this case, the Derived class overrides the baseMethod();

therefore, the Derived class method is called.

Core Java - OOPs Concepts: final keyword Interview Questions

Questions in this topic (11)

90) What is the final variable?
91) What is the final method?
92) What is the final class?
93) What is the final blank variable?
94) Can we initialize the final blank variable?
95) Can we declare the main() method as final?
96) What is the output of the following Java program?
97) What is the output of the following Java program?
98) Can we declare a constructor as final?
99) Can we declare an interface as final?
100) What are the differences between the final method and the abstract method?

90) What is the final variable?

In Java, the final variable is used to restrict the user from updating it. If we initialize the final variable, we cannot change its value. The final keyword is used to declare a final variable. In other words, we can say that the final variable, once assigned to a value, can never be changed after that. The final variable, which is not assigned to any value, can only be assigned through the class constructor.

```
class Main {
 final int speedlimit=90;//final variable
 void run(){
  speedlimit=400;
 }
 public static void main(String args[]){
Main obj=new Main();
 obj.run();
 }
}//end of class
```

Output:

```
Compile Time Error
```

To read more: final Variable in Java

91) What is the final method?

If we change any method to a final method, we cannot override it. In Java, a final method is a method that subclasses cannot override. When a method is declared as final in a superclass, it means that no subclass can provide a different implementation for that method.

```
class Bike{
  final void run(){System.out.println("running");}
}
class Main extends Bike{
   void run(){System.out.println("running safely with 100kmph");}
   public static void main(String args[]){
   Main obj = new Main();
   obj.run();
   }
}
```

Output:

```
Compile Time Error
```

92) What is the final class?

If we make a class final, we cannot inherit it into any of the subclasses.

In Java, a final class is a class that cannot be subclassed. When a class is declared as final, it means that no other class can extend (inherit from) it.

```
final class Bike{}
class Main extends Bike{
  void run(){System.out.println("running safely with 100kmph");}
  public static void main(String args[]){
  Main obj = new Main();
  Obj.run();
  }
}
```

Output:

```
Compile Time Error
```

To read more: Fina Keyword in Java

93) What is the final blank variable?

In Java, a final blank variable refers to a blank final variable that has not been initialized at the time of declaration but must be initialized exactly once before it is used. The term "blank" implies that the variable is declared without an initial value. It is useful in the case when the user has some data that must not be changed by others, for example, the PAN Number. Consider the following example:

```
class Student{
int id;
String name;
final String PAN_CARD_NUMBER;
...
}
```

To read more: Blank final in Java

94) Can we initialize the final blank variable?

Yes, if it is not static, we can initialize it in the constructor. If it is a static, blank final variable, it can be initialized only in the static block.

95) Can we declare the main() method as final?

Yes, we can declare the main() method as public static final void main(String[] args){}. it is technically allowed to declare the main method as final, but it would not have any impact or special significance. The final keyword, when applied to a method, indicates that subclasses cannot override the method.

96) What is the output of the following Java program?

```
class Main {
 public static void main(String args[]){
   final int i;
   i = 20;
   System.out.println(i);
 }
}
```

Output

```
20
```

Explanation

Since i is the blank final variable, it can be initialized only once. We have initialized it to 20. Therefore, 20 will be printed.

97) What is the output of the following Java program?

```
class Base
{
    protected final void getInfo()
    {
        System.out.println("method of Base class");
    }
}
public class Derived extends Base
{
    protected final void getInfo()
    {
        System.out.println("method of Derived class");
    }
    public static void main(String[] args)
    {
        Base obj = new Base();
        obj.getInfo();
    }
}
```

Output

```
Main.java:10: error: getInfo() in Main cannot override getInfo() in Base
    protected final void getInfo()
                         ^
  overridden method is final
1 error
```

Explanation

The getDetails() method is final; therefore, it cannot be overridden in the subclass.

98) Can we declare a constructor as final?

No, constructors cannot be declared as final in Java because it is never inherited. Constructors are not ordinary methods; therefore, there is no sense in declaring constructors as final. However, if we try to do so, the compiler will throw an error.

99) Can we declare an interface as final?

No, we cannot declare an interface as final because some class must implement the interface to provide its definition. The whole idea of an interface is to provide a contract that classes can implement. Making an interface final would defeat the

purpose of allowing classes to implement it. However, if we try to do so, the compiler will show an error.

100) What are the differences between the final method and the abstract method?

Final method: A final method in a class cannot be overridden by subclasses. Once a method is declared final, it cannot be modified or overridden by any subclass.

Abstract method: An abstract method is a method declared in an abstract class or interface that has no implementation in the class/interface where it is declared. Subclasses or implementing classes must provide a concrete implementation for abstract methods.

The main difference between the final method and abstract method is that the abstract method cannot be final, as we need to override it in the subclass to give its definition.

To read more: Abstract Method in Java

Java Basics Interview Questions	Java OOPs Interview Questions
Java Multithreading Interview Questions	Java String & Exception Interview Questions
Java Collection Interview Questions	JDBC Interview Questions
Servlet Interview Questions	JSP Interview Questions
Spring Interview Questions	Hibernate Interview Questions
PL/SQL Interview Questions	SQL Interview Questions
Oracle Interview Questions	Android Interview Questions
SQL Server Interview Questions	MySQL Interview Questions

Core Java - Part 2

Core Java - OOPs: Polymorphism Interview Questions

Questions in this topic (6)

101) What is the difference between compile-time polymorphism and runtime polymorphism?
102) What is Runtime Polymorphism?
103) Can we achieve runtime polymorphism by data members?
104) What is the difference between static binding and dynamic binding?
105) What is the output of the following Java program?
106) What is Java instanceof operator?

101) What is the difference between compile-time polymorphism and runtime polymorphism?

There are the following differences between compile-time polymorphism and runtime polymorphism.

S.N.	Compile-Time Polymorphism	Runtime Polymorphism
1	In compile-time polymorphism, call to a method is resolved at compile-time.	In runtime polymorphism, call to an overridden method is resolved at runtime.
2	It is also known as static binding, early binding, or overloading.	It is also known as dynamic binding, late binding, overriding, or dynamic method dispatch.
3	Overloading is a way to achieve compile-time polymorphism in which, we can define multiple methods or constructors with different signatures.	Overriding is a way to achieve runtime polymorphism in which, we can redefine some particular method or variable in the derived class. By using overriding, we can give some specific implementation to the base class properties in the derived class.
4	It provides fast execution because the type of an object is determined at compile-time.	It provides slower execution as compare to compile-time because the type of an object is determined at run-time.
5	Compile-time polymorphism provides less flexibility because all the things are resolved at compile-time.	Run-time polymorphism provides more flexibility because all the things are resolved at runtime.

To read more: Compile-Time Polymorphism in Java

102) What is Runtime Polymorphism?

Runtime polymorphism also known as dynamic method dispatch is a process in which a call to an overridden method is resolved at runtime rather than at compile-time. In this process, an overridden method is called through the reference variable of a superclass. It is achieved through method overriding in object-oriented programming, where a subclass provides a specific implementation for a method that is already defined in its superclass.

```
class Bike{
  void run(){System.out.println("running");}
}
class Splendor extends Bike{
  void run(){System.out.println("running safely with 60km");}
  public static void main(String args[]){
    Bike b = new Splendor();//upcasting
    b.run();
  }
}
```

Output:

```
running safely with 60km.
```

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

To read more: Polymorphism in Java

103) Can we achieve runtime polymorphism by data members?

No, runtime polymorphism is primarily related to methods/functions, not data members because method overriding is used to achieve runtime polymorphism and data members cannot be overridden. We can override the member functions but not the data members. Data members (fields or properties) do not exhibit polymorphic behavior in the same way. Consider the example given below.

```
 class Bike{
  int speedlimit=90;
 }
 class Honda3 extends Bike{
  int speedlimit=150;
  public static void main(String args[]){
   Bike obj=new Honda3();
   System.out.println(obj.speedlimit);//90
    }
}
```

Output:

```
90
```

104) What is the difference between static binding and dynamic binding?

In case of the static binding, the type of the object is determined at compile-time whereas, in the dynamic binding, the type of the object is determined at runtime.

Static Binding

```
class Dog{
 private void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Dog d1=new Dog();
  d1.eat();
 }
}
```

Dynamic Binding

```
class Animal{
 void eat(){System.out.println("animal is eating...");}
}

class Dog extends Animal{
 void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Animal a=new Dog();
  a.eat();
 }
}
```

To read more: Static and Dynamic Binding in Java

105) What is the output of the following Java program?

```
class BaseTest
{
  void print()
  {
    System.out.println("BaseTest:print() called");
  }
}
public class Test extends BaseTest
{
  void print()
  {
    System.out.println("Test:print() called");
  }
  public static void main (String args[])
  {
    BaseTest b = new Test();
    b.print();
  }
}
```

Output

```
  Test:print() called
```

Explanation

It is an example of Dynamic method dispatch. The type of reference variable b is determined at runtime. At compile-time, it is checked whether that method is present in the Base class. In this case, it is overridden in the child class, therefore, at runtime the derived class method is called.

To read more: Java Programs

106) What is Java instanceof operator?

The instanceof operator in Java is also known as type comparison operator because it compares the instance with type. It is used to test whether an object is an instance of a particular class, interface, or type. It returns either true or false. If we apply the instanceof operator with any variable that has a null value, it returns false. Consider the following example.

```
class Simple1{
 public static void main(String args[]){
 Simple1 s=new Simple1();
 System.out.println(s instanceof Simple1);//true
 }
}
```

Output

```
true
```

To read more: Java instanceof operator

An object of subclass type is also a type of parent class. For example, if Dog extends Animal then object of Dog can be referred by either Dog or Animal class.

Core Java - OOPs Concepts: Abstraction Interview Questions

Questions in this topic (17)

107) What is the abstraction?
108) What is the difference between abstraction and encapsulation?
109) What is the abstract class?
110) Can there be an abstract method without an abstract class?
111) Is the following program written correctly? If yes, what will be the output of the program?
112) Can we use abstract and final both with a method?
113) Is it possible to instantiate the abstract class?
114) What is the interface?
115) Can we declare an interface method static?
116) Can the Interface be final?
117) What is a marker interface?
118) What are the differences between abstract class and interface?
119) Can we define private and protected modifiers for the members in interfaces?
120) When can an object reference be cast to an interface reference?
121) How to make a read-only class in Java?
122) How to make a write-only class in Java?
123) What are the advantages of Encapsulation in Java?

107) What is the abstraction?

Abstraction is a process of hiding the implementation details and showing only functionality to the user. It displays just the essential things to the user and hides the internal information, it involves simplifying the system by modelling classes based on real-world entities and focusing on relevant attributes and behaviors. For example, sending SMS where you type the text and send the message. We do not know the internal processing about the message delivery. Abstraction enables us to focus on what the object does instead of how it does it. Abstraction lets you focus on what the object does instead of how it does it.

In Java, there are two ways to achieve the abstraction.

- Abstract Class
- Interface

To read more: Abstraction in Java

108) What is the difference between abstraction and encapsulation?

Abstraction and encapsulation are related concepts but have distinct meanings. Abstraction is about showing only the essential features of an object while hiding the unnecessary details. Encapsulation, on the other hand, is the bundling of data and methods that operate on that data into a single unit, often a class. Abstraction is more about design and modelling, while encapsulation is about implementation and data hiding.

To read more: Abstraction Vs Encapsulation

109) What is the abstract class?

A class that is declared as abstract is known as an abstract class. It needs to be extended and its method implemented. It cannot be instantiated. Abstract classes can have both abstract and concrete methods and can be extended by other classes. It can also have the final methods which will force the subclass not to change the body of the method.

```
abstract class Bike{
  abstract void run();
}
class Honda4 extends Bike{
void run(){System.out.println("running safely");}
public static void main(String args[]){
 Bike obj = new Honda4();
 obj.run();
}
}
```

Output

```
running safely
```

To read more: Abstract class in Java

110) Can there be an abstract method without an abstract class?

No, an abstract method must belong to an abstract class. Abstract methods are defined in abstract classes and must be implemented by the subclasses that extend the abstract class.

To read more: Abstract Method in Java

111) Is the following program written correctly? If yes, what will be the output of the program?

```
abstract class Calculate
{
    abstract int multiply(int a, int b);
}

public class Main
{
    public static void main(String[] args)
    {
        int result = new Calculate()
        {
            @Override
            int multiply(int a, int b)
            {
                return a*b;
            }
        }.multiply(12,32);
        System.out.println("result = "+result);
    }
}
```

Yes, the program is written correctly. The Main class provides the definition of abstract method multiply declared in abstract class Calculation. The output of the program will be:

Output

```
384
```

Yes, the program is written correctly. The Main class provides the definition of abstract method multiply declared in abstract class Calculation. The output of the program will be:

112) Can we use abstract and final both with a method?

No, we cannot use both abstract and final together for a method. An abstract method is meant to be overridden by subclasses, while a final method cannot be overridden.

113) Is it possible to instantiate the abstract class?

No, we cannot instantiate an abstract class directly in Java. Abstract classes are meant to be extended by concrete subclasses, and we can only create objects of concrete classes.

114) What is the interface? Corrected

CORRECTED The source said an interface can contain 'only abstract methods'. That was true before Java 8.

Checked against: JLS chapter 9: interfaces.

An **interface** is a reference type that defines a contract: a set of methods that implementing classes provide. A class can implement many interfaces, which is how Java supports multiple inheritance of type.

What an interface can contain today:

- **Constants:** fields are implicitly `public static final`.
- **Abstract methods:** implicitly `public abstract`.
- **Default methods** (Java 8): methods with a body that implementing classes inherit.
- **Static methods** (Java 8): called on the interface itself.
- **Private methods** (Java 9): helpers shared by default and static methods.

An interface cannot have instance fields or constructors. An interface with exactly one abstract method is a **functional interface** and can be used with lambdas.

115) Can we declare an interface method static?

No, interface methods cannot be declared as static prior to Java 8. However, starting from Java 8, we can declare static methods in interfaces.

116) Can the Interface be final?

No, because an interface needs to be implemented by the other class and if it is final, it cannot be implemented by any class.

117) What is a marker interface?

A marker interface in Java is an interface that does not declare any methods. Its purpose is to mark or tag a class, indicating that the class possesses certain behavior or characteristics. For example, Serializable, Cloneable are marker interfaces. The marker interface can be declared as follows.

```
public interface Serializable{
}
```

118) What are the differences between abstract class and interface? Corrected

CORRECTED The source said an interface 'has only abstract methods' and 'cannot have static methods'. Both stopped being true in Java 8.

Checked against: JLS chapter 9: interfaces.

Aspect	Abstract class	Interface
Methods	Abstract and concrete methods	Abstract, **default** and **static** methods (Java 8), **private** methods (Java 9)
Fields	Any fields, including instance variables	Only constants (`public static final`)
Constructors	Yes	No

Inheritance	A class extends only one class	A class can implement many interfaces; an interface can extend many interfaces
State	Can hold per-object state	Cannot hold instance state
Typical use	Shared base class with common code and state	A capability or contract that unrelated classes can offer

Before Java 8 an interface could contain only abstract methods; this is the old rule that many sources still repeat.

119) Can we define private and protected modifiers for the members in interfaces? Corrected

CORRECTED The source said all interface members are implicitly 'public and abstract'. Default and static methods are not abstract, and Java 9 added private methods.

Checked against: JLS chapter 9: interfaces.

- **Fields:** always `public static final`.
- **Methods:** `public` by default (abstract, default or static). `protected` is not allowed.
- **Private methods** (Java 9) **are** allowed: they are helpers for default and static methods inside the interface.

A class that implements an interface method must declare it `public`, because it cannot weaken the access of the interface method.

120) When can an object reference be cast to an interface reference?

An object reference can be cast to an interface reference when the object implements the referenced interface.

121) How to make a read-only class in Java?

A class can be made read-only by making all of the fields private. The read-only class will have only getter methods which return the private property of the class to the main method. We cannot modify this property because there is no setter method available in the class. Consider the following example.

```
  //A Java class which has only getter methods.
public class Student{
//private data member
private String college="AKG";
//getter method for college
public String getCollege(){
return college;
}
}
```

122) How to make a write-only class in Java?

A class can be made write-only by making all of the fields private. The write-only class will have only setter methods which set the value passed from the main() method to the private fields. We cannot read the properties of the class because there is no getter method in this class. Consider the following example.

```
  //A Java class which has only setter methods.
public class Student{
//private data member
private String college;
//getter method for college
public void setCollege(String college){
this.college=college;
}
}
```

123) What are the advantages of Encapsulation in Java?

There are the following advantages of Encapsulation in Java:

By providing only the setter or getter method, we can make the class read-only or write-only. In other words, we can skip the getter or setter methods.

It provides control over the data. Suppose we want to set the value of id which should be greater than 100 only, we can write the logic inside the setter method. We can write the logic not to store the negative numbers in the setter methods.

It is a way to achieve data hiding in Java because other class will not be able to access the data through the private data members.

The encapsulate class is easy to test. So, it is better for unit testing.

The standard IDE's are providing the facility to generate the getters and setters. So, it is easy and fast to create an encapsulated class in Java.

To read more: Encapsulation in Java

Core Java - OOPs Concepts: Package Interview Questions

Questions in this topic (7)

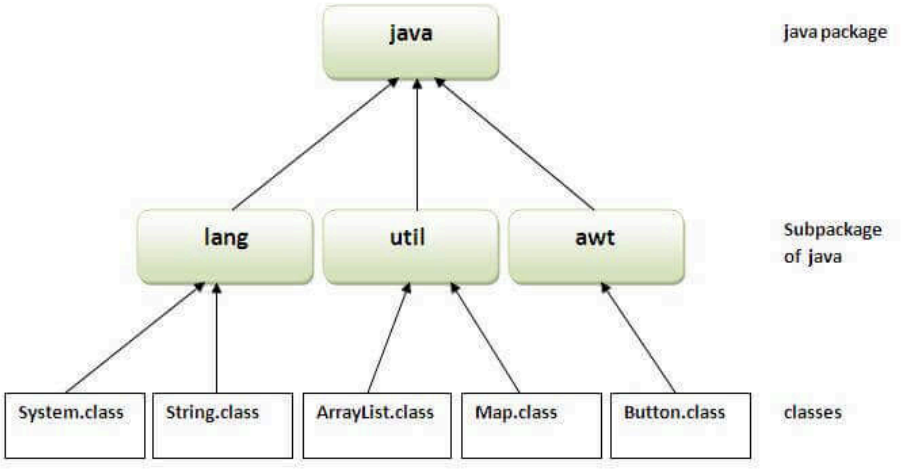
124) What is the package?
125) What are the advantages of defining packages in Java?
126) How to create packages in Java?
127) How can we access some class in another class in Java?
128) Do I need to import java.lang package any time? Why?
129) Can I import same package/class twice? Will the JVM load the package twice at runtime?
130) What is the static import?

124) What is the package?

A package is a group of similar type of classes, interfaces, and sub-packages. It provides access protection and removes naming collision. The packages in Java can be categorized into two forms, inbuilt package, and user-defined package. There are many built-in packages such as Java, lang, awt, javax, swing, net, io, util, sql, etc. Consider the following example to create a package in Java.

Simple.java

```
package mypack;
public class Simple{
 public static void main(String args[]){
    System.out.println("Welcome to package");
   }
}
```



To read more: Packages in Java

125) What are the advantages of defining packages in Java?

By defining packages, we can avoid the name conflicts between the same class names defined in different packages. Packages also enable the developer to organize the similar classes more effectively. For example, one can clearly understand that the classes present in java.io package are used to perform io related operations.

126) How to create packages in Java?

If you are using the programming IDEs like Eclipse, NetBeans, MyEclipse, etc. click on **file->new->project** and eclipse will ask you to enter the name of the package. It will create the project package containing various directories such as src, etc.

If we are using an editor like notepad for Java programming, use the following steps to create the package.

To create packages in Java, you use the package keyword at the beginning of your Java source file and organize your classes in directories based on the package name. Define a **package package_name. Create the class with the name class_name and save this file with your_class_name.java**.

Now compile the file by running the following command on the terminal.

```
javac -d . your_class_name.java
```

The above command creates the package with the name **package_name** in the present working directory.

Now, run the class file by using the absolute class file name, like following.

```
java package_name.class_name
```

To read more: How to Create Package in Java

127) How can we access some class in another class in Java?

There are two ways to access a class in another class.

By using the fully qualified name: To access a class in a different package, either we must use the fully qualified name of that class, or we must import the package containing that class.

```
// Using fully qualified name
com.example.otherpackage.OtherClass obj = new com.example.otherpackage.OtherClass();
// Using import statement
import com.example.otherpackage.OtherClass;
// ...
OtherClass obj = new OtherClass();
```

By using the relative path, we can use the path of the class that is related to the package that contains our class. It can be the same or subpackage.

128) Do I need to import java.lang package any time? Why?

No. we do not need to import the java.lang package explicitly as it is automatically imported in every Java program. It is by default loaded internally by the JVM.

To read more: Which Package is Imported by Default in Java

129) Can I import same package/class twice? Will the JVM load the package twice at runtime?

Yes, one can import the same package or the same class multiple times. Neither compiler nor JVM complains about it. However, the JVM will load the package or class only once at runtime.

130) What is the static import?

Static import is a feature introduced in Java 5 that allows members (fields and methods) of a class to be used in a Java program without qualifying them with the class name.

```
import static java.lang.Math.*;
public class MyMathClass {
    public static void main(String[] args) {
        double result = sqrt(25); // No need to use Math.sqrt here
        // ...
    }
}
```

To read more: Java Static Import

Java: Exception Handling Interview Questions

Questions in this topic (18)

131) How many types of exception occurs in a Java program?
132) What is Exception Handling?
133) Explain the hierarchy of Java Exception classes?
134) What is the difference between Checked Exception and Unchecked Exception?
135) What is the base class for Error and Exception?
136) Is it necessary that each try block must be followed by a catch block?
137) What is the output of the following Java program?
138) What is finally block?
139) Can finally block be used without a catch?
140) Is there any case when finally block will not be executed?
141) What is the difference between throw and throws?
142) What is the output of the following Java program?
143) What is the output of the following Java program?
144) Can an exception be rethrown?
145) Can subclass overriding method declare an exception if parent class method doesn't throw an exception?
146) What is exception propagation?
147) What is the output of the following Java program?
148) What is the output of the following Java program?

131) How many types of exception occurs in a Java program?

There are mainly two types of exceptions: checked and unchecked. Here, an error is considered as the unchecked exception. According to Oracle, there are three types of exceptions:

- **Checked Exception:** Checked exceptions are checked at compile-time, and the programmer must handle them using try-catch or declare them in the method's throws clause. For example, SQLException, ClassNotFoundException, etc.
- **Unchecked Exception:** Unchecked exceptions are the one which are handled at runtime because they cannot be checked at compile-time. For example, ArithmaticException, NullPointerException, ArrayIndexOutOfBoundsException, etc.

To read more: Types of Exception in Java with Examples

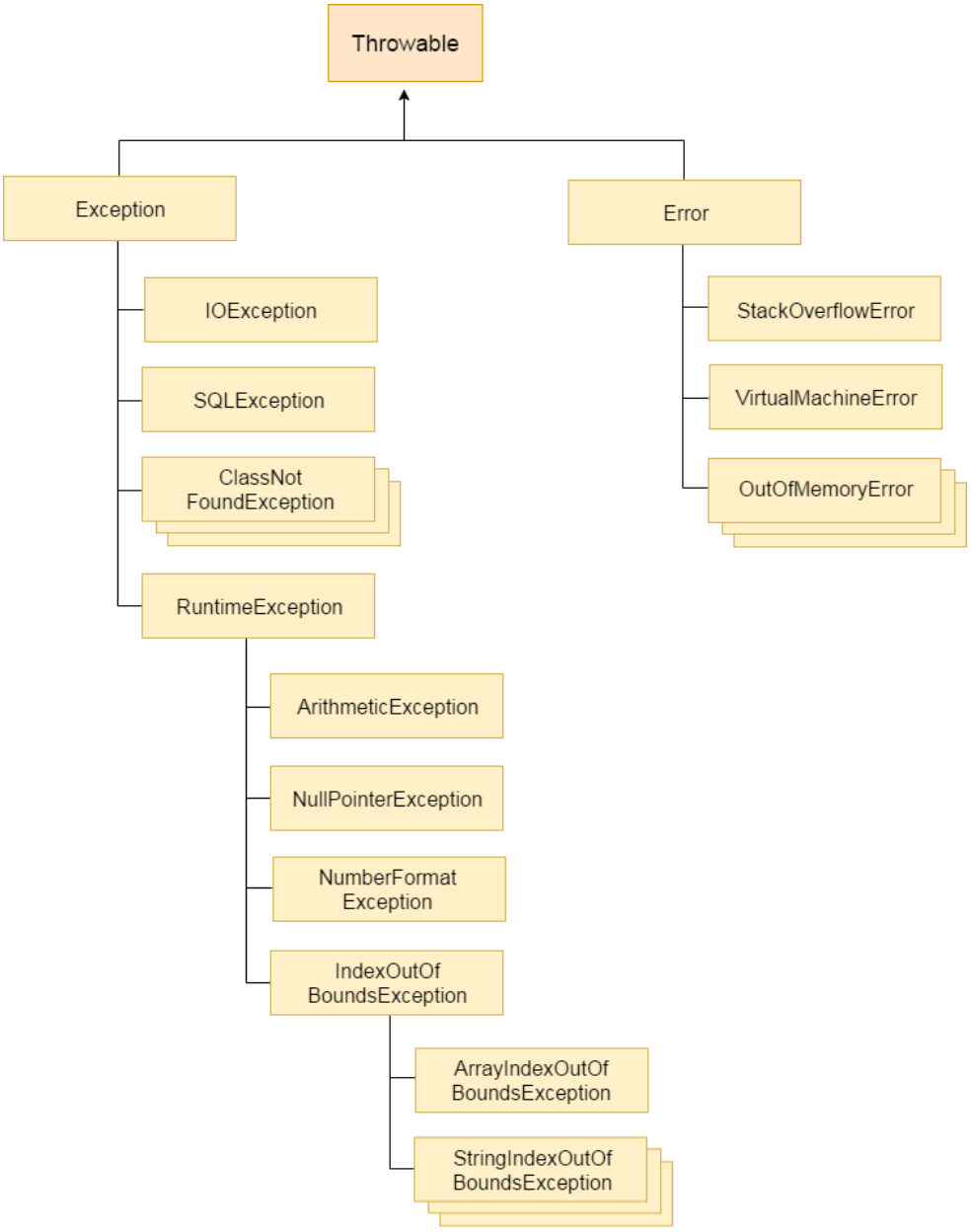
132) What is Exception Handling?

Exception Handling is a mechanism that is used to handle runtime errors (exceptions) to prevent the abnormal termination of a program. It is used primarily to handle checked exceptions. Exception handling maintains the normal flow of the program. There are mainly two types of exceptions: checked and unchecked. Here, the error is considered as the unchecked exception.

To read more: Exception Handling in Java

133) Explain the hierarchy of Java Exception classes?

The hierarchy of Java Exception classes includes the top-level class java.lang.Throwable class is the root class of Java Exception hierarchy which is inherited by two subclasses: Exception and Error. A hierarchy of Java Exception classes are given below:



134) What is the difference between Checked Exception and Unchecked Exception?

1) Checked Exception

Checked exceptions are checked at compile-time, and the programmer must handle them using try-catch or declare them in the method's throws clause. The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions, for example, IOException, SQLException, etc.

2) Unchecked Exception

Checked exceptions are not checked at compile-time and include runtime exceptions and errors. The classes that extend RuntimeException are known as unchecked exceptions, for example, ArithmeticException, NullPointerException, etc.

More details.

135) What is the base class for Error and Exception?

The Throwable class is the base class for Error and Exception.

136) Is it necessary that each try block must be followed by a catch block?

Corrected

CORRECTED The source forgot that a try-with-resources block needs neither catch nor finally.

Checked against: JLS 14.20: the try statement.

No. A `try` block must be followed by at least one of: a `catch` block, a `finally` block, or it must be a **try-with-resources** statement (Java 7), which needs neither:

```
try (BufferedReader in = new BufferedReader(new FileReader("a.txt"))) {
    System.out.println(in.readLine());
}   // the reader is closed automatically; the IOException must be declared with 'throws' or caught by the
caller
```

So all of these are valid: `try-catch`, `try-finally`, `try-catch-finally` and `try (resource) { }`. Exceptions that are not caught here must be declared in the method's `throws` clause (if they are checked).

137) What is the output of the following Java program?

```
public class ExceptionHandlingExample {
public static void main(String args[])
{
        try
        {
                int a = 1/0;
                System.out.println("a = "+a);
        }
        catch(Exception e){System.out.println(e);}
        catch(ArithmeticException ex){System.out.println(ex);}
}
}
```

Output

```
ExceptionHandlingExample.java:10: error: exception ArithmeticException has already been caught
        catch(ArithmeticException ex){System.out.println(ex);}
        ^
1 error
```

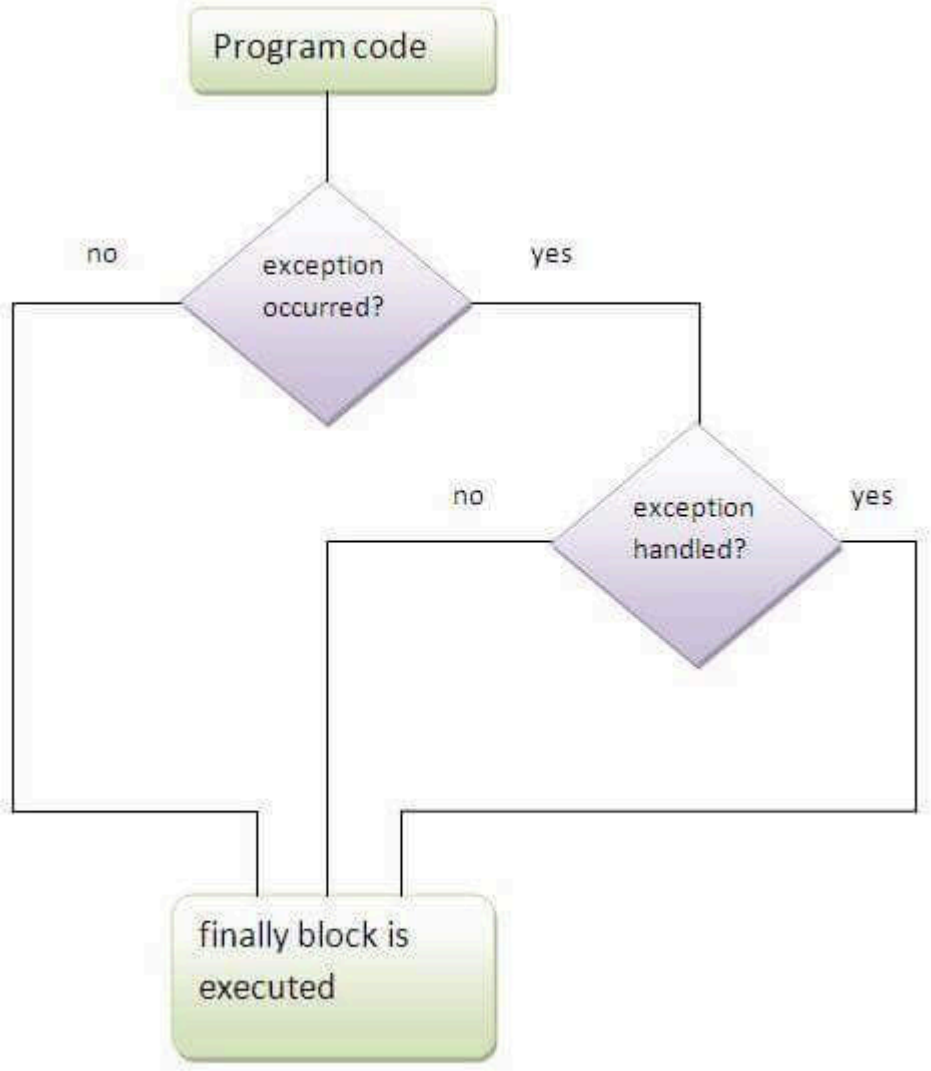
Explanation

ArithmaticException is the subclass of Exception. Therefore, it cannot be used after Exception. Since Exception is the base class for all the exceptions, therefore, it must be used at last to handle the exception. No class can be used after this.

138) What is finally block?

In programming languages like Java, the finally block is a part of exception handling. It contains code that will be executed regardless of whether an exception is thrown or not. The finally block is often used to ensure that certain clean-up or resource release operations take place, whether an exception occurs or not.

Finally block follows try or catch block. If we do not handle the exception, before terminating the program, JVM runs finally block, (if any). The finally block is mainly used to place the clean-up code such as closing a file or closing a connection. Here, we must know that for each try block there can be zero or more catch blocks, but only one finally block. The finally block will not be executed if program exits (either by calling System.exit() or by causing a fatal error that causes the process to abort).



To read more: Java finally block

139) Can finally block be used without a catch?

Yes, a finally block can be used without a catch block. In exception handling, we can have a try block followed by either a catch block, a finally block, or both. According to the definition of finally block, it must be followed by a try or catch block, therefore, we can use try block instead of catch. More details.

140) Is there any case when finally block will not be executed?

The finally block will not be executed if program exits (either by calling System.exit() or by causing a fatal error that causes the process to abort). One such case is when the program is terminated abruptly, for example, by calling System.exit() or if there is a hardware failure or an error in the JVM that prevents normal program termination. More details.

141) What is the difference between throw and throws?

throw keyword	throws keyword
The **throw** keyword is used to throw an exception explicitly.	The **throws** keyword is used to declare an exception.
The checked exceptions cannot be propagated with throw only.	The checked exception can be propagated with throws
The **throw** keyword is followed by an instance.	The **throws** keyword is followed by class.

The **throw** keyword is used within the method.	The **throws** keyword is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions, e.g., public void method()throws IOException, SQLException.

To read more: Difference Between throw and throws in Java

142) What is the output of the following Java program?

```
  public class Main{
     public static void main(String []args){
        try
        {
            throw 90;
        }
        catch(int e){
            System.out.println("Caught the exception "+e);
        }

     }
}
```

Output

```
Main.java:6: error: incompatible types: int cannot be converted to Throwable
            throw 90;
            ^
Main.java:8: error: unexpected type
        catch(int e){
              ^
  required: class
  found:    int
2 errors
```

Explanation

In Java, the throwable objects can only be thrown. If we try to throw an integer object, the compiler will show an error since we cannot throw basic data type from a block of code.

143) What is the output of the following Java program?

```
class Calculation extends Exception
{
    public Calculation()
    {
        System.out.println("Calculation class is instantiated");
    }
    public void add(int a, int b)
    {
        System.out.println("The sum is "+(a+b));
    }
}
public class Main{
     public static void main(String []args){
        try
        {
            throw new Calculation();
        }
        catch(Calculation c){
            c.add(10,20);
        }
    }
}
```

Output

```
Calculation class is instantiated
The sum is 30
```

Explanation

The object of Calculation is thrown from the try block which is caught in the catch block. The add() of Calculation class is called with the integer values 10 and 20 by using the object of this class. Therefore, there sum 30 is printed. The object of the Main class can only be thrown in the case when the type of the object is throwable. To do so, we need to extend the throwable class.

144) Can an exception be rethrown?

Yes, an exception can be rethrown. In Java, we can catch an exception in one catch block and then throw the same exception (or a different one) again. It can be useful for logging or handling the exception at a higher level in the program.

145) Can subclass overriding method declare an exception if parent class method doesn't throw an exception?

Corrected

> **CORRECTED** The source answered 'yes' without a limit. For **CHECKED** exceptions the answer is no; it also contradicts the correct rule given in Core Java Q85 of this set.
>
> Checked against: JLS 8.4.8.3: overriding and the throws clause.

Only unchecked exceptions. An overriding method may **not** declare a new or broader **checked** exception than the method it overrides. If the parent method declares no checked exception, the overriding method cannot declare one (compile error).

- It **may** throw any **unchecked** exception (`RuntimeException` and its subclasses, or `Error`).
- If the parent method declares a checked exception, the overriding method may declare the same exception, a subclass of it, or none.
- It cannot declare a parent (broader) checked exception.

146) What is exception propagation?

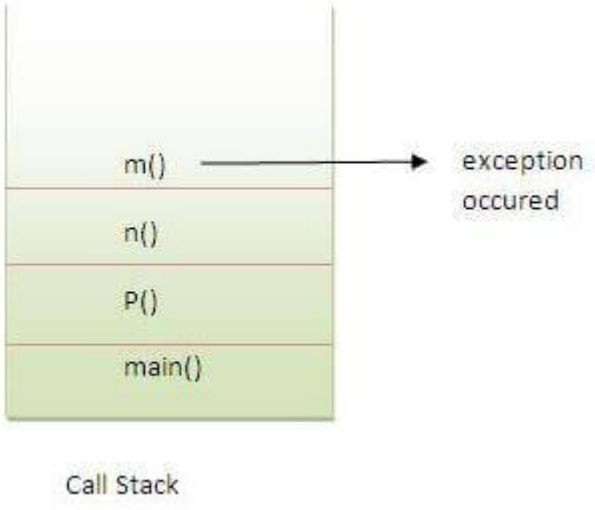
Exception propagation refers to the process by which an exception is thrown in a program and then propagated up the call stack until it is caught and handled by an appropriate catch block or until it reaches the top level of the program.

An exception is first thrown from the top of the stack and if it is not caught, it drops down the call stack to the previous method, if not caught there, the exception again drops down to the previous method, and so on until they are caught or until they reach the very bottom of the call stack. This procedure is called exception propagation. By default, checked exceptions are not propagated.

```
class TestExceptionPropagation1{
  void m(){
    int data=50/0;
  }
  void n(){
    m();
  }
  void p(){
   try{
    n();
   }catch(Exception e){System.out.println("exception handled");}
  }
  public static void main(String args[]){
   TestExceptionPropagation1 obj=new TestExceptionPropagation1();
   obj.p();
   System.out.println("normal flow...");
  }
}
```

Output:

```
exception handled
normal flow...
```



More details.

To read more: Java Exception Propagation

147) What is the output of the following Java program?

```
public class Main
{
    void a()
    {
        try{
        System.out.println("a(): Main called");
        b();
        }catch(Exception e)
        {
            System.out.println("Exception is caught");
        }
    }
    void b() throws Exception
    {
     try{
         System.out.println("b(): Main called");
         c();
     }catch(Exception e){
         throw new Exception();
     }
     finally
     {
         System.out.println("finally block is called");
     }
    }
    void c() throws Exception
    {
        throw new Exception();
    }

    public static void main (String args[])
    {
        Main m = new Main();
        m.a();
    }
}
```

Output

```
a(): Main called
b(): Main called
finally block is called
Exception is caught
```

Explanation

In the main() method, a() of the Main class is called that prints a message and call b(). The method b() prints some message and then call the method c(). The method c() throws an exception that is handled by the catch block of method b(). However, It propagates this exception by using **throw Exception()** to be handled by the method a(). As we know, finally block is always executed therefore the finally block in the method b() is executed first and prints a message. At last, the exception is handled by the catch block of the method a().

148) What is the output of the following Java program?

```
public class Calculation
{
    int a;
    public Calculation(int a)
    {
        this.a = a;
    }
    public int add()
    {
        a = a+10;
        try
        {
            a = a+10;
            try
            {
                a = a*10;
                throw new Exception();
            }catch(Exception e){
                a = a - 10;
            }
        }catch(Exception e)
        {
            a = a - 10;
        }
        return a;
    }

    public static void main (String args[])
    {
        Calculation c = new Calculation(10);
        int result = c.add();
        System.out.println("result = "+result);
    }
}
```

Output

```
result = 290
```

Explanation

The instance variable a of class Calculation is initialized to 10 using the class constructor which is called while instantiating the class. The add method is called which returns an integer value result. In the add() method, a is incremented by 10 to be 20. Then, in the first try block, 10 is again incremented by 10 to be 30. In the second try block, a is multiplied by 10 to be 300. The second try block throws the exception which is caught by the catch block associated with this try block. The catch block again alters the value of a by decrementing it by 10 to make it 290. Thus the add() method returns 290 which is assigned to result. However, the catch block associated with the outermost try block will never be executed since there is no exception which can be handled by this catch block.

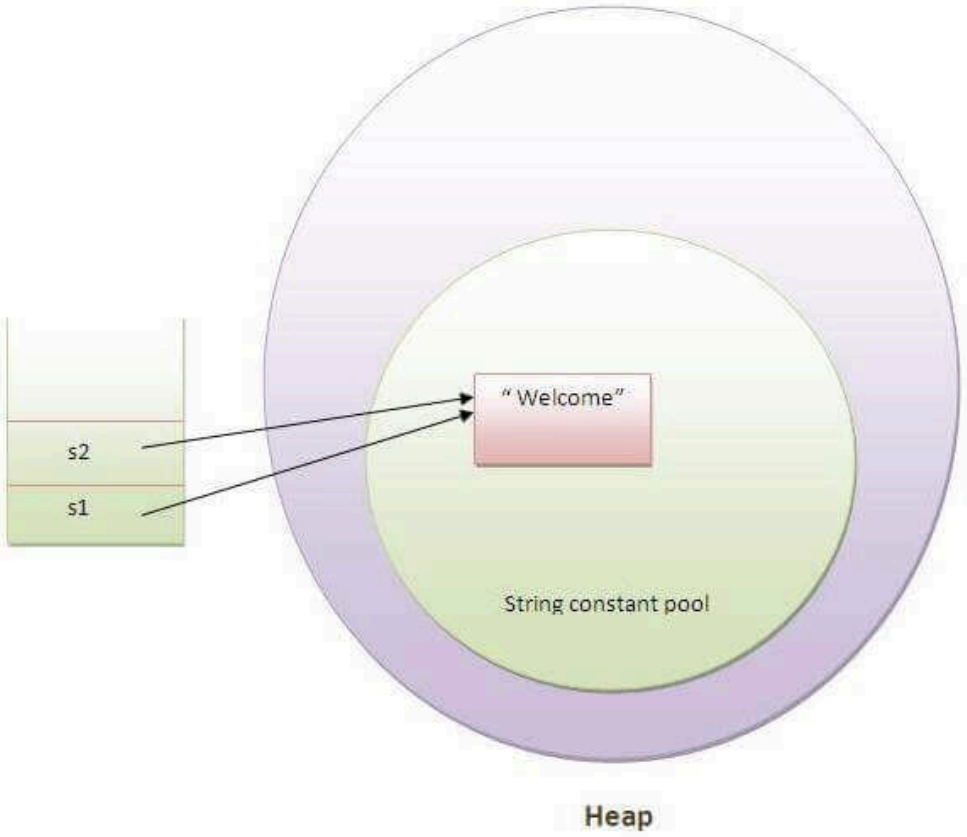
Java: String Handling Interview Questions

Questions in this topic (19)

149) What is String Pool?
150) What is the meaning of immutable regarding String?
151) Why are the objects immutable in Java?
152) What are the ways to create string object?
153) How many objects will be created in the following code?
154) Why Java uses the concept of the string literal?
155) How many objects will be created in the following code?
156) What is the output of the following Java program?
157) What is the output of the following Java program?
158) What are the differences between String and StringBuffer?
159) What are the differences between StringBuffer and StringBuilder?
160) How can we create an immutable class in Java?
161) What is the purpose of the toString() method in Java?
162) Why toCharArray() method is preferred over String to store the password?
163) Write a Java program to count the number of words present in a string?
164) Name some classes present in java.util.regex package.
165) How the metacharacters are different from the ordinary characters?
166) Write a regular expression to validate a password. A password must start with an alphabet and followed by alphanumeric characters. It must 8 to 20 characters long.
167) What is the output of the following Java program?

149) What is String Pool?

The String Pool is a special memory area in Java where string literals are stored. The main advantage of using the String pool is whenever we create a string literal; the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string does not exist in the pool, a new string instance is created and placed in the pool. This mechanism helps in memory optimization by reducing the number of duplicate string objects. Therefore, it saves the memory by avoiding the duplicacy.



To read more: String Pool in Java

150) What is the meaning of immutable regarding String?

The simple meaning of immutable is unmodifiable or unchangeable. In Java, String is immutable, i.e., once string object has been created, its value cannot be changed. Consider the following example for better understanding.

```
class Testimmutablestring{
 public static void main(String args[]){
   String s="Sachin";
   s.concat(" Tendulkar");//concat() method appends the string at the end
   System.out.println(s);//will print Sachin because strings are immutable objects
 }
}
```

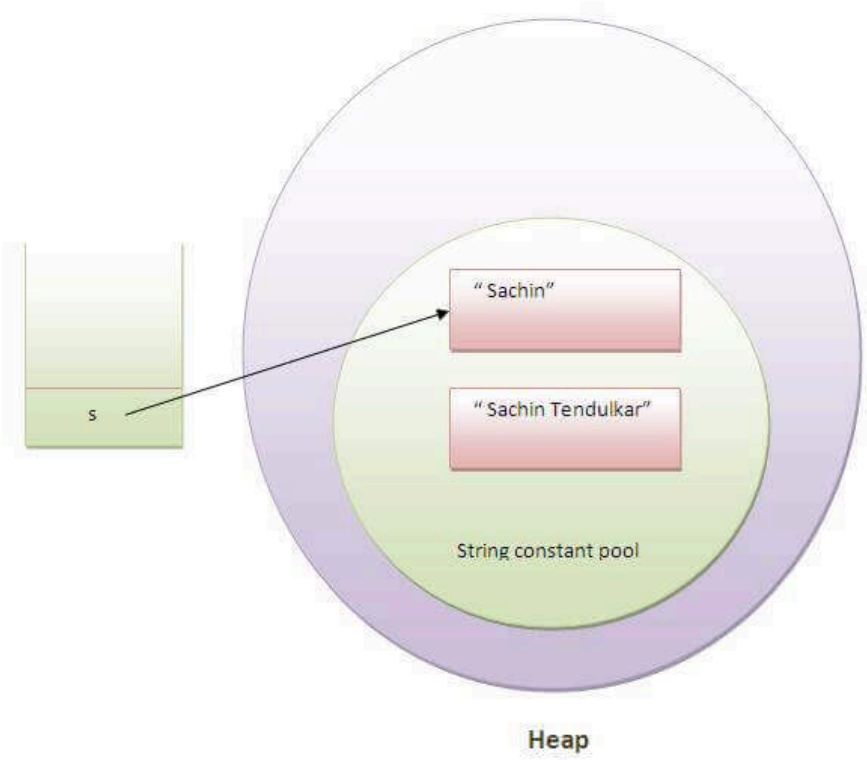
Output:

```
Sachin
```

To read more: Why String is Immutable in Java

151) Why are the objects immutable in Java?

Objects are immutable in Java for several reasons, Java uses the concept of the string literal. Suppose, there are five reference variables, all refer to one object "sachin". If one reference variable changes the value of the object, it will be affected by all the reference variables. Thread Safety, Immutable objects are inherently thread-safe. That is why string objects are immutable in Java.



More details.

152) What are the ways to create string object?

There are several ways to create a string object in Java:

1) Using String Literal

Java String literal is created by using double quotes. For Example:

```
String s="welcome";
```

Every time when we create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string does not exist in the pool, a new string instance is created and placed in the pool. String objects are stored in a special memory area known as the **string constant pool.** For example,

```
String s1="Welcome";
String s2="Welcome";    //it does not create a new instance
```

2) Using the new Keyword

```
String s=new String("Welcome");//creates two objects and one reference variable
```

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the constant string pool. The variable s will refer to the object in a heap (non-pool).

153) How many objects will be created in the following code?

Corrected

CORRECTED The source gave the wrong reason. One object is created because identical string **LITERALS SHARE ONE POOLED OBJECT** , not because strings are immutable.

Checked against: JLS 3.10.5: string literals.

```
String s1 = "Welcome";
String s2 = "Welcome";
String s3 = "Welcome";
```

One object. The first literal creates a `String` in the **string constant pool**; the next two identical literals reuse it, so `s1 == s2 == s3`.

Immutability is what makes this sharing **safe**: since no one can change the shared object, many references can point to it.

154) Why Java uses the concept of the string literal?

Java uses the concept of string literals for efficiency and memory optimization. To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool). It reduces memory consumption and improves performance.

To read more: Literals in Java

155) How many objects will be created in the following code?

```
String s = new String("Welcome");
```

Two objects, one in string constant pool and other in non-pool (heap).

To read more: String Literal Vs String Object in Java

156) What is the output of the following Java program?

```
  public class Test
{
    public static void main (String args[])
    {
        String a = new String("Sharma is a good player");
        String b = "Sharma is a good player";
        if(a == b)
        {
            System.out.println("a == b");
        }
        if(a.equals(b))
        {
            System.out.println("a equals b");
        }
    }
}
```

Output

```
  a equals b
```

Explanation

The operator **==** also check whether the references of the two string objects are equal or not. Although both of the strings contain the same content, their references are not equal because both are created by different ways (Constructor and String literal) therefore, **a == b** is unequal. On the other hand, the equal() method always check for the content. Since their content is equal hence, **a equals b** is printed.

157) What is the output of the following Java program?

```
public class Test
{
    public static void main (String args[])
    {
        String s1 = "Sharma is a good player";
        String s2 = new String("Sharma is a good player");
        s2 = s2.intern();
        System.out.println(s1 ==s2);
    }
}
```

Output

```
true
```

Explanation

The intern method returns the String object reference from the string pool. In this case, s1 is created by using string literal whereas, s2 is created by using the String pool. However, s2 is changed to the reference of s1, and the operator **==** returns true.

158) What are the differences between String and StringBuffer?

The differences between the String and StringBuffer is given in the table below.

No.	String	StringBuffer
1)	The String class is immutable.	The StringBuffer class is mutable.
2)	The String is slow and consumes more memory when we concat too many strings because every time it creates a new instance.	The StringBuffer is fast and consumes less memory when you concat strings.
3)	The String class overrides the equals() method of Object class. So we can compare the contents of two strings by equals() method.	The StringBuffer class does not override the equals() method of Object class.

To read more: Difference between String and StringBuffer in Java

159) What are the differences between StringBuffer and StringBuilder?

The differences between the StringBuffer and StringBuilder is given below.

No.	StringBuffer	StringBuilder
1)	StringBuffer is *synchronized*, i.e., thread safe. It means two threads can't call the methods of StringBuffer simultaneously.	StringBuilder is *non-synchronized*,i.e., not thread safe. It means two threads can call the methods of StringBuilder simultaneously.
2)	StringBuffer is *less efficient* than StringBuilder.	StringBuilder is *more efficient* than StringBuffer.

To read more: StringBuffer vs StringBuilder in Java

160) How can we create an immutable class in Java?

We can create an immutable class by defining a final class having all of its members as final. Consider the following example.

1. Make the class final, so it cannot be subclassed.
2. Declare all fields private and final.
3. Do not provide setter methods for the fields.
4. Ensure that any mutable objects within the class are defensively copied to prevent external modification.

5. If the class has mutable fields, make sure to return defensive copies in getter methods.

```
public final class Employee{
final String pancardNumber;

public Employee(String pancardNumber){
this.pancardNumber=pancardNumber;
}

public String getPancardNumber(){
return pancardNumber;
}

}
```

To read more: How to create Immutable Class in Java

161) What is the purpose of the toString() method in Java?

The toString() method is used to obtain a string representation of an object. It is defined in the Object class and can be overridden by custom classes to provide a meaningful representation of the object's state depending on implementation. By overriding the toString() method of the Object class, we can return the values of the object, so we do not need to write much code. Consider the following example.

```
class Student {
 int rollno;
 String name;
 String city;
Student(int rollno, String name, String city){
 this.rollno=rollno;
 this.name=name;
 this.city=city;
 }
public String toString(){    //overriding the toString() method
  return rollno+" "+name+" "+city;
 }
 public static void main(String args[]){
   Student s1=new Student(101,"Raj","Lucknow");
   Student s2=new Student(102,"Vijay","Ghaziabad");
   System.out.println(s1);//compiler writes here s1.toString()
   System.out.println(s2);//compiler writes here s2.toString()
 }
}
```

Output:

```
101 Raj Lucknow
102 Vijay Ghaziabad
```

To read more: Java toString() Method with Examples

162) Why toCharArray() method is preferred over String to store the password?

Corrected

CORRECTED The question is worded confusingly. The real advice is to keep a password in a **CHAR[]** from the start, not to call toCharArray() on a String.

Checked against: Console.readPassword.

A `String` is immutable, so a password stored in one **cannot be wiped**: it stays in memory until the garbage collector reclaims it (longer if it was interned), and could show up in a heap dump. A `char[]` can be **overwritten** as soon as you are done:

```
char[] pwd = console.readPassword("Password: ");
try {
    authenticate(pwd);
} finally {
    Arrays.fill(pwd, ' ');      // wipe it
}
```

- Security APIs follow this rule: `Console.readPassword()` and `JPasswordField.getPassword()` return a `char[]`.
- Calling `toCharArray()` on an existing `String` does not help, because the original `String` still exists. The point is never to create the String in the first place.
- This reduces exposure; it is defence in depth, not a complete guarantee.

163) Write a Java program to count the number of words present in a string?

Program:

```
  public class Test
{
    public static void main (String args[])
    {
        String s = "Sharma is a good player and he is so punctual";
        String words[] = s.split(" ");
        System.out.println("The Number of words present in the string are : "+words.length);
    }
}
```

Output

```
The Number of words present in the string are : 10
```

164) Name some classes present in java.util.regex package.

Corrected

CORRECTED The source called Pattern and Matcher 'abstract classes'. They are **FINAL** classes (confirmed on JDK 21), not abstract.

Checked against: Pattern.

The main types in `java.util.regex`:

- **`Pattern`**: a `final` class that holds a compiled regular expression. It has no public constructor; use `Pattern.compile(regex)`. Instances are immutable and thread-safe.
- **`Matcher`**: a `final` class that matches a pattern against input, created with `pattern.matcher(input)`. It is **not** thread-safe.
- **`MatchResult`**: an interface for the result of a match (implemented by `Matcher`).
- **`PatternSyntaxException`**: an unchecked exception (it extends `IllegalArgumentException`) thrown for an invalid regex.

165) How the metacharacters are different from the ordinary characters?

Metacharacters in regular expressions have special meanings and are used to construct patterns. The metacharacters are ^, $, ., *, +, etc. The regular expression engine does not consider them as the regular characters. To enable the regular expression engine treating the metacharacters as ordinary characters, we need to escape the metacharacters with the backslash.

166) Write a regular expression to validate a password. A password must start with an alphabet and followed by alphanumeric characters. It must 8 to 20 characters long.

Corrected

CORRECTED The source's regex `{8,19}` accepts 9 to 20 characters in total (one first letter plus 8 to 19 more), not the 8 to 20 the question asks for. It also had no `$` to anchor the end. Confirmed by running it on JDK 21.

Checked against: Pattern (quantifiers and anchors).

The regular expression for the above criteria will be:

```
^[a-zA-Z][a-zA-Z0-9]{7,19}$
```

Where,

- **^** represents the start of the regex.
- **[a-zA-Z]** represents that the first character must be an alphabet.
- **[a-zA-Z0-9]** represents the alphanumeric character.
- **{7,19}** means 7 to 19 more characters after the first one, so the whole password is 8 to 20 characters long. The $ anchors the end of the pattern.

167) What is the output of the following Java program?

```
import java.util.regex.*;
class RegexExample2{
public static void main(String args[]){
System.out.println(Pattern.matches(".s", "as")); //line 4
System.out.println(Pattern.matches(".s", "mk")); //line 5
System.out.println(Pattern.matches(".s", "mst")); //line 6
System.out.println(Pattern.matches(".s", "amms")); //line 7
System.out.println(Pattern.matches("..s", "mas")); //line 8
}}
```

Output

```
true
false
false
false
true
```

Explanation

line 4 prints true since the second character of string is s, line 5 prints false since the second character is not s, line 6 prints false since there are more than 3 characters in the string, line 7 prints false since there are more than 2 characters in the string, and it contains more than 2 characters as well, line 8 prints true since the third character of the string is s.

Core Java: Nested classes and Interfaces Interview Questions

Questions in this topic (11)

168) What are the advantages of Java inner classes?
169) What is a nested class?
170) What are the disadvantages of using inner classes?
171) What are the types of inner classes (non-static nested class) used in Java?
172) Is there any difference between nested classes and inner classes?
173) Can we access the non-final local variable, inside the local inner class?
174) How many class files will be created when we compile the outer class of the following program?
175) What are anonymous inner classes?
176) What is the nested interface?
177) Can a class have an interface?
178) Can an Interface have a class?

168) What are the advantages of Java inner classes?

There are following advantages of Java inner classes:

- Encapsulation of logic within a class.
- Improved organization and readability of code.
- Enhanced code modularity.
- Access to the private members of the outer class.
- **Code Optimization:** It requires less code to write.
- Nested classes represent a special type of relationship that is it can access all the members (data members and methods) of the outer class including private.
- Nested classes are used to develop a more readable and maintainable code because it logically groups classes and interfaces in one place only.

To read more: Inner Classes in Java

169) What is a nested class?

The nested class can be defined as the class which is defined inside another class or interface. It can be a static nested class or an inner class (non-static nested class). We use the nested class to logically group classes and interfaces in one place so that it can be more readable and maintainable. A nested class can access all the data members of the outer class including private data members and methods. The syntax of the nested class is defined below.

```
class Java_Outer_class{
 //code
 class Java_Nested_class{
  //code
 }
}
```

There are two types of nested classes, static nested class, and non-static nested class. The non-static nested class can also be called as inner-class.

170) What are the disadvantages of using inner classes?

There are the following disadvantages of using inner classes:

- Increased complexity and reduced readability.
- Larger bytecode size.
- Tight coupling with the outer class.

- Inner classes increase the total number of classes used by the developer and therefore increases the workload of JVM since it has to perform some routine operations for those extra classes which result in slower performance.
- IDEs provide less support to the inner classes as compare to the top level classes and therefore it annoys the developers while working with inner classes.

171) What are the types of inner classes (non-static nested class) used in Java?

There are the following three types of inner classes used in Java.

Type	Description
Member Inner Class	A class created within class and outside method.
Anonymous Inner Class	A class created for implementing an interface or extending class. Its name is decided by the java compiler.
Local Inner Class	A class created within the method.

172) Is there any difference between nested classes and inner classes?

Yes, inner classes are non-static nested classes. In other words, we can say that inner classes are the part of nested classes.

To read more: Difference Between Nested Class and Inner Class in Java

173) Can we access the non-final local variable, inside the local inner class?

No, in Java, a local inner class can access final or effectively final local variables. It cannot access non-final local variables.

More details.

174) How many class files will be created when we compile the outer class of the following program?

```
public class Person {
String name, age, address;
class Employee{
  float salary=10000;
}
class BusinessMen{
  final String gstin="£4433drt3$";
}
public static void main (String args[])
{
  Person p = new Person();
}
}
```

On compiling the above code, we get three class-files named as Person.class, Person$BusinessMen.class, and Person$Employee.class.

175) What are anonymous inner classes?

Anonymous inner classes in Java are unnamed classes defined on the fly, often used for one-time use. They are declared and instantiated at the same time. We cannot apply different access modifiers to them. Anonymous class cannot be static, and cannot define any static fields, method, or class. In other words, we can say that it a class without the name and can have only one object that is created by its definition. Consider the following example.

```
abstract class Person{
  abstract void eat();
}
class TestAnonymousInner{
 public static void main(String args[]){
  Person p=new Person(){
  void eat(){System.out.println("nice fruits");}
  };
  p.eat();
 }
}
```

Output:

```
nice fruits
```

Consider the following example for the working of the anonymous class using interface.

```
interface Eatable{
 void eat();
}
class TestAnnonymousInner1{
 public static void main(String args[]){
 Eatable e=new Eatable(){
  public void eat(){System.out.println("nice fruits");}
 };
 e.eat();
 }
}
```

Output:

```
nice fruits
```

176) What is the nested interface?

An Interface that is declared inside the interface or class is known as the nested interface. A nested interface in Java is an interface declared within another interface or a class. It is static by default. The nested interfaces are used to group related interfaces so that they can be easy to maintain. The external interface or class must refer to the nested interface. It cannot be accessed directly. The nested interface must be public if it is declared inside the interface but it can have any access modifier if declared within the class. The syntax of the nested interface is given as follows.

```
interface interface_name{
 ...
 interface nested_interface_name{
  ...
 }
}
```

More details.

177) Can a class have an interface?

Yes, an interface can be defined within the class. It is called a nested interface. A class in Java can implement multiple interfaces.

More details.

178) Can an Interface have a class?

Yes, we can define a class inside an interface. The class can be either inner or nested class. In general, if the methods of the interface use this class and if we are not using it anywhere else we will declare a class within an interface. Interfaces may contain member type declarations.

A member type declaration in an interface is implicitly static and public. It is permitted to redundantly specify either or both of these modifiers. It is not possible to declare non-static classes inside a Java interface.

```
interface UserChecker {
   Ticket validateUser(Credentials credentials);
   class Credentials {
      // user and password
   }
   class Ticket {
      // some obscure implementation
   }
}
```

Garbage Collection Interview Questions

Questions in this topic (10)

179) What is Garbage Collection?
180) What is gc()?
181) How garbage collection is controlled?
182) How can an object be unreferenced?
183) What is the purpose of the finalize() method?
184) Can an unreferenced object be referenced again?
185) What kind of thread is the Garbage collector thread?
186) What is the difference between final, finally and finalize?
187) What is the purpose of the Runtime class?
188) How will you invoke any external process in Java?

179) What is Garbage Collection?

Corrected

CORRECTED The source said garbage collection 'prevents memory leaks' and that it is the reason uninitialized variables give 0. Both are wrong.

Checked against: JEP 248.

Garbage collection is the JVM's automatic reclaiming of memory used by objects that are no longer **reachable** (no live thread, static field or other reachable object can get to them). It replaces `free()` in C and `delete` in C++, and removes whole classes of bugs such as dangling pointers and double frees.

- It does **not** prevent memory leaks: an object that is still reachable but no longer needed (for example kept in a static collection, a listener list or a ThreadLocal) is never collected.
- It has nothing to do with default values. Fields get `0`, `false` or `null` when the object is created; **local variables** have no default and must be assigned before use.

180) What is gc()?

Corrected

CORRECTED The source said gc() 'is not a standard method'. `System.gc()` and `Runtime.getRuntime().gc()` are standard; they are only a request.

Checked against: System.gc().

`System.gc()` (and `Runtime.getRuntime().gc()`) is a standard method. It is a **request** that the JVM run a garbage collection: the JVM may do it, or ignore it. For example, the flag `-XX:+DisableExplicitGC` makes the JVM ignore the call.

```
Runtime rt = Runtime.getRuntime();
long before = rt.freeMemory();
System.gc();                        // a hint, not a command
long after = rt.freeMemory();
```

Do not call it in production code to 'fix' memory problems; the JVM normally decides better than you when to collect.

181) How garbage collection is controlled?

Corrected

CORRECTED The original is out of date: it says collection happens 'when memory is low' and omits the collectors and flags you actually use.

Checked against: JEP 248.

The **JVM decides when** garbage collection runs (for example when the young generation or a region fills up), and which algorithm does the work. You can influence it, but you cannot command it:

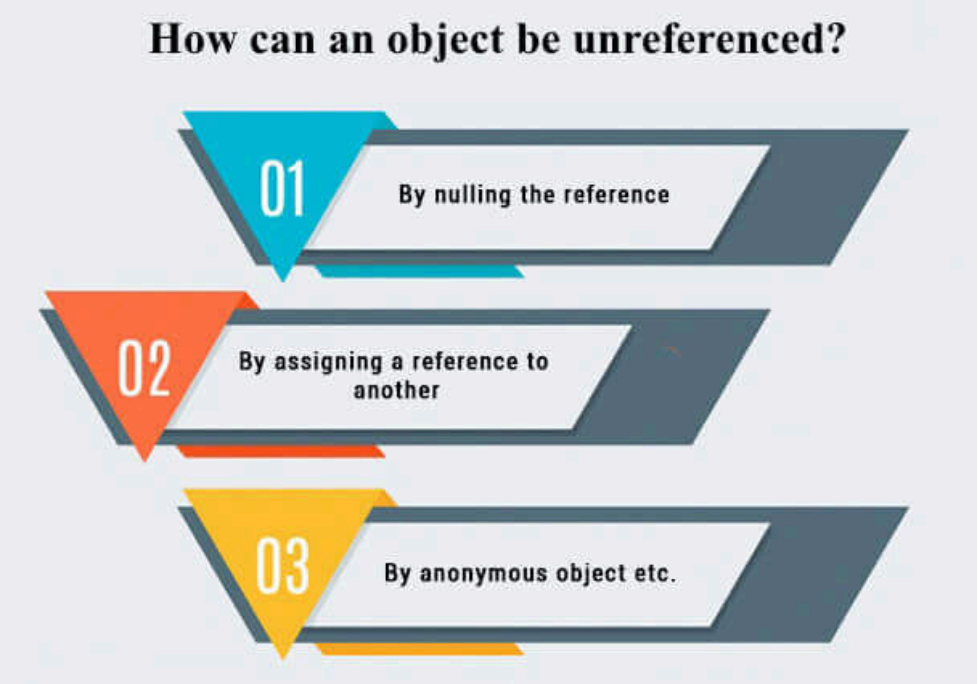
- **Heap size:** `-Xms` and `-Xmx`.
- **Collector:** `-XX:+UseG1GC` (the default since Java 9), `-XX:+UseParallelGC`, `-XX:+UseSerialGC`, `-XX:+UseZGC`.
- **Pause goal:** for example `-XX:MaxGCPauseMillis=200` for G1.
- **Logging:** `-Xlog:gc`.
- `System.gc()` is only a request.

See the modern JVM and GC questions at the end of this app for the collectors and their trade-offs.

182) How can an object be unreferenced?

An object becomes unreferenced when there are no more references to it from any part of the program. In Java, objects are automatically garbage collected when they are no longer reachable, meaning there are no references pointing to them. It can happen in several ways:

- By nulling the reference
- By assigning a reference to another
- By anonymous object etc.



1) By nulling a reference:

```
Employee e=new Employee();
e=null;
```

2) By assigning a reference to another:

```
Employee e1=new Employee();
Employee e2=new Employee();
e1=e2;//now the first object referred by e1 is available for garbage collection
```

3) By anonymous object:

```
new Employee();
```

183) What is the purpose of the finalize() method?

Corrected

> **CORRECTED** The source said finalize() is for cleaning up 'objects created without new' and did not mention that finalization is deprecated for removal. Neither is correct.
>
> Checked against: JEP 421, Object.

`finalize()` is a method of `Object` that the garbage collector may call once before reclaiming an object. It is **deprecated for removal** and should not be used.

- **Unpredictable:** it can run an arbitrarily long time after the object becomes unreachable, or never.
- It can **resurrect** the object by storing `this` somewhere.
- It slows allocation and enables finalization for every instance of the class, and finalizers run on unspecified threads in no set order.
- It is not needed for 'objects created without new': the collector reclaims every unreachable object on the heap.

Use instead: `try-with-resources` (implement `AutoCloseable`) for deterministic cleanup, and `java.lang.ref.Cleaner` only as a safety net.

```
class Connection implements AutoCloseable {
    @Override public void close() { /* release the resource */ }
}

try (Connection c = new Connection()) {
    // use it
}   // close() is called here, always
```

184) Can an unreferenced object be referenced again?

Corrected

> **CORRECTED** The source answered 'Yes' and then explained that there is no way to reference it again, which contradicts itself.
>
> Checked against: JEP 421, Object.

No. Once an object is **unreachable**, your code has no way to get a reference to it again, and the garbage collector is free to reclaim it.

- If you can still reach it through some other live reference, it was never unreferenced.
- **Resurrection** was possible inside `finalize()` (the finalizer could store `this` in a static field). That is one of the reasons finalization is deprecated for removal.
- A `WeakReference` still returns the object until the collector clears it, but a weakly reachable object is not 'unreferenced' yet.

A normal strong reference set to `null` or reassigned makes the object eligible for collection; it cannot be brought back.

185) What kind of thread is the Garbage collector thread?

Daemon thread. The Garbage Collector thread in Java is a special thread responsible for reclaiming memory by removing objects that are no longer reachable. It runs in the background and is managed by the Java Virtual Machine (JVM).

186) What is the difference between final, finally and finalize?

No.	final	finally	finalize
1)	Final is used to apply restrictions on class, method, and variable. The final class cannot be inherited, final method cannot be overridden, and final variable value cannot be changed.	Finally is used to place important code, it will be executed whether an exception is handled or not.	Finalize is used to perform clean up processing just before an object is garbage collected.
2)	Final is a keyword.	Finally is a block.	Finalize is a method.

187) What is the purpose of the Runtime class?

Java Runtime class is used to interact with a Java Runtime Environment. Java Runtime class provides methods to execute a process, invoke GC, get total and free memory, etc. It allows Java applications to interact with the environment in which they are running, such as executing system commands, managing process execution, accessing environment variables, etc. There is only one instance of java.lang.Runtime class is available for one Java application. The Runtime.getRuntime() method returns the singleton instance of Runtime class.

To read more: Java Runtime class

188) How will you invoke any external process in Java?

We can invoke an external process in Java using the ProcessBuilder or Runtime class. Here's an example using ProcessBuilder:

```
public class Runtime1{
 public static void main(String args[])throws Exception{
  Runtime.getRuntime().exec("notepad");//will open a new notepad
 }
}
```

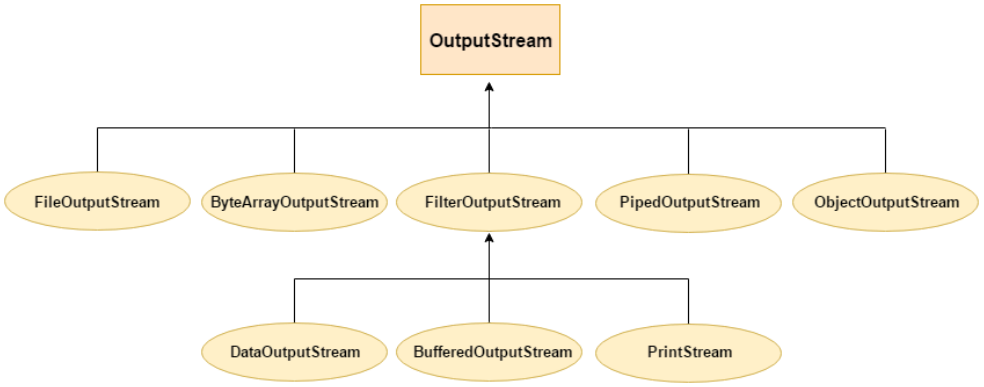
I/O Interview Questions

Questions in this topic (10)

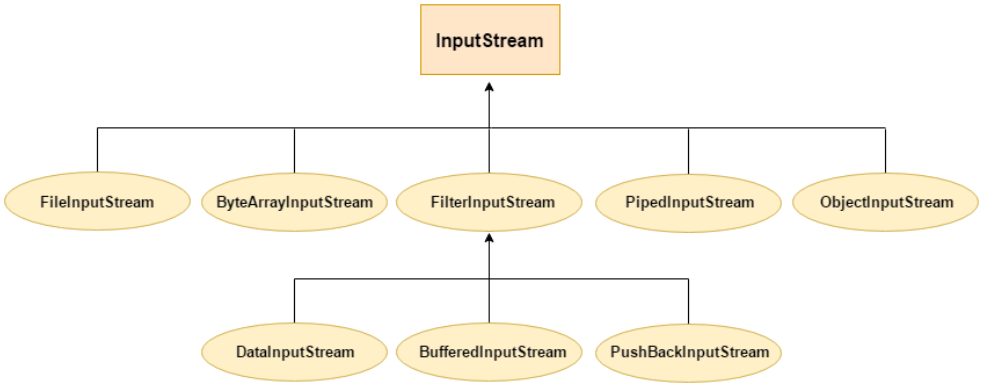
189) Give the hierarchy of InputStream and OutputStream classes.
190) What do you understand by an IO stream?
191) What is the difference between the Reader/Writer class hierarchy and the InputStream/OutputStream class hierarchy?
192) What are the super most classes for all the streams?
193) What are the FileInputStream and FileOutputStream?
194) What is the purpose of using BufferedInputStream and BufferedOutputStream classes?
195) How to set the Permissions to a file in Java?
196) What are FilterStreams?
197) What is an I/O filter?
198) What are the ways to take input from the console?

189) Give the hierarchy of InputStream and OutputStream classes.

OutputStream Hierarchy



InputStream Hierarchy



To read more: Java I/O Tutorial

190) What do you understand by an IO stream?

An I/O stream in Java represents a sequence of data. It is used for input and output operations, allowing data to be read from or written to various sources such as files, network connections, memory, etc. In Java, the following three streams are created automatically.

- System.out: standard output stream
- System.in: standard input stream
- System.err: standard error stream

191) What is the difference between the Reader/Writer class hierarchy and the InputStream/OutputStream class hierarchy?

The Reader/Writer class hierarchy is character-oriented, and the InputStream/OutputStream class hierarchy is byte-oriented. The ByteStream classes are used to perform input-output of 8-bit bytes whereas the CharacterStream classes are used to perform the input/output for the 16-bit Unicode system. There are many classes in the ByteStream class hierarchy, but the most frequently used classes are FileInputStream and FileOutputStream. The most frequently used classes CharacterStream class hierarchy is FileReader and FileWriter.

The Reader/Writer class hierarchy in Java is used for character-based input/output operations. These classes deal with Unicode characters and are suitable for handling text data.	The InputStream/OutputStream class hierarchy in Java is used for byte-based input/output operations. These classes deal with raw bytes and are suitable for handling binary data.

To read more: Hierarchical Inheritance in Java

192) What are the super most classes for all the streams?

All the stream classes can be divided into two types of classes that are ByteStream classes and CharacterStream Classes. The ByteStream classes are further divided into InputStream classes and OutputStream classes. CharacterStream classes are also divided into Reader classes and Writer classes. The super-most classes for all streams are InputStream and OutputStream. These classes define the basic functionality for reading from and writing to streams. Similarly, for all the reader classes, the super-most class is java.io.Reader, and for all the writer classes, it is java.io.Writer.

193) What are the FileInputStream and FileOutputStream?

Java FileOutputStream is an output stream used for writing data to a file. If you have some primitive values to write into a file, use FileOutputStream class. They are subclasses of InputStream and OutputStream, respectively. We can write byte-oriented as well as character-oriented data through the FileOutputStream class. However, for character-oriented data, it is preferred to use FileWriter than FileOutputStream. Consider the following example of writing a byte into a file.

```
import java.io.FileOutputStream;
public class FileOutputStreamExample {
    public static void main(String args[]){
           try{
             FileOutputStream fout=new FileOutputStream("D:\\testout.txt");
             fout.write(65);
             fout.close();
             System.out.println("success...");
            }catch(Exception e){System.out.println(e);}
      }
}
```

Java FileInputStream class obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video, etc. we can also read character-stream data. However, for reading streams of characters, it is recommended to use FileReader class. Consider the following example for reading bytes from a file.

```
import java.io.FileInputStream;
public class DataStreamExample {
     public static void main(String args[]){
          try{
            FileInputStream fin=new FileInputStream("D:\\testout.txt");
            int i=fin.read();
            System.out.print((char)i);

            fin.close();
          }catch(Exception e){System.out.println(e);}
         }
        }
```

To read more: Java FileOutputStream Class

To read more: Java FileInputStream Class

194) What is the purpose of using BufferedInputStream and BufferedOutputStream classes?

In Java, the purpose of using BufferedInputStream and BufferedOutputStream classes is to improve I/O performance by reducing the number of disk or network accesses. It internally uses a buffer to store data. It adds more efficiency than to write data directly into a stream. So, it makes the performance fast. Whereas, Java BufferedInputStream class is used to read information from the stream. It internally uses the buffer mechanism to make the performance fast.

To read more: Java BufferedInputStream Class

To read more: Java BufferedOutputStream Class

195) How to set the Permissions to a file in Java?

In Java, FilePermission class is used to alter the permissions set on a file. Java FilePermission class contains the permission related to a directory or file. We can set permissions to a file using the setReadable(), setWritable(), and setExecutable() methods of the java.io.File class. These methods allow us to set the read, write, and execute permissions, respectively, for the specified file. All the permissions are related to the path. The path can be of two types:

- D:\\IO\\-: It indicates that the permission is associated with all subdirectories and files recursively.
- D:\\IO*: It indicates that the permission is associated with all directory and files within this directory excluding subdirectories.

Let's see the simple example in which permission of a directory path is granted with read permission and a file of this directory is granted for write permission.

```
package com.Tpointtech;
import java.io.*;
import java.security.PermissionCollection;
public class FilePermissionExample{
     public static void main(String[] args) throws IOException {
      String srg = "D:\\IO Package\\java.txt";
      FilePermission file1 = new FilePermission("D:\\IO Package\\-", "read");
      PermissionCollection permission = file1.newPermissionCollection();
      permission.add(file1);
           FilePermission file2 = new FilePermission(srg, "write");
           permission.add(file2);
         if(permission.implies(new FilePermission(srg, "read,write"))) {
           System.out.println("Read, Write permission is granted for the path "+srg );
             }else {
            System.out.println("No Read, Write permission is granted for the path "+srg);            }
     }
}
```

Output:

```
Read, Write permission is granted for the path D:\IO Package\java.txt
```

196) What are FilterStreams?

FilterStream classes are used to add additional functionalities to the other stream classes. FilterStream classes act like an interface which read the data from a stream, filters it, and pass the filtered data to the caller. These are streams that "filter" data as it is being read from or written to an underlying stream. The FilterStream classes provide extra functionalities like adding line numbers to the destination file, etc.

197) What is an I/O filter?

An I/O filter is an object that reads from one stream and writes to another. Usually altering the data in some way as it is passed from one stream to another. Filters can be applied to both input and output streams to perform tasks such as encryption, compression, buffering, etc. Many Filter classes that allows a user to make a chain using multiple input streams. It generates a combined effect on several filters.

198) What are the ways to take input from the console?

In Java, there are three ways by using which, we can take input from the console.

- Using the Scanner class
- Using the BufferedReader class along with System.in
- Using the Console class

Let's brief each.

- **Using BufferedReader Class:** We can take input from the console by wrapping System.in into an InputStreamReader and passing it into the BufferedReader. It provides an efficient reading as the input gets buffered. Consider the following example.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
public class Person
{
    public static void main(String[] args) throws IOException
    {
      System.out.println("Enter the name of the person");
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        String name = reader.readLine();
        System.out.println(name);
    }
}
```

- **Using Scanner Class:** The Java Scanner class breaks the input into tokens using a delimiter that is whitespace by default. It provides many methods to read and parse various primitive values. Java Scanner class is widely used to parse text for string and primitive types using a regular expression. Java Scanner class extends Object class and implements Iterator and Closeable interfaces. Consider the following example.

```
import java.util.*;
public class ScannerClassExample2 {
      public static void main(String args[]){
          String str = "Hello/This is Tpointtech/My name is Abhishek.";
          //Create scanner with the specified String Object
          Scanner scanner = new Scanner(str);
          System.out.println("Boolean Result: "+scanner.hasNextBoolean());
          //Change the delimiter of this scanner
          scanner.useDelimiter("/");
          //Printing the tokenized Strings
          System.out.println("---Tokenizes String---");
        while(scanner.hasNext()){
            System.out.println(scanner.next());
        }
          //Display the new delimiter
          System.out.println("Delimiter used: " +scanner.delimiter());
          scanner.close();
          }
}
```

- **Using Console Class:** The Java Console class is used to get input from the console. It provides methods to read texts and passwords. If we read the password using the Console class, it will not be displayed to the user. The java.io.Console class is attached to the system console internally. The Console class is introduced since 1.5. Consider the following example.

```
import java.io.Console;
class ReadStringTest{
public static void main(String args[]){
Console c=System.console();
System.out.println("Enter your name: ");
String n=c.readLine();
System.out.println("Welcome "+n);
}
}
```

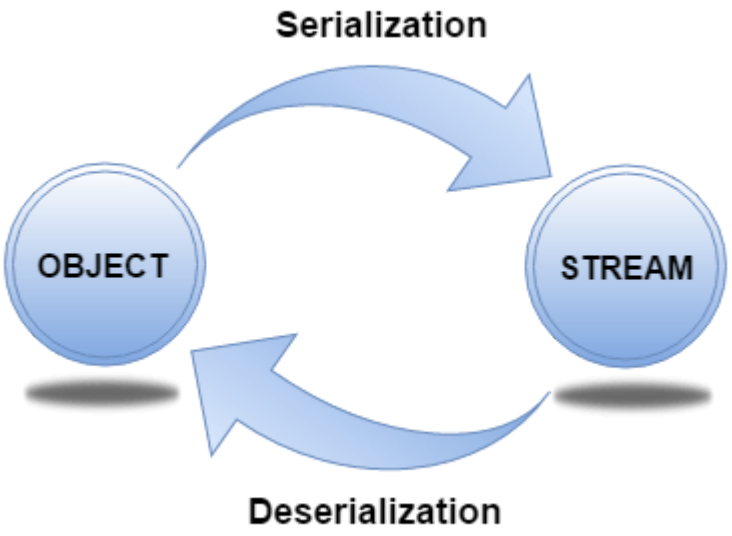
Serialization Interview Questions

Questions in this topic (8)

199) What is serialization?
200) How to make a class serializable in Java?
201) How to avoid serialization in child class if the base class is implementing the Serializable interface?
202) Can a Serialized object be transferred via network?
203) What is Deserialization?
204) What is the use of transient keyword?
205) What is Externalizable?
206) What is the difference between Serializable and Externalizable interface?

199) What is serialization?

Serialization in Java is a mechanism of writing the state of an object into a byte stream. It is used primarily in Hibernate, RMI, JPA, EJB and JMS technologies. It is mainly used to travel object's state on the network (which is known as marshalling). Serializable interface is used to perform serialization. It is helpful when you require to save the state of a program to storage such as the file. At a later point of time, the content of this file can be restored using deserialization. Deserialization is the reverse process of converting the byte stream back into an object. It is also required to implement RMI (Remote Method Invocation). With the help of RMI, it is possible to invoke the method of a Java object on one machine to another machine.



To read more: Serialization in Java

200) How to make a class serializable in Java?

A class can become serializable by implementing the Serializable interface. This interface acts as a marker interface, indicating to the Java runtime that instances of the class can be serialized.

201) How to avoid serialization in child class if the base class is implementing the Serializable interface?

It is very tricky to prevent serialization of child class if the base class is intended to implement the Serializable interface. However, we cannot do it directly, but the serialization can be avoided by implementing the writeObject() or readObject() methods in the subclass and throw NotSerializableException from these methods. Consider the following example.

If a base class implements the Serializable interface and you want to avoid serialization in a child class, you can mark the fields of the child class as transient. his will prevent those fields from being serialized when objects of the child class are serialized, even though the base class implements Serializable.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.NotSerializableException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
class Person implements Serializable
{
    String name = " ";
    public Person(String name)
    {
        this.name = name;
    }
}
class Employee extends Person
{
    float salary;
    public Employee(String name, float salary)
    {
        super(name);
        this.salary = salary;
    }
    private void writeObject(ObjectOutputStream out) throws IOException
    {
        throw new NotSerializableException();
    }
    private void readObject(ObjectInputStream in) throws IOException
    {
        throw new NotSerializableException();
    }

}
public class Test
{
    public static void main(String[] args)
            throws Exception
    {
        Employee emp = new Employee("Sharma", 10000);

        System.out.println("name = " + emp.name);
        System.out.println("salary = " + emp.salary);

        FileOutputStream fos = new FileOutputStream("abc.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);

        oos.writeObject(emp);

        oos.close();
        fos.close();

        System.out.println("Object has been serialized");

        FileInputStream f = new FileInputStream("ab.txt");
        ObjectInputStream o = new ObjectInputStream(f);

        Employee emp1 = (Employee)o.readObject();

        o.close();
        f.close();

        System.out.println("Object has been deserialized");

        System.out.println("name = " + emp1.name);
        System.out.println("salary = " + emp1.salary);
```

```
    }
}
```

202) Can a Serialized object be transferred via network?

Yes, a serialized object can be transferred via a network. Once an object is serialized into a stream of bytes and can be transmitted over the network. We can also write the serialized object to the disk or the database.

203) What is Deserialization?

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. Deserialization involves reading the serialized data from a file, network stream, or any other source, and reconstructing the original object from that data. An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

```
import java.io.*;
class Depersist{
 public static void main(String args[])throws Exception{

  ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
  Student s=(Student)in.readObject();
  System.out.println(s.id+" "+s.name);

  in.close();
 }
}
```

Output:

```
211 ravi
```

To read more: Difference Between Serialization and Deserialization in Java

204) What is the use of transient keyword?

In Java, the transient keyword is used to indicate that a field should not be serialized when the object is serialized. By determining transient keyword, the value of variable need not persist when it is restored. More details.

To read more: Java transient Keyword

205) What is Externalizable?

The Externalizable interface is used to write the state of an object into a byte stream in a compressed format. It allows more control over the serialization and deserialization process compared to Serializable. Classes that implement Externalizable must provide implementations for the writeExternal() and readExternal() methods, which are called during serialization and deserialization, respectively. It is not a marker interface.

206) What is the difference between Serializable and Externalizable interface?

No.	Serializable	Externalizable
1)	The Serializable interface does not have any method, i.e., it is a marker interface.	The Externalizable interface contains is not a marker interface, It contains two methods, i.e., writeExternal() and readExternal().
2)	It is used to "mark" Java classes so that objects of these classes may get the certain capability.	The Externalizable interface provides control of the serialization logic to the programmer.

3)	It is easy to implement but has the higher performance cost.	It is used to perform the serialization and often result in better performance.
4)	No class constructor is called in serialization.	We must call a public default constructor while using this interface.

To read more: Serialization and Externalizable Interface

Networking Interview Questions

Questions in this topic (5)

207) Give a brief description of Java socket programming?
208) What is Socket?
209) What are the steps that are followed when two computers connect through TCP?
210) Write a program in Java to establish a connection between client and server?
211) How do I convert a numeric IP address like 192.18.97.39 into a hostname like java.sun.com?

207) Give a brief description of Java socket programming?

Java Socket programming is used for communication between the applications running on different JRE. Java Socket programming can be connection-oriented or connectionless. Socket and ServerSocket classes are used for connection-oriented socket programming and DatagramSocket, and DatagramPacket classes are used for connectionless socket programming. The client in socket programming must know two information:

- IP Address of the Server
- Port Number

To read more: Java Socket Programming

208) What is Socket?

A socket is simply an endpoint for communications between the machines. It provides the connection mechanism to connect the two computers using TCP. Sockets allow data to be sent and received between client and server processes. The Socket class can be used to create a socket.

209) What are the steps that are followed when two computers connect through TCP?

There are the following steps that are performed when two computers connect through TCP.

- **Server Socket Creation:** The ServerSocket object is instantiated by the server which denotes the port number to which, the connection will be made.
- **Listening for Connections:** After instantiating the ServerSocket object, the server invokes accept() method of ServerSocket class which makes server wait until the client attempts to connect to the server on the given port.
- **Client Socket Creation:** Meanwhile, the server is waiting, a socket is created by the client by instantiating Socket class. The socket class constructor accepts the server port number and server name.
- **Server Accepts Connection:** The Socket class constructor attempts to connect with the server on the specified name. If the connection is established, the client will have a socket object that can communicate with the server.
- **Connection Established:** The accept() method invoked by the server and returns a reference to the new socket on the server that is connected with the server.

210) Write a program in Java to establish a connection between client and server?

Consider the following program where the connection between the client and server is established.

File: MyServer.java

```
import java.io.*;
import java.net.*;
public class MyServer {
public static void main(String[] args){
try{
ServerSocket ss=new ServerSocket(6666);
Socket s=ss.accept();//establishes connection
DataInputStream dis=new DataInputStream(s.getInputStream());
String  str=(String)dis.readUTF();
System.out.println("message= "+str);
ss.close();
}catch(Exception e){System.out.println(e);}
}
}
```

File: MyClient.java

```
import java.io.*;
import java.net.*;
public class MyClient {
public static void main(String[] args) {
try{
Socket s=new Socket("localhost",6666);
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
dout.writeUTF("Hello Server");
dout.flush();
dout.close();
s.close();
}catch(Exception e){System.out.println(e);}
}
}
```

211) How do I convert a numeric IP address like 192.18.97.39 into a hostname like java.sun.com?

To convert a numeric IP address like "192.18.97.39" into a hostname like "java.sun.com", we can use the InetAddress class in Java. By using the InetAddress.getByName("192.18.97.39").getHostName() method, we get the hostname. Consider the following example.

```
import java.io.*;
import java.net.*;
public class InetDemo{
public static void main(String[] args){
try{
InetAddress ip=InetAddress.getByName("195.201.10.8");

System.out.println("Host Name: "+ip.getHostName());
}catch(Exception e){System.out.println(e);}
}
}
```

Reflection Interview Questions

Questions in this topic (6)

212) What is the reflection?
213) What is the purpose of using java.lang.Class class?
214) What are the ways to instantiate the Class class?
215) What is the output of the following Java program?
216) What is the purpose of using javap?
217) Can we access the private method from outside the class?

212) What is the reflection?

Reflection refers to the ability of a program to inspect and modify its own structure and behavior at runtime. The java.lang.Class class provides various methods that can be used to get metadata, examine and change the runtime behavior of a class. It allows us to analyze the capabilities of classes, interfaces, and objects, as well as dynamically invoke methods, access fields, and create new objects. The java.lang and java.lang.reflect packages provide classes for Java reflection. It is used in:

- IDE (Integrated Development Environment). Like, Eclipse, MyEclipse, NetBeans.
- Debugger
- Test Tools, etc.

To read more: Java Reflection API

213) What is the purpose of using java.lang.Class class?

The java.lang.Class class performs mainly two tasks:

- Provides methods to get the metadata of a class at runtime.
- Provides methods to examine and change the runtime behavior of a class.

214) What are the ways to instantiate the Class class?

There are three ways to instantiate the Class class.

- **Using the .class syntax:** Class<?> clazz = MyClass.class;
- **Using the getClass() method:** Class<?> clazz = obj.getClass();
- **Using the forName() method:** Class<?> clazz = Class.forName("fully.qualified.ClassName");

Using Class.forName() Method: The forName() method is used to load the class dynamically. It returns the instance of Class class. It should be used if we know the fully qualified name of the class. This cannot be used for primitive types.

Using Object.getClass() Method: It returns the instance of Class class. It should be used if we know the type. Moreover, it can be used with primitives.

Using .class Syntax: If a type is available, but there is no instance then it is possible to obtain a Class by appending ".class" to the name of the type. It can be used for primitive data type also.

215) What is the output of the following Java program?

```
class Simple{
 public Simple()
 {
   System.out.println("Constructor of Simple class is invoked");
 }
 void message(){System.out.println("Hello Java");}
}

class Test1{
 public static void main(String args[]){
  try{
  Class c=Class.forName("Simple");
  Simple s=(Simple)c.newInstance();
  s.message();
  }catch(Exception e){System.out.println(e);}
 }
}
```

Output

```
Constructor of Simple class is invoked
Hello Java
```

Explanation

The newInstance() method of the Class class is used to invoke the constructor at runtime. In this program, the instance of the Simple class is created.

216) What is the purpose of using javap?

The javap command disassembles a class file. It prints information about the fields, methods, and constructors of a class in human-readable format. javap can be useful for understanding the internal structure of Java classes and analyzing bytecode.

Syntax

```
javap fully_class_name
```

217) Can we access the private method from outside the class?

No, we cannot access a private method from outside the class in Java. Private methods are accessible only within the class in which they are declared. They cannot be accessed from subclasses or from other classes in the same package.

More details.

Miscellaneous Interview Questions

Questions in this topic (19)

218) What are wrapper classes?
219) What are auto-boxing and unboxing? When does it occur?
220) What is the output of the following Java program?
221) What is object cloning?
222) What are the advantages and disadvantages of object cloning?
223) What is a native method?
224) What is the purpose of the strictfp keyword?
225) What is the purpose of the System class?
226) What comes to mind when someone mentions a shallow copy in Java?
227) What is a singleton class?
228) Write a Java program that prints all the values given at command-line.
229) Which containers use a border layout as their default layout?
230) Which containers use a FlowLayout as their default layout?
231) What are peerless components?
232) Is there is any difference between a Scrollbar and a ScrollPane?
233) What is a lightweight component?
234) What is a heavyweight component?
235) What is an applet?
236) Can you write a Java class that could be used both as an applet as well as an application?

218) What are wrapper classes?

In Java, wrapper classes are the classes that allow primitive types to be accessed as objects. In other words, we can say that wrapper classes are built-in Java classes that allows the conversion of objects to primitives and primitives to objects. The process of converting primitives to objects is called auto-boxing, and the process of converting objects to primitives is called unboxing. There are eight wrapper classes present in java.lang package is given below.

Primitive Type	Wrapper class
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

To read more: Wrapper classes in Java

219) What are auto-boxing and unboxing? When does it occur?

The auto-boxing is the process of converting primitive data type to the corresponding wrapper class object. For example, int to Integer.

The unboxing is the process of converting wrapper class object to primitive data type. For example, Integer to int.

Unboxing and auto-boxing occur automatically in Java. However, we can externally convert one into another by using the methods like valueOf() or xxxValue().

- It can occur whenever a wrapper class object is expected, and primitive data type is provided or vice versa.
- Adding primitive types into Collection like ArrayList in Java.
- Creating an instance of parameterized classes. For example, ThreadLocal which expect Type.
- Java automatically converts primitive to object whenever one is required and another is provided in the method calling.
- When a primitive type is assigned to an object type.

220) What is the output of the following Java program?

```
public class Test1
{
  public static void main(String[] args) {
  Integer i = new Integer(201);
  Integer j = new Integer(201);
  if(i == j)
  {
    System.out.println("hello");
  }
  else
  {
    System.out.println("bye");
  }
  }
}
```

Output

```
bye
```

Explanation

The Integer class caches integer values from -127 to 127. Therefore, the Integer objects can only be created in the range -128 to 127. The operator **==** will not work for the value greater than 127; thus **bye** is printed.

221) What is object cloning?

Object cloning refers to the process of creating an exact copy of an object. The clone() method of the Object class is used to clone an object. The java.lang.Cloneable interface must be implemented by the class whose object clone we want to create. Object cloning creates a new object with the same state as the original object, but the two objects are independent of each other. If we do not implement Cloneable interface, the clone() method generates CloneNotSupportedException. The clone() method is defined in the Object class. The syntax of the clone() method is as follows:

protected Object clone() throws CloneNotSupportedException

222) What are the advantages and disadvantages of object cloning?

Advantage of Object Cloning

- We do not need to write lengthy and repetitive codes. Just use an abstract class with a 4 or 5-line long clone() method.
- It is the easiest and most efficient way of copying objects, especially if we are applying it to an already developed or an old project. Just define a parent class, implement Cloneable in it, provide the definition of the clone() method and the task will be done.
- Using the clone() method is the fastest way to copy the array.

Disadvantage of Object Cloning

- To use the Object.clone() method, we have to change many syntaxes to our code, like implementing a Cloneable interface, defining the clone() method and handling CloneNotSupportedException, and finally, calling Object.clone(), etc.

- We have to implement the Cloneable interface while it does not have any methods in it. We have to use it to tell the JVM that we can perform a clone() on our object.
- clone() is protected, so we have to provide our own clone() and indirectly call Object.clone() from it.
- clone() does not invoke any constructor, so we do not have any control over object construction.
- If we want to write a clone method in a child class, then all of its super classes should define the clone() method in them or inherit it from another parent class. Otherwise, the super.clone() chain will fail.
- clone() method supports only shallow copying, but we will need to override it if we need deep cloning.

To read more: Object Cloning in Java

223) What is a native method?

A native method is a method that is implemented in a language other than Java. Natives methods are sometimes also referred to as foreign methods. Native methods are declared using the native keyword, and their implementations are provided by native libraries or external sources.

224) What is the purpose of the strictfp keyword?

Java strictfp keyword ensures that we will get the same result on every platform if we perform operations in the floating-point variable. The precision may differ from platform to platform that is why Java programming language has provided the strictfp keyword so that we get the same result on every platform. It ensures consistent behavior of floating-point operations across different JVM implementations. Therefore, we have better control over the floating-point arithmetic.

To read more: Strictfp Keyword in Java

225) What is the purpose of the System class?

The purpose of the System class is to provide access to system resources such as standard input and output. It cannot be instantiated. Facilities provided by System class are given below.

- Standard input
- Error output streams
- Standard output
- utility method to copy the portion of an array
- utilities to load files and libraries

There are the three fields of Java System class, i.e., static printstream err, static inputstream in, and standard output stream.

226) What comes to mind when someone mentions a shallow copy in Java?

Object cloning. When someone mentions a shallow copy in Java, they are referring to a copy operation that creates a new object and copies the values of the fields from the original object to the new object.

227) What is a singleton class?

A singleton class in Java is a class that is restricted to having only one instance throughout the lifetime of the application. To make a class singleton, we either make its constructor private or use the static getInstance() method. Consider the following example.

```
class Singleton{
    private static Singleton single_instance = null;
    int i;
     private Singleton ()
     {
         i=90;
     }
     public static Singleton getInstance()
     {
         if(single_instance == null)
         {
             single_instance = new Singleton();
         }
         return single_instance;
     }
}
public class Main
{
    public static void main (String args[])
    {
        Singleton first = Singleton.getInstance();
        System.out.println("First instance integer value:"+first.i);
        first.i=first.i+90;
        Singleton second = Singleton.getInstance();
        System.out.println("Second instance integer value:"+second.i);
    }
}
```

To read more: Singleton Class in Java

228) Write a Java program that prints all the values given at command-line.

Program

```
class A{
public static void main(String args[]){

for(int i=0;i<args.length;i++)
System.out.println(args[i]);

}
}
```

```
compile by > javac A.java
run by > java A sonoo jaiswal 1 3 abc
```

Output

```
sonoo
jaiswal
1
3
abc
```

229) Which containers use a border layout as their default layout?

The Window, Frame and Dialog classes use a border layout as their default layout. Containers that use a BorderLayout as their default layout include JFrame, JDialog, JWindow, and Applet.

230) Which containers use a FlowLayout as their default layout?

Containers that use a FlowLayout as their default layout include JPanel and JApplet.

231) What are peerless components?

The lightweight component of Swing is called peerless components. Peerless components in Java Swing are components that do not have a native operating system peer. The Spring framework own its libraries, so it does not use resources from the Operating System, and hence it has lightweight components.

232) Is there is any difference between a Scrollbar and a ScrollPane?

Yes, there is a difference between a Scrollbar and a ScrollPane: The Scrollbar is a Component whereas the ScrollPane is a Container. A ScrollPane handles its events and performs its scrolling.

233) What is a lightweight component?

A lightweight component in Java Swing is a component that is drawn entirely by Java code and does not have a native operating system peer. It does not go with the native call to obtain the graphical units. They share their parent component graphical units to render them. For example, Swing components, and JavaFX Components.

234) What is a heavyweight component?

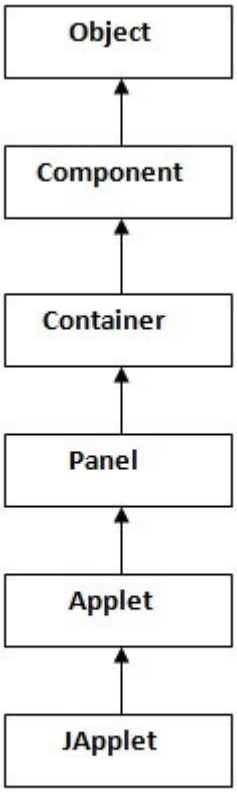
A heavyweight component in Java Swing is a component that has a native operating system peer. The portable elements provided by the operating system are called heavyweight components. AWT is limited to the graphical classes provided by the operating system and therefore, it implements only the minimal subset of screen elements supported by all platforms. The Operating system dependent UI discovery tools are called heavyweight components.

235) What is an applet?

Updated

> **UPDATED** Added: applets are obsolete.
>
> Checked against: JEP 398.

An applet is a small java program that runs inside the browser and generates dynamic content. It is embedded in the webpage and runs on the client side. It is secured and takes less response time. It can be executed by browsers running under many platforms, including Linux, Windows, macOS, etc. They were popular in the early days of the internet but have been largely replaced by other web technologies such as JavaScript and HTML5. However, the plugins are required at the client browser to execute the applet. The following image shows the architecture of Applet.



When an applet is created, the following methods are invoked in order.

- init()
- start()
- paint()

When an applet is destroyed, the following functions are invoked in order.

- stop()
- destroy()

To read more: Java Applet

Applets are no longer practical: every browser removed support for the Java plug-in. The Applet API was deprecated in Java 9 and **deprecated for removal in Java 17**. Do not use applets in new code.

236) Can you write a Java class that could be used both as an applet as well as an application?

Yes. we can write a Java class that could be used both as an applet as well as an application. Add a main() method to the applet.

Internationalization Interview Questions

Questions in this topic (2)

237) What is Locale?
238) How will you load a specific locale?

237) What is Locale?

A Locale object represents a specific geographical, political, or cultural region. Locale objects are used to specify formats for dates, numbers, currencies, and messages to be displayed to users in their preferred language and format. This object can be used to get the locale-specific information such as country name, language, variant, etc.

```
import java.util.*;
public class LocaleExample {
public static void main(String[] args) {
Locale locale=Locale.getDefault();
//Locale locale=new Locale("fr","fr");//for the specific locale

System.out.println(locale.getDisplayCountry());
System.out.println(locale.getDisplayLanguage());
System.out.println(locale.getDisplayName());
System.out.println(locale.getISO3Country());
System.out.println(locale.getISO3Language());
System.out.println(locale.getLanguage());
System.out.println(locale.getCountry());

}
}
```

Output:

```
United States
English
English (United States)
USA
eng
en
US
```

238) How will you load a specific locale?

One can load specific locale by using ResourceBundle.getBundle(?) method.

Java Bean Interview Questions

Questions in this topic (3)

239) What is a JavaBean?
240) What is the purpose of using the Java bean?
241) What do you understand by the bean persistent property?

239) What is a JavaBean?

JavaBean is a reusable software component written in the Java programming language, designed to be manipulated visually by a software development environment, like JBuilder or VisualAge for Java. JavaBeans follow a set of conventions, such as providing getter and setter methods for accessing properties, being serializable, and providing a no-argument constructor. A JavaBean encapsulates many objects into one object so that we can access this object from multiple places. Moreover, it provides the easy maintenance. Consider the following example to create a JavaBean class.

Employee.java

```
//Employee.java
package mypack;
public class Employee implements java.io.Serializable{
private int id;
private String name;
public Employee(){}
public void setId(int id){this.id=id;}
public int getId(){return id;}
public void setName(String name){this.name=name;}
public String getName(){return name;}
}
```

To read more: JavaBean

240) What is the purpose of using the Java bean?

According to Java white paper, it is a reusable software component. A bean encapsulates many objects into one object so that we can access this object from multiple places. Java beans follow a standard naming convention and design patterns, making them interoperable with various development tools and frameworks. Moreover, it provides the easy maintenance.

241) What do you understand by the bean persistent property?

The persistence property of Java bean comes into the act when the properties, fields, and state information are saved to or retrieve from the storage. This persistence is typically achieved by storing the property's value in a persistent storage mechanism such as a database, file system, or remote storage service. Bean persistent properties are commonly used in enterprise applications to store and retrieve configuration settings, user preferences, and other application state data.

RMI Interview Questions

Questions in this topic (6)

242) What is RMI?
243) What is the purpose of stub and skeleton?
244) What are the steps involved to write RMI based programs?
245) What is the use of HTTP-tunneling in RMI?
246) What is JRMP?
247) Can RMI and CORBA based applications interact?

242) What is RMI?

The RMI (Remote Method Invocation) is an API that provides a mechanism to create the distributed application in Java. The RMI allows an object to invoke methods on an object running in another JVM. It enables communication between Java applications running on different machines. The RMI provides remote communication between the applications using two objects stub and skeleton.

To read more: RMI

243) What is the purpose of stub and skeleton?

Stub

A stub is a client-side proxy object that represents a remote object. It acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes the method on the stub object, it does the following tasks:

- It initiates a connection with remote Virtual Machine (JVM).
- It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM).
- It waits for the result.
- It reads (unmarshals) the return value or exception.
- It finally, returns the value to the caller.

Skeleton

A skeleton is a server-side object that acts as an intermediary between the server application and the remote object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

- It reads the parameter for the remote method.
- It invokes the method on the actual remote object.
- It writes and transmits (marshals) the result to the caller.

244) What are the steps involved to write RMI based programs? Updated

UPDATED Added: the rmic step is obsolete.

Checked against: Oracle JDK migration guide: removed tools.

There are the following six steps which are performed to write RMI based programs.

- Create the remote interface.
- Provide the implementation of the remote interface.
- Compile the implementation class and create the stub and skeleton objects using the rmic tool.
- Start the registry service by the rmiregistry tool.
- Create and start the remote application.
- Create and start the client application.

Generating stubs with `rmic` is no longer needed. RMI can generate **stubs dynamically** at runtime, `rmic` was deprecated for removal in JDK 13 and **removed in JDK 15** (Oracle's migration guide). Today you export the remote object with `UnicastRemoteObject.exportObject()` (or by extending `UnicastRemoteObject`) and register it in the registry.

245) What is the use of HTTP-tunneling in RMI?

HTTP tunnelling in RMI is a technique used to allow RMI communication over HTTP protocols, such as HTTP or HTTPS. It can be defined as the method that does not need any setup to work within the firewall environment. It handles the HTTP connections through the proxy servers. However, it does not allow outbound TCP connections.

246) What is JRMP?

JRMP (Java Remote Method Protocol) can be defined as the Java-specific, stream-based protocol which looks up and refers to the remote objects. It requires both client and server to use Java objects. JRMP operates over TCP/IP and is responsible for marshalling and un-marshalling method parameters, managing remote object references, and handling communication between Java Virtual Machines (JVMs). It is wire level protocol which runs under RMI and over TCP/IP.

247) Can RMI and CORBA based applications interact?

Yes, they can. RMI is available with IIOP as the transport protocol instead of JRMP. For example, the RMI/IIOP (RMI over IIOP) bridge allows RMI-based Java applications to communicate with CORBA-based applications using the Internet Inter-ORB Protocol (IIOP). Similarly, the CORBA/IIOP-to-RMI bridge enables CORBA-based applications to invoke RMI-based Java objects.

Core Java: Data Structure Interview Questions

Questions in this topic (9)

248) How to perform Bubble Sort in Java?
249) How to perform Binary Search in Java?
250) How to perform Selection Sort in Java?
251) How to perform Linear Search in Java?
252) How to perform merge sort in Java?
253) How to perform quicksort in Java?
254) Write a program in Java to create a doubly linked list containing n nodes.
255) Write a program in Java to find the maximum and minimum value node from a circular linked list.
256) Write a program in Java to calculate the difference between the sum of the odd level and even level nodes of a Binary Tree.

248) How to perform Bubble Sort in Java?

Consider the following program to perform Bubble sort in Java.

```
public class BubbleSort {
  public static void main(String[] args) {
  int[] a = {10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
  for(int i=0;i<10;i++)
  {
    for (int j=0;j<10;j++)
    {
      if(a[i]<a[j])
      {
        int temp = a[i];
        a[i]=a[j];
        a[j] = temp;
      }
    }
  }
  System.out.println("Printing Sorted List ...");
  for(int i=0;i<10;i++)
  {
    System.out.println(a[i]);
  }
}
}
```

Output:

```
Printing Sorted List . . .
7
9
10
12
23
34
34
44
78
101
```

To read more: Bubble Sort in Java

249) How to perform Binary Search in Java?

Consider the following program to perform the binary search in Java.

```
import java.util.*;
public class BinarySearch {
public static void main(String[] args) {
  int[] arr = {16, 19, 20, 23, 45, 56, 78, 90, 96, 100};
  int item, location = -1;
  System.out.println("Enter the item which you want to search");
  Scanner sc = new Scanner(System.in);
  item = sc.nextInt();
  location = binarySearch(arr,0,9,item);
  if(location != -1)
  System.out.println("the location of the item is "+location);
  else
    System.out.println("Item not found");
  }
public static int binarySearch(int[] a, int beg, int end, int item)
{
  int mid;
  if(end >= beg)
  {
    mid = (beg + end)/2;
    if(a[mid] == item)
    {
      return mid+1;
    }
    else if(a[mid] < item)
    {
      return binarySearch(a,mid+1,end,item);
    }
    else
    {
      return binarySearch(a,beg,mid-1,item);
    }
  }
  return -1;
}
}
```

Output:

```
Enter the item which you want to search
45
the location of the item is 5
```

To read more: Binary Search in Java

250) How to perform Selection Sort in Java?

Consider the following program to perform selection sort in Java.

```
public class SelectionSort {
public static void main(String[] args) {
  int[] a = {10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
  int i,j,k,pos,temp;
  for(i=0;i<10;i++)
  {
    pos = smallest(a,10,i);
    temp = a[i];
    a[i]=a[pos];
    a[pos] = temp;
  }
  System.out.println("\nprinting sorted elements...\n");
  for(i=0;i<10;i++)
  {
    System.out.println(a[i]);
  }
}
public static int smallest(int a[], int n, int i)
{
  int small,pos,j;
  small = a[i];
  pos = i;
  for(j=i+1;j<10;j++)
  {
    if(a[j]<small)
    {
      small = a[j];
      pos=j;
    }
  }
  return pos;
}
}
```

Output:

```
printing sorted elements...
7
9
10
12
23
23
34
44
78
101
```

To read more: Selection Sort in Java

251) How to perform Linear Search in Java?

Consider the following program to perform Linear search in Java.

```
import java.util.Scanner;
public class Leniear_Search {
public static void main(String[] args) {
  int[] arr = {10, 23, 15, 8, 4, 3, 25, 30, 34, 2, 19};
  int item,flag=0;
  Scanner sc = new Scanner(System.in);
  System.out.println("Enter Item?");
  item = sc.nextInt();
  for(int i = 0; i<10; i++)
  {
    if(arr[i]==item)
    {
      flag = i+1;
      break;
    }
    else
      flag = 0;
  }
  if(flag != 0)
  {
    System.out.println("Item found at location" + flag);
  }
  else
    System.out.println("Item not found");
}
}
```

Output:

```
Enter Item?
23
Item found at location 2
Enter Item?
22
Item not found
```

To read more: Linear Search in Java

252) How to perform merge sort in Java?

Consider the following program to perform merge sort in Java.

```
public class MyMergeSort
{
void merge(int arr[], int beg, int mid, int end)
{

int l = mid - beg + 1;
int r = end - mid;

intLeftArray[] = new int [l];
intRightArray[] = new int [r];

for (int i=0; i<l; ++i)
LeftArray[i] = arr[beg + i];

for (int j=0; j<r; ++j)
RightArray[j] = arr[mid + 1+ j];


int i = 0, j = 0;
int k = beg;
while (i<l&&j<r)
{
if (LeftArray[i] <= RightArray[j])
{
arr[k] = LeftArray[i];
i++;
}
else
{
arr[k] = RightArray[j];
j++;
}
k++;
}
while (i<l)
{
arr[k] = LeftArray[i];
i++;
k++;
}

while (j<r)
{
arr[k] = RightArray[j];
j++;
k++;
}
}

void sort(int arr[], int beg, int end)
{
if (beg<end)
{
int mid = (beg+end)/2;
sort(arr, beg, mid);
sort(arr , mid+1, end);
merge(arr, beg, mid, end);
}
}
public static void main(String args[])
{
intarr[] = {90,23,101,45,65,23,67,89,34,23};
MyMergeSort ob = new MyMergeSort();
ob.sort(arr, 0, arr.length-1);
```

```
System.out.println("\nSorted array");
for(int i =0; i<arr.length;i++)
{
  System.out.println(arr[i]+"");
}
}
}
```

Output:

```
Sorted array
23
23
23
34
45
65
67
89
90
101
```

To read more: Merge Sort in Java

253) How to perform quicksort in Java?

Consider the following program to perform quicksort in Java.

```
public class QuickSort {
public static void main(String[] args) {
    int i;
    int[] arr={90,23,101,45,65,23,67,89,34,23};
    quickSort(arr, 0, 9);
    System.out.println("\n The sorted array is: \n");
    for(i=0;i<10;i++)
    System.out.println(arr[i]);
  }
  public static int partition(int a[], int beg, int end)
  {

    int left, right, temp, loc, flag;
    loc = left = beg;
    right = end;
    flag = 0;
    while(flag != 1)
    {
      while((a[loc] <= a[right]) && (loc!=right))
      right--;
      if(loc==right)
      flag =1;
      elseif(a[loc]>a[right])
      {
        temp = a[loc];
        a[loc] = a[right];
        a[right] = temp;
        loc = right;
      }
      if(flag!=1)
      {
        while((a[loc] >= a[left]) && (loc!=left))
        left++;
        if(loc==left)
        flag =1;
        elseif(a[loc] <a[left])
        {
          temp = a[loc];
          a[loc] = a[left];
          a[left] = temp;
          loc = left;
        }
      }
    }
    returnloc;
  }
  static void quickSort(int a[], int beg, int end)
  {

    int loc;
    if(beg<end)
    {
      loc = partition(a, beg, end);
      quickSort(a, beg, loc-1);
      quickSort(a, loc+1, end);
    }
  }
}
```

Output:

```
The sorted array is:
23
23
23
34
45
65
67
89
90
101
```

254) Write a program in Java to create a doubly linked list containing n nodes.

Consider the following program to create a doubly linked list containing n nodes.

```
public class CountList {

  //Represent a node of the doubly linked list

  class Node{
      int data;
      Node previous;
      Node next;

      public Node(int data) {
          this.data = data;
      }
  }

  //Represent the head and tail of the doubly linked list
  Node head, tail = null;

  //addNode() will add a node to the list
  public void addNode(int data) {
      //Create a new node
      Node newNode = new Node(data);

      //If list is empty
      if(head == null) {
          //Both head and tail will point to newNode
          head = tail = newNode;
          //head's previous will point to null
          head.previous = null;
          //tail's next will point to null, as it is the last node of the list
          tail.next = null;
      }
      else {
          //newNode will be added after tail such that tail's next will point to newNode
          tail.next = newNode;
          //newNode's previous will point to tail
          newNode.previous = tail;
          //newNode will become new tail
          tail = newNode;
          //As it is last node, tail's next will point to null
          tail.next = null;
      }
  }

  //countNodes() will count the nodes present in the list
  public int countNodes() {
      int counter = 0;
      //Node current will point to head
      Node current = head;

      while(current != null) {
          //Increment the counter by 1 for each node
          counter++;
          current = current.next;
      }
      return counter;
  }

  //display() will print out the elements of the list
  public void display() {
      //Node current will point to head
      Node current = head;
      if(head == null) {
          System.out.println("List is empty");
          return;
      }
```

```
        System.out.println("Nodes of doubly linked list: ");
        while(current != null) {
            //Prints each node by incrementing the pointer.

            System.out.print(current.data + " ");
            current = current.next;
        }
    }

    public static void main(String[] args) {

        CountList dList = new CountList();
        //Add nodes to the list
        dList.addNode(1);
        dList.addNode(2);
        dList.addNode(3);
        dList.addNode(4);
        dList.addNode(5);

        //Displays the nodes present in the list
        dList.display();

        //Counts the nodes present in the given list
        System.out.println("\nCount of nodes present in the list: " + dList.countNodes());
    }
}
```

Output:

```
Nodes of doubly linked list:
1 2 3 4 5
Count of nodes present in the list: 5
```

255) Write a program in Java to find the maximum and minimum value node from a circular linked list.

Consider the following program.

```
public class MinMax {
    //Represents the node of list.
    public class Node{
        int data;
        Node next;
        public Node(int data) {
            this.data = data;
        }
    }

    //Declaring head and tail pointer as null.
    public Node head = null;
    public Node tail = null;

    //This function will add the new node at the end of the list.
    public void add(int data){
        //Create new node
        Node newNode = new Node(data);
        //Checks if the list is empty.
        if(head == null) {
             //If list is empty, both head and tail would point to new node.
            head = newNode;
            tail = newNode;
            newNode.next = head;
        }
        else {
            //tail will point to new node.
            tail.next = newNode;
            //New node will become new tail.
            tail = newNode;
            //Since, it is circular linked list tail will points to head.
            tail.next = head;
        }
    }

    //Finds out the minimum value node in the list
    public void minNode() {
        Node current = head;
        //Initializing min to initial node data
        int min = head.data;
        if(head == null) {
            System.out.println("List is empty");
        }
        else {
             do{
                //If current node's data is smaller than min
                //Then replace value of min with current node's data
                if(min > current.data) {
                    min = current.data;
                }
                current= current.next;
            }while(current != head);

            System.out.println("Minimum value node in the list: "+ min);
        }
    }

    //Finds out the maximum value node in the list
    public void maxNode() {
        Node current = head;
        //Initializing max to initial node data
        int max = head.data;
        if(head == null) {
            System.out.println("List is empty");
        }
```

```
        else {
             do{
                 //If current node's data is greater than max
                 //Then replace value of max with current node's data
                 if(max < current.data) {
                     max = current.data;
                 }
                 current= current.next;
                }while(current != head);

            System.out.println("Maximum value node in the list: "+ max);
        }
    }

    public static void main(String[] args) {
        MinMax cl = new MinMax();
        //Adds data to the list
        cl.add(5);
        cl.add(20);
        cl.add(10);
        cl.add(1);
        //Prints the minimum value node in the list
        cl.minNode();
        //Prints the maximum value node in the list
        cl.maxNode();
    }
}
```

Output:

```
Minimum value node in the list: 1
Maximum value node in the list: 20
```

256) Write a program in Java to calculate the difference between the sum of the odd level and even level nodes of a Binary Tree.

Consider the following program.

```
import java.util.LinkedList;
import java.util.Queue;

public class DiffOddEven {

    //Represent a node of binary tree
    public static class Node{
        int data;
        Node left;
        Node right;

        public Node(int data){
            //Assign data to the new node, set left and right children to null
            this.data = data;
            this.left = null;
            this.right = null;
            }
        }

    //Represent the root of binary tree
    public Node root;

    public DiffOddEven(){
        root = null;
    }

    //difference() will calculate the difference between sum of odd and even levels of binary tree
    public int difference() {
          int oddLevel = 0, evenLevel = 0, diffOddEven = 0;

          //Variable nodesInLevel keep tracks of number of nodes in each level
          int nodesInLevel = 0;

          //Variable currentLevel keep track of level in binary tree
          int currentLevel = 0;

          //Queue will be used to keep track of nodes of tree level-wise
          Queue<Node> queue = new LinkedList<Node>();

          //Check if root is null
          if(root == null) {
              System.out.println("Tree is empty");
              return 0;
          }
          else {
              //Add root node to queue as it represents the first level
              queue.add(root);
              currentLevel++;

              while(queue.size() != 0) {

                  //Variable nodesInLevel will hold the size of queue i.e. number of elements in queue
                  nodesInLevel = queue.size();

                  while(nodesInLevel > 0) {
                      Node current = queue.remove();

                    //Checks if currentLevel is even or not.
                      if(currentLevel % 2 == 0)
                          //If level is even, add nodes's to variable evenLevel
                          evenLevel += current.data;
                      else
                          //If level is odd, add nodes's to variable oddLevel
                          oddLevel += current.data;
```

```
                    //Adds left child to queue
                    if(current.left != null)
                        queue.add(current.left);
                    //Adds right child to queue
                    if(current.right != null)
                        queue.add(current.right);
                   nodesInLevel--;
                }
                currentLevel++;
            }
            //Calculates difference between oddLevel and evenLevel
            diffOddEven = Math.abs(oddLevel - evenLevel);
        }
        return diffOddEven;
    }

    public static void main (String[] args) {

        DiffOddEven bt = new DiffOddEven();
        //Add nodes to the binary tree
        bt.root = new Node(1);
        bt.root.left = new Node(2);
        bt.root.right = new Node(3);
        bt.root.left.left = new Node(4);
        bt.root.right.left = new Node(5);
        bt.root.right.right = new Node(6);

        //Display the difference between sum of odd level and even level nodes
        System.out.println("Difference between sum of odd level and even level nodes: " + bt.difference());
}
}
```

Output:

```
Difference between sum of odd level and even level nodes: 11
```

Multithreading & Concurrency

Multithreading Interview Questions and Answers

Questions in this topic (50)

1) What is multithreading?
2) What is the thread?
3) Differentiate between process and thread?
4) What do you understand by inter-thread communication?
5) What is the purpose of the wait() method in Java?
6) Why wait() method must be called from the synchronized block?
7) What are the advantages of multithreading?
8) Explain lifecycle of a Thread?
9) What is the difference between preemptive scheduling and time slicing?
10) What is context switching?
11) Differentiate between the Thread class and Runnable interface for creating a Thread?
12) What does join() method?
13) Describe the purpose and working of the sleep() method.
14) What is the difference between wait() and sleep() method?
15) Is it possible to start a thread twice?
16) Can we call the run() method instead of start()?
17) What about the daemon threads?
18) Can we make the user thread as daemon thread if the thread is started?
19) What is shutdown hook?
20) When should we interrupt a thread?
21) What is synchronization?
22) What is the purpose of the Synchronized block?
23) Can Java object be locked down for exclusive use by a given thread?
24) What is static synchronization?
25) What is the difference between notify() and notifyAll() method?
26) What is the deadlock situation in programming?
27) How to detect a deadlock condition? How can it be avoided?
28) What is Thread Scheduler in Java?
29) Does each thread have its stack in multithreaded programming?
30) How can we achieved thread safety?
31) What is race-condition?
32) What is the volatile keyword in Java?
33) What do you understand by thread pool?
34) What is the difference between user-level and kernel-level threads?
35) Explain the concept of race conditions in multithreading and how they can be mitigated.
36) What is the significance of the volatile keyword in Java multithreading?
37) How does synchronization impact performance in multithreaded programs?
38) What is the purpose of the Atomic classes?
39) Explain the concept of thread-local variables and their usage.
40) Can we implement mutual exclusion without using synchronized blocks or methods?
41) Discuss the benefits and limitations of using locks in multithreaded programming.
42) What is the Java Memory Model, and how does it relate to multithreading?
43) How does Java handle thread priorities, and what are their implications?
44) What is a daemon thread, and how does it differ from user threads?
45) Discuss the advantages and disadvantages of using wait() and notify() methods for inter-thread communication.
46) Explain the concept of thread safety and why it is important in concurrent programming.

47) What are the challenges associated with debugging multithreaded applications?

48) Explain the differences between optimistic and pessimistic locking?

49) How does Java support parallelism and concurrency in modern processors with multiple cores?

50) What is the purpose of the Executor framework in Java, and how does it simplify multithreaded programming?

1) What is multithreading?

Multithreading is a process of executing multiple threads simultaneously. Multithreading is used to obtain the multitasking. It consumes less memory and gives the fast and efficient performance. Its main advantages are:

- Threads share the same address space.
- A thread is lightweight process.
- The cost of communication between two processes is low.

More details.

2) What is the thread?

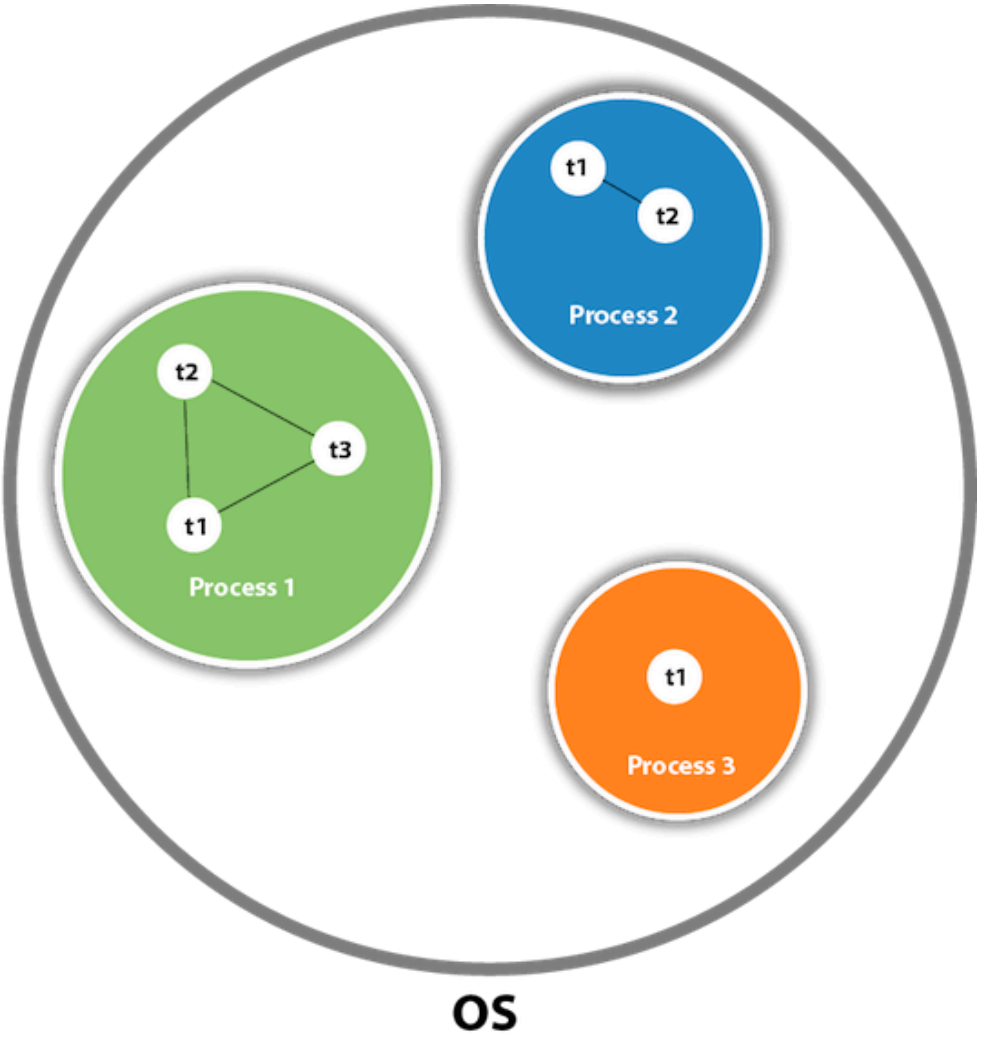
A thread is a lightweight subprocess. It is a separate path of execution because each thread runs in a different stack frame. A process may contain multiple threads. Threads share the process resources, but still, they execute independently.

More details.

3) Differentiate between process and thread?

There are the following differences between the process and thread.

- A Program in the execution is called the process whereas; A thread is a subset of the process
- Processes are independent whereas threads are the subset of process.
- Process have different address space in memory, while threads contain a shared address space.
- Context switching is faster between the threads as compared to processes.
- Inter-process communication is slower and expensive than inter-thread communication.
- Any change in Parent process doesn't affect the child process whereas changes in parent thread can affect the child thread.



4) What do you understand by inter-thread communication?

- The process of communication between synchronized threads is termed as inter-thread communication.
- Inter-thread communication is used to avoid thread polling in Java.
- The thread is paused running in its critical section, and another thread is allowed to enter (or lock) in the same critical section to be executed.
- It can be achieved by using the wait(), notify(), and notifyAll() methods.

5) What is the purpose of the wait() method in Java?

The wait() method is provided by the Object class in Java. It is used for inter-thread communication. The java.lang.Object.wait() method is used to pause the current thread, and wait until another thread does not call the notify() or notifyAll() method. Its syntax is given below.

```
public final void wait()
```

6) Why wait() method must be called from the synchronized block?

We must call the wait() method otherwise it will throw **java.lang.IllegalMonitorStateException** exception. Moreover, we need wait() method for inter-thread communication with notify() and notifyAll() method. Therefore, it must be present in the synchronized block for the proper and correct communication.

7) What are the advantages of multithreading?

Multithreading programming has the following advantages:

- Multithreading allows an application/program to be always reactive for input, even already running with some background tasks.
- Multithreading allows the faster execution of tasks, as threads execute independently.
- Multithreading provides better utilization of cache memory as threads share the common memory resources.
- Multithreading reduces the number of the required server as one server can execute multiple threads at a time.

8) Explain lifecycle of a Thread?

A thread can have one of the following states during execution:

New: In this state, a Thread class object is created using a new operator, but the thread is not alive. Thread does nott start until we call the start() method.

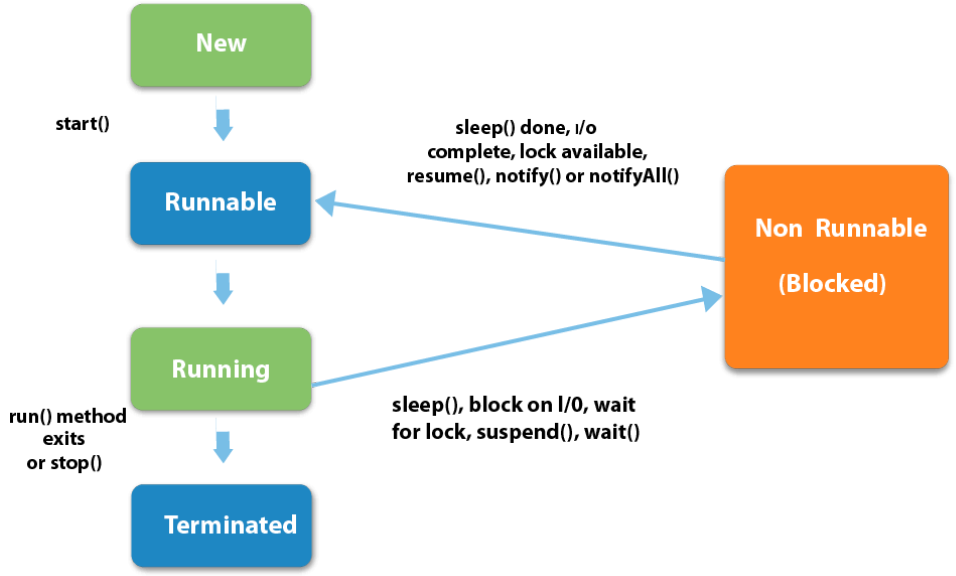
Runnable: In this state, the thread is ready to run after calling the start() method. However, the thread is not yet selected by the thread scheduler.

Running: In this state, the thread scheduler picks the thread from the ready state, and starts running.

Waiting/Blocked: In this state, a thread is not running but still alive, or it is waiting for the other thread to finish.

Dead/Terminated: A thread is in terminated or dead state, if it satisfies any of the following condition:

- If it exits normally. It happens when the code of the thread has been entirely executed by the program.
- Because there occurred some unusual erroneous event, like a segmentation fault or an unhandled exception.



9) What is the difference between preemptive scheduling and time slicing?

In preemptive scheduling, the highest priority task executes until it enters the waiting or dead states or a higher priority task comes into existence. In time slicing, a task executes for a predefined slice of time and then re-enters the pool of ready tasks. The scheduler then determines which task should execute next, based on priority and other factors.

10) What is context switching?

In Context switching the state of the process (or thread) is stored so that it can be restored and execution can be resumed from the same point later. Context switching enables the multiple processes to share the same CPU.

11) Differentiate between the Thread class and Runnable interface for creating a Thread?

The Thread can be created by using two ways.

- By extending the Thread class
- By implementing the Runnable interface

However, the primary differences between both the ways are given below:

- By extending the Thread class, we cannot extend any other class, as Java does not allow multiple inheritances while implementing the Runnable interface; we can also extend other base class (if required).
- By extending the Thread class, each of thread creates the unique object and associates with it while implementing the Runnable interface; multiple threads share the same object
- Thread class provides various inbuilt methods such as getPriority(), isAlive() and more while the Runnable interface provides a single method, i.e., run().

12) What does join() method?

The join() method waits for a thread to die. In other words, it causes the currently running threads to stop executing until the thread it joins with completes its task. The join() method is overloaded in Thread class in the following ways.

- public void join()throws InterruptedException
- public void join(long milliseconds)throws InterruptedException

More details.

13) Describe the purpose and working of the sleep() method.

The sleep() method in Java is used to block a thread for a particular time, which means it pause the execution of a thread for a specific time. There are the following two methods to achieve the same:

Syntax:

- public static void sleep(long milliseconds)throws InterruptedException
- public static void sleep(long milliseconds, int nanos) throws InterruptedException

Working of sleep() Method

When we call the sleep() method, it pauses the execution of the current thread for the given time and gives priority to another thread (if available). Moreover, when the waiting time completed then again previous thread changes its state from waiting to runnable and comes in running state, and the whole process works so on until the execution does not complete.

14) What is the difference between wait() and sleep() method?

wait()	sleep()
The wait() method is defined in Object class.	The sleep() method is defined in Thread class.
The wait() method releases the lock.	The sleep() method does not release the lock.

15) Is it possible to start a thread twice?

No, we cannot restart the thread, once a thread started and executed, it goes to the Dead state. Therefore, if we try to start a thread twice, it will give a RuntimeException "java.lang.IllegalThreadStateException". Consider the following example.

```
public class Multithread1 extends Thread
{
   public void run()
    {
      try {
          System.out.println("thread is executing now........");
      } catch(Exception e) {
      }
    }
        public static void main (String[] args) {
        Multithread1 m1= new Multithread1();
        m1.start();
        m1.start();
    }
}
```

Output

```
thread is executing now........
Exception in thread "main" java.lang.IllegalThreadStateException
        at java.lang.Thread.start(Thread.java:708)
        at Multithread1.main(Multithread1.java:13)
```

More details.

16) Can we call the run() method instead of start()?

Yes, calling run() method directly is valid, but it will not work as a thread instead it will work as a normal object. There will not be context-switching between the threads. When we call the start() method, it internally calls the run() method that creates a new stack for a thread while directly calling the run() will not create a new stack.

More details.

17) What about the daemon threads?

The daemon threads are the low priority threads that provide the background support and services to the user threads. Daemon thread gets automatically terminated by the JVM if the program remains with the daemon thread only, and all other user threads are ended/died. There are two methods for daemon thread available in the Thread class:

- **public void setDaemon(boolean status):** It used to mark the thread daemon thread or a user thread.
- **public boolean isDaemon():** It checks the thread is daemon or not.

More details.

18) Can we make the user thread as daemon thread if the thread is started?

No, if we do such, it will throw IllegalThreadStateException. Therefore, we can only create a daemon thread before starting the thread.

```
class Testdaemon1 extends Thread{
public void run(){
          System.out.println("Running thread is daemon...");
}
public static void main (String[] args) {
      Testdaemon1 td= new Testdaemon1();
      td.start();
      setDaemon(true);// It will throw the exception: td.
   }
}
```

Output

```
Running thread is daemon...
Exception in thread "main" java.lang.IllegalThreadStateException
at java.lang.Thread.setDaemon(Thread.java:1359)
at Testdaemon1.main(Testdaemon1.java:8)
```

More details.

19) What is shutdown hook?

The shutdown hook is a thread that is invoked implicitly before JVM shuts down. So we can use it to perform clean up the resource or save the state when JVM shuts down normally or abruptly. We can add shutdown hook by using the following method:

```
public void addShutdownHook(Thread hook){}
Runtime r=Runtime.getRuntime();
r.addShutdownHook(new MyThread());
```

Some important points about shutdown hooks are :

- Shutdown hooks initialized but can only be started when JVM shutdown occurred.
- Shutdown hooks are more reliable than the finalizer() because there are very fewer chances that shutdown hooks not run.
- The shutdown hook can be stopped by calling the halt(int) method of Runtime class.

More details.

20) When should we interrupt a thread?

We should interrupt a thread when we want to break out the sleep or wait state of a thread. We can interrupt a thread by calling the interrupt() throwing the InterruptedException.

More details.

21) What is synchronization?

Synchronization is the capability to control the access of multiple threads to any shared resource. It is used:

1. To prevent thread interference.
2. To prevent consistency problem.

When the multiple threads try to do the same task, there is a possibility of an erroneous result, hence to remove this issue, Java uses the process of synchronization which allows only one thread to be executed at a time. Synchronization can be achieved in three ways:

- by the synchronized method
- by synchronized block
- by static synchronization

Syntax for synchronized block

```
synchronized(object reference expression)
        {
                //code block
        }
```

More details.

22) What is the purpose of the Synchronized block?

The Synchronized block can be used to perform synchronization on any specific resource of the method. Only one thread at a time can execute on a particular resource, and all other threads which attempt to enter the synchronized block are blocked.

- Synchronized block is used to lock an object for any shared resource.
- The scope of the synchronized block is limited to the block on which, it is applied. Its scope is smaller than a method.

More details.

23) Can Java object be locked down for exclusive use by a given thread?

Yes, we can lock an object by putting it in a "synchronized" block. The locked object is inaccessible to any thread other than the one that explicitly claimed it.

24) What is static synchronization?

If we declare any static method as synchronized, the lock will be on the class not on the object. If we use the synchronized keyword before a method so it will lock the object (one thread can access an object at a time) but if we use static synchronized so it will lock a class (one thread can access a class at a time).

More details.

25) What is the difference between notify() and notifyAll() method?

The notify() method is used to unblock one waiting thread whereas notifyAll() method is used to unblock all the threads in waiting state.

26) What is the deadlock situation in programming?

Deadlock is a situation in which every thread is waiting for a resource which is held by some other waiting thread. In this situation, neither of the thread executes nor it gets the chance to be executed. Instead, there exists a universal waiting state among all the threads. Deadlock is a very complicated situation that can break our code at runtime.

More details.

27) How to detect a deadlock condition? How can it be avoided?

We can detect the deadlock condition by running the code on CMD and collecting the Thread Dump, and if any deadlock is present in the code, then a message will appear on prompt (CMD).

There are the following ways to avoid the deadlock condition in Java:

- **Avoid Nested Lock:** Nested lock is the common reason for deadlock. Deadlock occurs when we provide locks to various threads, so we should give one lock to only one thread at particular time.
- **Avoid Unnecessary Locks:** We must avoid the locks which are not required.
- **Using Thread Join:** Thread join helps to wait for a thread until another thread does not finish its execution. Therefore, we can avoid deadlock by maximizing the use of join() method.

28) What is Thread Scheduler in Java?

In Java, when we create the threads, they are supervised with the help of a Thread Scheduler that is the part of JVM. Thread scheduler is only responsible for deciding which thread should be executed. Thread scheduler uses two mechanisms for scheduling the threads: Preemptive and Time Slicing.

Java thread scheduler also works for deciding the following for a thread:

- It selects the priority of the thread.
- It determines the waiting time for a thread
- It checks the Nature of thread

29) Does each thread have its stack in multithreaded programming?

Yes, in multithreaded programming every thread maintains its own or separate stack area in memory due to which every thread is independent of each other.

30) How can we achieved thread safety?

When a method or class object is used by multiple threads at a time without any race condition, then the class is thread-safe. Thread safety is used to make a program safe to use in multithreaded programming. It can be achieved by the following ways:

- Using Synchronization
- Using Volatile keyword
- Using Lock Based Mechanism
- Using Atomic Wrapper Classes

31) What is race-condition?

A race condition is a problem which occurs in the multithreaded programming when multiple threads try to execute simultaneously by accessing a shared resource at the same time. The proper use of synchronization can avoid the race condition.

32) What is the volatile keyword in Java?

Volatile keyword is used in multithreaded programming to achieve the thread safety, as a change in one volatile variable is visible to all other threads so one variable can be used by one thread at a time.

33) What do you understand by thread pool?

- Java Thread pool represents a group of worker threads, which are waiting for the task to be allocated.
- Threads in the thread pool are supervised by the service provider which pulls one thread from the pool and assign a job to it.
- After completion of the given task, thread again came to the thread pool.
- The size of the thread pool depends on the total number of threads kept at reserve for execution.

The advantages of the thread pool are :

- Performance can be enhanced.
- Better system stability can occur.

34) What is the difference between user-level and kernel-level threads?

User-level threads are managed by the application, and the operating system is unaware of their existence. On the other hand, Kernel-level threads are managed by the operating system kernel, providing better support for multitasking and parallel execution.

35) Explain the concept of race conditions in multithreading and how they can be mitigated.

Race conditions occur when the outcome of a program depends on the timing or interleaving of multiple threads accessing shared resources. They can be mitigated using synchronization mechanisms like locks, atomic variables, or by using immutable data structures.

36) What is the significance of the volatile keyword in Java multithreading?

The volatile keyword ensures that the value of a variable is always read from and written to main memory, rather than from a thread's cache. It guarantees visibility of changes made by one thread to all other threads, preventing caching-related inconsistencies.

37) How does synchronization impact performance in multithreaded programs?

Synchronization can impact performance due to the overhead of acquiring and releasing locks. Excessive synchronization can lead to contention and reduced parallelism. It is important to balance synchronization to ensure thread safety without sacrificing performance.

38) What is the purpose of the Atomic classes?

The Atomic classes provide atomic operations on variables without requiring explicit synchronization. They are used to perform compound actions like incrementing a counter or updating a shared variable in a thread-safe manner.

39) Explain the concept of thread-local variables and their usage.

Thread-local variables are variables that have a separate copy for each thread. They are typically used to maintain thread-specific data without the need for synchronization. ThreadLocal class in Java provides support for creating thread-local variables.

40) Can we implement mutual exclusion without using synchronized blocks or methods?

Yes, mutual exclusion can be implemented using other synchronization mechanisms like ReentrantLock or Semaphore by using the java.util.concurrent package. These classes provide more flexibility and features compared to synchronized blocks.

41) Discuss the benefits and limitations of using locks in multithreaded programming.

Locks provide explicit control over synchronization and can handle more complex scenarios than intrinsic locks (synchronized blocks). However, they require careful management to avoid issues like deadlock and can introduce additional overhead compared to synchronized blocks.

42) What is the Java Memory Model, and how does it relate to multithreading?

The Java Memory Model defines how threads interact through memory. It ensures that changes made by one thread to shared variables are visible to other threads according to specific rules. Understanding the Java Memory Model is crucial for writing correct and efficient multithreaded programs.

43) How does Java handle thread priorities, and what are their implications?

Java provides thread priorities to influence the scheduling order of threads by the thread scheduler. Higher priority threads are scheduled with preference, but priority-based thread scheduling behavior can vary across different platforms and JVM implementations.

44) What is a daemon thread, and how does it differ from user threads?

A daemon thread is a low-priority thread that runs in the background, providing services to user threads. Unlike user threads, daemon threads do not prevent the JVM from exiting when all user threads have finished executing.

45) Discuss the advantages and disadvantages of using wait() and notify() methods for inter-thread communication.

Advantages: The wait() and notify() methods provide a simple mechanism for inter-thread communication, allowing threads to efficiently wait for specific conditions. It can be used to implement producer-consumer patterns and other synchronization scenarios.

Disadvantages: They require proper synchronization using synchronized blocks, and incorrect usage can lead to issues like missed signals or deadlock. Additionally, notify() wakes up only one waiting thread, which may not be desirable in some situations.

46) Explain the concept of thread safety and why it is important in concurrent programming.

Thread safety ensures that shared resources are accessed in a manner that avoids race conditions and maintains data consistency. It is crucial in concurrent programming to prevent unpredictable behavior and ensure the correctness of multithreaded programs.

47) What are the challenges associated with debugging multithreaded applications?

Multithreaded applications introduce non-deterministic behavior, making it difficult to reproduce and debug issues. Challenges include race conditions, deadlocks, and thread interleaving that require careful analysis and debugging techniques like thread dumps and synchronization debugging tools.

48) Explain the differences between optimistic and pessimistic locking?

Optimistic locking assumes that conflicts between threads are rare and allows multiple threads to access shared resources concurrently. On the other hand, pessimistic locking, assumes conflicts are common and restricts access to shared resources, typically using locks to enforce mutual exclusion.

49) How does Java support parallelism and concurrency in modern processors with multiple cores?

Java provides support for parallelism and concurrency through features like the Executor framework, ForkJoinPool, and java.util.concurrent package, allowing developers to leverage multiple cores for efficient execution of concurrent tasks.

50) What is the purpose of the Executor framework in Java, and how does it simplify multithreaded programming?

The Executor framework provides a higher-level abstraction for managing thread execution and scheduling tasks. It decouples task submission from execution, simplifies thread management, and allows for easy configuration of thread pools and task execution policies.

Concurrency Interview Questions

Questions in this topic (35)

51) What are the main components of concurrency API?
52) What is the Executor interface in Concurrency API in Java?
53) What is BlockingQueue?
54) How to implement producer-consumer problem by using BlockingQueue?
55) What is the difference between Java Callable interface and Runnable interface?
56) What is the Atomic action in Concurrency in Java?
57) What is Lock interface in Concurrency API in Java?
58) Explain the ExecutorService Interface.
59) What are the differences between Synchronous programming and Asynchronous programming regarding a thread?
60) What do you understand by Callable and Future in Java?
61) What is the difference between ScheduledExecutorService and ExecutorService interface?
62) Define FutureTask class in Java?
63) What is a Semaphore?
64) Explain the concept of a CountdownLatch.
65) What is a CyclicBarrier and how is it used?
66) Discuss the purpose of the Phaser class.
67) What is the purpose of the Exchanger class?
68) Explain the concept of thread confinement.
69) What is the purpose of the ReadWriteLock interface?
70) Discuss the concept of thread-safe collections.
71) What is the purpose of the ForkJoinPool class?
72) Explain the concept of parallel streams.
73) What is the purpose of the CompletableFuture class?
74) Discuss the concept of non-blocking algorithms.
75) What is the purpose of the Phaser class?
76) Discuss the concept of the ThreadLocal class.
77) What is the purpose of the LockSupport class?
78) How to define the stack size explicitly for a Thread?
79) Can we override Thread.start() method?
80) What is thread priority? What is the default thread priority?
81) What is daemon thread?
82) What are the differences between Daemon Thread and User Thread?
83) Can we declare synchronized method as abstract. If, no why?
84) What are the differences between submit() and execute() method?
85) How do you create and use Streams in Java 8 concurrency?

51) What are the main components of concurrency API?

Concurrency API can be developed using the class and interfaces of java.util.Concurrent package. There are the following classes and interfaces in java.util.Concurrent package.

- Executor
- FarkJoinPool
- ExecutorService
- ScheduledExecutorService
- Future
- TimeUnit(Enum)
- CountDownLatch
- CyclicBarrier

- Semaphore
- ThreadFactory
- BlockingQueue
- DelayQueue
- Locks
- Phaser

52) What is the Executor interface in Concurrency API in Java?

The Executor interface belong to the package java.util.concurrent. It is an interface used to execute the new task. The execute() method of the Executor interface is used to execute specified command. The syntax of the execute() method is given below.

```
void execute(Runnable command)
```

Consider the following example:

```
import java.util.concurrent.Executor;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class TestThread {
   public static void main(final String[] arguments) throws InterruptedException {
      Executor e = Executors.newCachedThreadPool();
      e.execute(new Thread());
      ThreadPoolExecutor pool = (ThreadPoolExecutor)e;
      pool.shutdown();
   }

   static class Thread implements Runnable {
      public void run() {
         try {
            Long duration = (long) (Math.random() * 5);
            System.out.println("Running Thread!");
            TimeUnit.SECONDS.sleep(duration);
            System.out.println("Thread Completed");
         } catch (InterruptedException ex) {
            ex.printStackTrace();
         }
      }
   }
}
```

Output

```
Running Thread!
Thread Completed
```

53) What is BlockingQueue?

The java.util.concurrent.BlockingQueue is the subinterface of Queue interface. It supports the operations such as waiting for the space availability before inserting a new value or waiting for the queue to become non-empty before retrieving an element from it. Consider the following example.

```
import java.util.Random;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class TestThread {

   public static void main(final String[] arguments) throws InterruptedException {
      BlockingQueue<Integer> queue = new ArrayBlockingQueue<Integer>(10);

      Insert i = new Insert(queue);
      Retrieve r = new Retrieve(queue);

      new Thread(i).start();
      new Thread(r).start();

      Thread.sleep(2000);
   }


   static class Insert implements Runnable {
      private BlockingQueue<Integer> queue;

      public Insert(BlockingQueue queue) {
         this.queue = queue;
      }

      @Override
      public void run() {
         Random random = new Random();

         try {
            int result = random.nextInt(200);
            Thread.sleep(1000);
            queue.put(result);
            System.out.println("Added: " + result);

            result = random.nextInt(10);
            Thread.sleep(1000);
            queue.put(result);
            System.out.println("Added: " + result);

            result = random.nextInt(50);
            Thread.sleep(1000);
            queue.put(result);
            System.out.println("Added: " + result);
         } catch (InterruptedException e) {
            e.printStackTrace();
         }
      }
   }

   static class Retrieve implements Runnable {
      private BlockingQueue<Integer> queue;

      public Retrieve(BlockingQueue queue) {
         this.queue = queue;
      }

      @Override
      public void run() {

         try {
            System.out.println("Removed: " + queue.take());
            System.out.println("Removed: " + queue.take());
```

```
            System.out.println("Removed: " + queue.take());
         } catch (InterruptedException e) {
            e.printStackTrace();
         }
      }
   }
}
```

Output

```
Added: 96
Removed: 96
Added: 8
Removed: 8
Added: 5
Removed: 5
```

54) How to implement producer-consumer problem by using BlockingQueue?

The producer-consumer problem can be solved by using BlockingQueue in the following way.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.logging.Level;
import java.util.logging.Logger;
public class ProducerConsumerProblem {
    public static void main(String args[]){
     //Creating shared object
     BlockingQueue sharedQueue = new LinkedBlockingQueue();

     //Creating Producer and Consumer Thread
     Thread prod = new Thread(new Producer(sharedQueue));
     Thread cons = new Thread(new Consumer(sharedQueue));

     //Starting producer and Consumer thread
     prod.start();
     cons.start();
    }

}

//Producer Class in Java
class Producer implements Runnable {

    private final BlockingQueue sharedQueue;

    public Producer(BlockingQueue sharedQueue) {
        this.sharedQueue = sharedQueue;
    }

    @Override
    public void run() {
        for(int i=0; i<10; i++){
            try {
                System.out.println("Produced: " + i);
                sharedQueue.put(i);
            } catch (InterruptedException ex) {
                Logger.getLogger(Producer.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }

}

//Consumer Class in Java
class Consumer implements Runnable{

    private final BlockingQueue sharedQueue;

    public Consumer (BlockingQueue sharedQueue) {
        this.sharedQueue = sharedQueue;
    }

    @Override
    public void run() {
        while(true){
            try {
                System.out.println("Consumed: "+ sharedQueue.take());
            } catch (InterruptedException ex) {
                Logger.getLogger(Consumer.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }
}
```

Output

```
Produced: 0
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
```

55) What is the difference between Java Callable interface and Runnable interface?

The Callable and Runnable interface both are used by the classes which wanted to execute with multiple threads. However, there are the following key differences between the both:

- A Callable <V> interface can return a result, whereas the Runnable interface cannot return any result.
- A Callable <V> interface can throw a checked exception, whereas the Runnable interface cannot throw checked exception.
- A Callable <V> interface cannot be used prior Java 5 version whereas the Runnable interface can be used.

56) What is the Atomic action in Concurrency in Java?

- The Atomic action is the operation which can be performed in a single unit of a task without any interference of the other operations.
- The Atomic action cannot be stopped in between the task. Once started it fill stop after the completion of the task only.
- An increment operation such as a++ does not allow an atomic action.
- All reads and writes operation for the primitive variable (except long and double) are the atomic operation.
- All reads and writes operation for the volatile variable (including long and double) are the atomic operation.
- The Atomic methods are available in java.util.Concurrent package.

57) What is Lock interface in Concurrency API in Java?

The java.util.concurrent.locks.Lock interface is used as the synchronization mechanism. It works similar to the synchronized block. There are a few differences between the lock and synchronized block that are given below.

- Lock interface provides the guarantee of sequence in which the waiting thread will be given the access, whereas the synchronized block does not guarantee it.
- Lock interface provides the option of timeout if the lock is not granted whereas the synchronized block does not provide that.
- The methods of Lock interface, i.e., lock() and unlock() can be called in different methods whereas single synchronized block must be fully contained in a single method.

58) Explain the ExecutorService Interface.

The ExecutorService Interface is the subinterface of Executor interface and adds the features to manage the lifecycle. Consider the following example.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TestThread {
   public static void main(final String[] arguments) throws InterruptedException {
      ExecutorService e = Executors.newSingleThreadExecutor();

      try {
         e.submit(new Thread());
         System.out.println("Shutdown executor");
         e.shutdown();
         e.awaitTermination(5, TimeUnit.SECONDS);
      } catch (InterruptedException ex) {
         System.err.println("tasks interrupted");
      } finally {

         if (!e.isTerminated()) {
            System.err.println("cancel non-finished tasks");
         }
         e.shutdownNow();
         System.out.println("shutdown finished");
      }
   }

   static class Task implements Runnable {

      public void run() {

         try {
            Long duration = (long) (Math.random() * 20);
            System.out.println("Running Task!");
            TimeUnit.SECONDS.sleep(duration);
         } catch (InterruptedException ex) {
            ex.printStackTrace();
         }
      }
   }
}
```

Output

```
Shutdown executor
shutdown finished
```

59) What are the differences between Synchronous programming and Asynchronous programming regarding a thread?

Synchronous Programming: In Synchronous programming model, a thread is assigned to complete a task and hence thread started working on it, and it is only available for other tasks once it will end the assigned task.

Asynchronous Programming: In Asynchronous programming, one job can be completed by multiple threads and hence it provides maximum usability of the various threads.

60) What do you understand by Callable and Future in Java?

Java Callable Interface: In Java 5, Callable interface defined in java.util.concurrent package. It is similar to the Runnable interface. But it can return a result, and it can throw an Exception. It also provides a run() method for execution of a thread. Java Callable can return any object as it uses Generic.

Syntax:

```
public interface Callable<V>
```

Java Future interface: Java Future interface gives the result of a concurrent process. The Callable interface returns the object of java.util.concurrent.Future.

Java Future interface provides following methods for implementation.

- **cancel(boolean mayInterruptIfRunning):** It is used to cancel the execution of the assigned task.
- **get():** It waits for the time if execution not completed and then retrieved the result.
- **isCancelled():** It returns the Boolean value as it returns true if the task was cancelled before the completion.
- **isDone():** It returns true if the job is completed successfully else returns false.

61) What is the difference between ScheduledExecutorService and ExecutorService interface?

ExecutorServcie and ScheduledExecutorService both are the interfaces of java.util.Concurrent package but scheduledExecutorService provides some additional methods to execute the Runnable and Callable tasks with the delay or every fixed time period.

62) Define FutureTask class in Java?

Java FutureTask class provides a base implementation of the Future interface. The result can only be obtained if the execution of one task is completed, and if the computation is not achieved then get method will be blocked. If the execution is completed, then it cannot be re-started and cannot be cancelled.

Syntax

```
public class FutureTask<V> extends Object implements RunnableFuture<V>
```

63) What is a Semaphore?

A Semaphore is a synchronization primitive that restricts the number of threads that can access a shared resource concurrently. It maintains a set of permits that control access to the resource, allowing a fixed number of threads to acquire permits and execute code that accesses the shared resource.

64) Explain the concept of a CountdownLatch.

CountdownLatch is a synchronization aid that allows one or more threads to wait until a set of operations being performed in other threads completes. It is initialized with a count, and each thread decrements the count when it completes its operation. Threads waiting on the latch are released when the count reaches zero.

65) What is a CyclicBarrier and how is it used?

A CyclicBarrier is a synchronization aid that allows a group of threads to wait at a predefined barrier until all threads reach the barrier before proceeding. Once all threads have reached the barrier, they are released simultaneously. CyclicBarrier can be reset to its initial state after all threads have crossed the barrier.

66) Discuss the purpose of the Phaser class.

Phaser is a synchronization barrier that allows multiple threads to synchronize their execution at predefined phases. It provides more flexible synchronization than CountDownLatch and CyclicBarrier, allowing threads to dynamically register and deregister at different phases. Phaser can be used for implementing multi-phase algorithms.

67) What is the purpose of the Exchanger class?

The Exchanger class provides a synchronization point at which two threads can exchange objects. Each thread calls the exchange() method, providing an object to exchange, and waits until the other thread also calls exchange(). Once both threads have called exchange(), they swap their objects and continue execution.

68) Explain the concept of thread confinement.

Thread confinement is a concurrency control technique where a particular data is accessible only by a single thread at a time. It ensures that the data remains consistent and avoids race conditions by preventing concurrent access from multiple threads. Thread confinement can be achieved by encapsulating the data within a thread or using synchronization mechanisms.

69) What is the purpose of the ReadWriteLock interface?

The ReadWriteLock interface provides a mechanism for controlling access to shared resources in a multi-threaded environment. Unlike traditional locks, ReadWriteLock allows multiple threads to read the resource concurrently while providing exclusive access for writing. It can improve performance in scenarios where reads are more frequent than writes.

70) Discuss the concept of thread-safe collections.

Thread-safe collections are data structures provided by the java.util.concurrent package that can be safely accessed and modified by multiple threads concurrently. These collections, such as ConcurrentHashMap and CopyOnWriteArrayList, handle synchronization internally to ensure thread safety without requiring external synchronization by the user.

71) What is the purpose of the ForkJoinPool class?

ForkJoinPool is a special-purpose ExecutorService implementation designed for parallelizing recursive divide-and-conquer algorithms. It manages a pool of worker threads that execute tasks using a work-stealing algorithm, where idle threads steal tasks from other threads' queues to maximize CPU utilization and minimize contention.

72) Explain the concept of parallel streams.

Parallel streams are a feature introduced in Java 8 to leverage multi-core processors for parallel execution of stream operations. They enable automatic parallelization of operations like map, filter, and reduce on collections, distributing the workload across multiple threads for improved performance on multi-core systems.

73) What is the purpose of the CompletableFuture class?

CompletableFuture is a class introduced in Java 8 to represent a future result of an asynchronous computation. It provides a powerful API for composing asynchronous operations, handling exceptions, and chaining dependent computations. CompletableFuture supports both synchronous and asynchronous execution, making it versatile for building complex asynchronous workflows.

74) Discuss the concept of non-blocking algorithms.

Non-blocking algorithms are concurrency control techniques that ensure progress even in the presence of contention or thread failures without requiring locks or blocking synchronization primitives. They use atomic operations and compare-and-

swap (CAS) instructions to perform operations on shared data without blocking threads, leading to better scalability and responsiveness.

75) What is the purpose of the Phaser class?

Phaser is a synchronization barrier introduced in Java 7 that allows synchronization of threads across multiple phases. It is similar to CountDownLatch and CyclicBarrier but provides more flexibility by allowing dynamic registration and deregistration of parties (threads) at different phases. Phaser supports both one-shot and cyclic synchronization scenarios.

76) Discuss the concept of the ThreadLocal class.

ThreadLocal is a class provided in Java to create thread-local variables, where each thread has its own independent copy of the variable. ThreadLocal variables are typically used to store thread-specific data without the need for synchronization, improving performance and reducing contention in multithreaded applications.

77) What is the purpose of the LockSupport class?

LockSupport is a utility class provided in Java for low-level thread synchronization. It allows threads to block and unblock based on the availability of permits, similar to wait() and notify() methods, but with more flexibility and control. LockSupport can be used to implement custom synchronization primitives and coordination mechanisms between threads.

78) How to define the stack size explicitly for a Thread?

The Thread class provides a constructor through which we can determine the stack size explicitly for the new thread.

```
new Thread(ThreadGroup group, Runnable target, String name, long stackSize);
```

The effect of the stackSize parameter, if any, is highly platform dependent. In some platforms, a higher value of stack size may allow a deeper recursion depth before throwing StackOverflowError, and in other platforms, it may not have any effect at all.

79) Can we override Thread.start() method?

Yes, we can override the Thread.start() method, but note that overridden method will be executed just like a normal method call executed by the main thread only. It will not seed a new Thread similar to the default start() method. Consider the following example.

```
class ChildThread extends Thread {
  public void start() {
    System.out.println("Overriding start() method in ChildThread");
  }
  public void run(){
    System.out.println("ChildThread run() method");
  }
}
```

The above code snippet creates a new ChildThread instance and call its start() method. It does not start a new thread and run() method is not executed.

```
ChildThread childThread = new ChildThread();
childThread.start();
```

Output:

```
Overriding start() method in ChildThread
```

To initialize a new Thread, we must call the super.start() method from the child thread; only then the run() method will be executed.

```
public void start() {
  super.start();  //Calling Thread.start()
  System.out.println("Overriding start() method in ChildThread");
}
```

Output:

```
Overriding start() method in ChildThread
ChildThread run() method
```

80) What is thread priority? What is the default thread priority?

In Java, JVM assigns a number between 1 to 10 to each thread that is known as thread priority. Priority is used by the thread scheduler at the time of thread execution. While allocating the processor to the thread, the thread scheduler gives the first chance to the thread with the highest priority.

Note that the default priority for any thread is inherited from the parent thread. The default priority for the main thread is 5, so all the child threads will have the default priority of 5.

The Thread class defines three constants to represent the min, max, and default priority of any thread.

Thread.MIN_PRIORITY = 1;

Thread.NORM_PRIORITY = 5;

Thread.MAX_PRIORITY = 10;

81) What is daemon thread?

Daemon Threads are the low-priority threads. It executes in the background to provide support for the non-daemon threads (a thread that executes the main logic of the project is called a non-daemon or user thread). Daemon Threads in Java are also known as **Service Provider Threads**.

Properties of Daemon Threads:

- The life cycle of daemon threads depends on user threads. It provides services to user threads for background supporting tasks.
- They cannot prevent the JVM from exiting when all the user threads finish their execution.
- When all the user threads terminate, JVM terminates Daemon threads automatically.
- It is a thread with the lowest priority possible.
- JVM would not be concerned about whether the Daemon thread is active or not.

82) What are the differences between Daemon Thread and User Thread?

Aspect	Daemon Thread	User Thread
Creation	It is created by JVM.	It is created by an application.
Usage	It provide service to the user threads and always run in the background.	It is used for foreground and I/O tasks of an application.
Priority	These are low-priority threads.	These are high-priority threads.
JVM Behavior	JVM does not wait for daemon threads execution to complete.	JVM waits till the execution of the user thread is finished.
Life Cycle	Its life cycle depends on user thread.	It has independent life cycle.

Termination	Daemon threads are terminated once the last user thread has been terminated.	User threads are terminated after completing their corresponding jobs.

83) Can we declare synchronized method as abstract. If, no why?

In general, synchronization requires a lock to be acquired before a method can be executed. We know that a lock is an implementation feature. Since an abstract method does not have an implementation (and lock), it cannot be synchronized.

Hence declaring the synchronized method as abstract makes no sense.

```
public abstract synchronized int debitAccount();
```

84) What are the differences between submit() and execute() method?

ExecutorService interface provides two methods submit() and execute() for submitting the new tasks to thread pool.

submit()	execute()
It belongs to the ExecutorService interface.	It belongs to the Executor interface.
It can accept both Runnable and Callable.	It accepts only Runnable.
Its return type is the Future object for checking if the task is completed, and for obtaining the result of the task or to cancel it.	Its return type is void.

85) How do you create and use Streams in Java 8 concurrency?

In Java 8, Streams API provides a way to work with large collections of data in parallel. We can use the stream() method of a collection or the Stream.of() method to create a stream. We can use a variety of intermediate operations like filter() or map() to manipulate the data in the stream before using a terminal operation like forEach() or collect() to obtain the final result.

Collections

Java Collections Interview Questions and Answers

Questions in this topic (34)

1) What is the Collection framework in Java?
2) What are the main differences between array and collection?
3) Explain various interfaces used in Collection framework?
4) What are the difference between ArrayList and Vector?
5) What are the difference between ArrayList and LinkedList?
6) What are the differences between Iterator and ListIterator?
7) What are the differences between Iterator and Enumeration?
8) What are the differences between List and Set?
9) What are the differences between HashSet and TreeSet?
10) What are the differences between Set and Map?
11) What are the differences between HashSet and HashMap?
12) What are the differences between HashMap and TreeMap?
13) What are the differences between HashMap and Hashtable?
14) What are the differences between Collection and Collections?
15) What are the differences between Comparable and Comparator?
16) What do you understand by BlockingQueue?
17) What is the advantage of Properties file?
18) What does the hashCode() method?
19) Why we override equals() method?
20) How to synchronize List, Set, and Map elements?
21) What are the advantages of the generic collection?
22) What is hash-collision in Hashtable and how it is handled in Java?
23) What is the Dictionary class?
24) What is the default size of load factor in hashing based collection?
25) What do you understand by fail-fast?
26) What are the differences between Array and ArrayList?
27) What are the differences between the length of an Array and the size of an ArrayList?
28) How to convert ArrayList to Array and Array to ArrayList?
29) How to make Java ArrayList Read-Only?
30) How to remove duplicates from ArrayList?
31) How to reverse ArrayList?
32) How to sort ArrayList in descending order?
33) How to synchronize ArrayList?
34) When to use ArrayList and LinkedList?

1) What is the Collection framework in Java?

A Collection Framework is a combination of classes and interfaces that is used to store and manipulate the data in the form of objects. It provides various classes such as ArrayList, Vector, Stack, and HashSet, etc. and interfaces such as List, Queue, Set, etc. for this purpose. This framework offers an efficient and flexible method of handling data structures by providing a standardised approach to working with collections of items.

2) What are the main differences between array and collection?

Corrected

CORRECTED The source said collections are 'heterogeneous' and arrays 'homogeneous'. A generic List<String> is just as homogeneous, and an Object[] can hold mixed types. The real differences are below.

Checked against: ArrayStoreException.

Aspect	Array	Collection
Size	Fixed when created	Grows and shrinks (for example ArrayList)
Element types	Primitives and objects	Objects only (primitives are boxed, e.g. Integer)
Type checking	At runtime (ArrayStoreException)	At compile time through generics (erased at runtime)
API	Only length and indexing	Rich API: add, remove, contains, iterate, stream, sort with Collections
Kinds of structure	Indexed only	List, Set, Queue, Map, each with different rules
Speed and memory	Arrays of primitives are the most compact and fastest	Extra object overhead, plus boxing for primitives

3) Explain various interfaces used in Collection framework?

Collection framework implements various interfaces, Collection interface and Map interface (java.util.Map) are the mainly used interfaces of Java Collection Framework. List of interfaces of Collection Framework is given below:

1. Collection interface:

This is the collection hierarchy's base interface. Its elements are a collection of items that it represents. Duplicate elements may be found in collections, and elemental orders may change. Commonly used classes like ArrayList, LinkedList, and HashSet are examples of implementations of this interface.

Syntax:

```
public interface Collection<E>extends Iterable
```

Where <E> represents that this interface is of Generic type

2. List interface:

The list extends the Collection interface and represents an ordered collection of elements. It allows duplicate elements and provides methods to access elements by their index. Implementations of the List interface include ArrayList, LinkedList, and Vector.

Syntax:

```
public interface List<E> extends Collection<E>
```

3. Set interface:

A collection that is incapable of having duplicate elements is represented by a set, which expands on the Collection interface. It does not ensure the components' order; instead, it simulates the abstraction of a mathematical set. HashSet, TreeSet, and LinkedHashSet implement the Set interface.

Syntax:

```
public interface Set<E> extends Collection<E>
```

4. Queue interface:

A queue is a collection intended to keep things before they are processed, and it extends the Collection interface. Its elements are arranged in FIFO (First-In-First-Out) order. The Queue interface is implemented by LinkedList, PriorityQueue, and ArrayDeque.

Syntax:

```
public interface Queue<E> extends Collection<E>
```

5. Deque Interface

Deque stands for "Double Ended Queue" and extends the Queue interface. It represents a linear collection that supports element insertion and removal at both ends. Deques can function as standard queues (FIFO) as well as stacks (LIFO). Implementations of the Deque interface include LinkedList and ArrayDeque.

Syntax:

```
public interface Dequeue<E> extends Queue<E>
```

5. Map interface:

The Map interface does not extend the Collection interface, in contrast to other collection interfaces. It shows a mapping of keys to values in which a single value is connected to each key. HashMap, TreeMap, and LinkedHashMap are examples of map implementations. Furthermore, a map that retains its keys in ascending order is provided by the SortedMap interface, which extends Map.

Syntax

```
public interface Map<K, V> {}
```

4) What are the difference between ArrayList and Vector?

Corrected

CORRECTED The source said Vector's iterator is 'fail-safe'. It is fail-fast, like ArrayList's. Only Vector's Enumeration is not fail-fast.

Checked against: Vector, ArrayList.

Aspect	ArrayList	Vector
Synchronization	Not synchronized	Every method is synchronized
Origin	Collections Framework (Java 1.2)	Legacy class from Java 1.0, later retrofitted into the framework
Growth	Grows by about 50% (an implementation detail; the docs only promise amortized constant-time add)	Doubles by default; the docs say the capacity doubles when capacityIncrement is zero or less
Iterators	`iterator()` and `listIterator()` are fail-fast	`iterator()` and `listIterator()` are fail-fast too; only the `Enumeration` from `elements()` is not
Thread safety	None	Each call is locked, but compound actions (check then add) are still not atomic
Advice	Default choice	Javadoc: if thread safety is not needed, use ArrayList instead of Vector

When you need a thread-safe list, use `Collections.synchronizedList()` or `CopyOnWriteArrayList`.

5) What are the difference between ArrayList and LinkedList?

Corrected

CORRECTED The source said ArrayList is 'not efficient for manipulation' and LinkedList is 'efficient for manipulation'. That is too simple: inserting into the middle of a LinkedList is only cheap once you already hold the position.

Checked against: LinkedList, ArrayList, ArrayDeque.

Aspect	ArrayList	LinkedList
Structure	Resizable array	Doubly linked list (also implements Deque)
get(i) / set(i)	O(1)	O(n): walks from the nearer end

Add at the end	Amortized O(1)	O(1)
Insert or remove in the middle	O(n): elements are shifted	O(1) only if you already have the node or iterator position; finding it is O(n)
Memory	Compact array of references	Extra node object with two links per element
Cache behaviour	Contiguous, cache friendly	Nodes scattered in memory

In practice `ArrayList` is faster for most workloads. For a queue or stack, `ArrayDeque` is usually better than `LinkedList`. The Javadoc notes that indexed operations on a LinkedList traverse from the beginning or the end, whichever is closer.

6) What are the differences between Iterator and ListIterator? Corrected

CORRECTED The source said Iterator 'does not maintain a current position' and listed `java.util.Iterator` as its 'implementation class' (that is the interface itself).

Checked against: ArrayList.

Aspect	Iterator	ListIterator
Direction	Forward only	Forward and backward (`hasPrevious()` , `previous()`)
Works with	Any Collection (List, Set, Queue)	List only
Changes while iterating	`remove()`	`remove()` , `set(e)` and `add(e)`
Index information	None	`nextIndex()` and `previousIndex()`
Relationship	Root interface	Extends Iterator

Both keep a cursor internally, and both are fail-fast.

7) What are the differences between Iterator and Enumeration? Corrected

CORRECTED The source listed hasNext() and next() as 'additional methods' of Iterator, said Enumeration has no position, and claimed Iterator is slower. The real differences are below.

Checked against: Vector, Hashtable.

Aspect	Iterator	Enumeration
Introduced	Java 1.2 (Collections Framework)	Java 1.0 (legacy)
Methods	`hasNext()` , `next()` , `remove()` (and `forEachRemaining` since Java 8)	`hasMoreElements()` , `nextElement()`
Remove while iterating	Yes, with `remove()`	No, read only
Fail-fast	Yes, on a best-effort basis	No: the Enumerations from `Vector.elements()` and `Hashtable.keys()` are not fail-fast
Used with	Every collection	Mainly legacy classes (Vector, Hashtable, Stack, Properties)

Use `Iterator` in new code. `Collections.enumeration()` and `Collections.list()` convert between the two.

8) What are the differences between List and Set? Corrected

CORRECTED The source said a Set 'does not preserve insertion order' (LinkedHashSet does), allows 'a single null' (TreeSet allows none), and listed ListIterator as a sub-interface of List (it is not).

Checked against: TreeMap.

Aspect	List	Set
Duplicates	Allowed	Not allowed
Order	Keeps the order of insertion and supports positional access (`get(i)`)	`HashSet` : no guaranteed order; `LinkedHashSet` : insertion order; `TreeSet` : sorted order
Null	ArrayList and LinkedList allow any number of nulls	HashSet and LinkedHashSet allow one null; TreeSet allows none with natural ordering
Index access	Yes	No
Special iterator	`ListIterator` (bidirectional)	None
What counts as a duplicate	-	`equals()` / `hashCode()` for HashSet, `compareTo()` /Comparator for TreeSet
Sub-interfaces	-	SortedSet, NavigableSet, SequencedSet (Java 21)
Common classes	ArrayList, LinkedList, Vector	HashSet, LinkedHashSet, TreeSet

9) What are the differences between HashSet and TreeSet?

Aspect	HashSet	TreeSet
Ordering of Elements	Maintains no specific order	Maintains elements in ascending order
Implementation Structure	Implemented using a hash table	Implemented using a tree structure
Performance	Generally faster due to constant-time operations	Slightly slower due to log(n) time operations
Backing Data Structure	Backed by a HashMap	Backed by a TreeMap
Ordering of Elements	Unordered	Ordered

10) What are the differences between Set and Map?

Aspect	Set	Map
Elements	It contains only values.	It contains key-value pairs.
Uniqueness of Elements	It contains unique values.	It can contain unique keys with duplicate values.
Handling of Null Values	It can hold a single null value.	It can have a single null key with multiple null values.
Accessing Elements	It is accessed by iteration or specific methods.	It is accessed by keys.
Examples	HashSet, TreeSet, LinkedHashSet	HashMap, TreeMap, LinkedHashMap

11) What are the differences between HashSet and HashMap?

Aspect	HashSet	HashMap
Elements	It contains only values.	It contains key-value pairs.
Interface Implementation	It implements Set interface.	It implements Map interface.
Iteration	It can be directly iterated	It needs to be converted into Set for iteration
Uniqueness of Elements	It cannot have duplicate values.	It can contain duplicate values with unique keys.
Handling of Null Values	It can hold a single null value.	It can have a single null key with multiple null values.
Internal Data Structure	It implements using a hash table.	It implements using an array of buckets (hash table and linked list).

12) What are the differences between HashMap and TreeMap? Corrected

CORRECTED The source said a HashMap 'can be sorted by key or value' and gave no reason for the null-key rule. A HashMap has no order at all, and a TreeMap rejects null keys because it has to compare them.

Checked against: HashMap, TreeMap.

Aspect	HashMap	TreeMap
Ordering	None guaranteed, and it can change over time	Sorted by key (natural order or a Comparator)
Structure	Hash table of buckets	Red-black tree
get / put / remove	O(1) on average	Guaranteed O(log n)
Null key	One null key is allowed	Not allowed with natural ordering (NullPointerException, because the key must be compared); depends on the Comparator
Null values	Allowed	Allowed
Key equality	`equals()` and `hashCode()`	`compareTo()` or the Comparator (should be consistent with equals)
Extra methods	-	NavigableMap methods: `firstKey`, `floorKey`, `headMap`, `subMap` ...

A HashMap is never 'sorted'. To order its entries, copy them to a list or stream and sort by key or by value. A TreeMap can only be ordered by key.

Null handling checked by running it on JDK 21: `new TreeMap<>().put(null, 1)` throws NullPointerException; a null value is accepted.

13) What are the differences between HashMap and Hashtable? Corrected

CORRECTED The source said Hashtable iterates 'in the order of insertion' and has a 'fail-safe' iterator, and had the typo 'Tthread-safe'. Hashtable has no insertion order, and its iterators are fail-fast.

Checked against: Hashtable, HashMap.

Aspect	HashMap	Hashtable
Synchronization	Not synchronized	Every method is synchronized
Null	One null key and any number of null values	No null key and no null value (NullPointerException)
Superclass	AbstractMap	Dictionary (obsolete); both implement Map
Iteration order	Not specified	Not specified (it is not insertion order)
Iterators	Fail-fast	Fail-fast for the iterators of its collection views; the Enumerations from `keys()` and `elements()` are not fail-fast
Defaults	Capacity 16, load factor 0.75	Capacity 11, load factor 0.75
Speed	Faster (no locking)	Slower because of locking

The Javadoc recommends `HashMap` when thread safety is not needed and `ConcurrentHashMap` when you need a highly concurrent map. Order, fail-fast behaviour and default sizes were also confirmed by running them on JDK 21.

14) What are the differences between Collection and Collections? Corrected

CORRECTED The source compared an interface with a class using mismatched rows (for example it listed `Collections.sort()` as an 'implementation' of Collection).

Checked against: ArrayList.

Aspect	Collection	Collections
What it is	An interface (`java.util.Collection`)	A final utility class (`java.util.Collections`)
Role	Root of List, Set and Queue (Map is separate): defines `add` , `remove` , `contains` , `size` , `iterator` , `stream` ...	Static helper methods: `sort` , `reverse` , `shuffle` , `max` , `min` , `binarySearch` , `unmodifiableList` , `synchronizedList` , `emptyList` ...
Implemented or extended by	ArrayList, LinkedList, HashSet and many more	Nothing: it cannot be extended or instantiated

15) What are the differences between Comparable and Comparator?

Aspect	Comparable	Comparator
Sort Sequences	It provides one sort sequence.	It provides multiple sort sequences.
Method	It requires implementing compareTo() method.	It requires implementing compare() method.
Location	It belongs to java.lang package.	It belongs to java.util package.
Class Change	It modifies the actual class.	It does not modify the actual class.
Flexibility	It is less flexible as it's tied to the object's class	It is more flexible, allows sorting based on different criteria

16) What do you understand by BlockingQueue?

BlockingQueue is an interface that extends the Queue interface. It provides concurrency in operations like retrieval, insertion, and deletion. When retrieving an element, it waits for the queue to be non-empty. When storing elements, it waits for available space. BlockingQueue cannot contain null elements, and its implementation is thread-safe.

It's crucial to remember that null components are not allowed in BlockingQueue implementations. These implementations are also thread-safe, which makes them appropriate in situations where several threads must concurrently access and change a shared queue. In concurrent programming, BlockingQueue is frequently used to manage data interchange between producer and consumer threads in a thread-safe way.

Syntax:

```
public interface BlockingQueue<E> extends Queue <E>
```

17) What is the advantage of Properties file?

If we change the value in the properties file, we do not need to recompile the Java class. So, it makes the application easy to manage. It is used to store information that is to be changed frequently. Consider the following example.

Test.java

```
import java.util.*;
import java.io.*;
public class Test {
public static void main(String[] args)throws Exception{
    FileReader reader=new FileReader("db.properties");
    Properties p=new Properties();
    p.load(reader);
    System.out.println(p.getProperty("user"));
    System.out.println(p.getProperty("password"));
}
}
```

Output

```
Tpointtech
12345
```

Explanation

The snippet of code that is provided loads the key-value pairs that indicate configuration settings from the db.properties file using the Properties class. The values linked to certain keys ("user" and "password") are then retrieved from the properties file using the getProperty() method, and they are subsequently shown on the console. By separating the configuration data from the application logic, this method facilitates flexibility and manageability.

db.properties

```
user= Tpointtech
password=12345
```

18) What does the hashCode() method?

Corrected

> **CORRECTED** The source said two objects that are equal may produce different hash codes. That breaks the hashCode contract: equal objects must have equal hash codes.
>
> Checked against: Object.

`hashCode()` returns an `int` that hash-based collections (`HashMap`, `HashSet`, `Hashtable`) use to choose a bucket. It is a bucket index hint, not a unique identifier.

The contract (from the `Object` Javadoc):

- While the program runs, the same object returns the same value, as long as the fields used in `equals()` do not change.
- **If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` must be true.**
- If two objects are not equal, their hash codes are **not** required to differ (collisions are allowed), but distinct codes make hash tables faster.

So equal objects always have equal hashes, but equal hashes do not mean equal objects. If you override `equals()` you must override `hashCode()` too, using the same fields. Otherwise a `HashSet` or `HashMap` may fail to find an object that is equal to one already stored (confirmed by running it).

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee e)) return false;
    return id == e.id && Objects.equals(name, e.name);
}
@Override public int hashCode() { return Objects.hash(id, name); }
```

The default `Object.hashCode()` is derived from the object's identity (typically based on its address, but the Javadoc does not guarantee that).

19) Why we override equals() method?

Corrected

> **CORRECTED** The source's example made two employees 'equal' just because their salary matched, and it did not override hashCode() (which must always be done together with equals()).
>
> Checked against: Object.

The default `equals()` from `Object` compares **identity** (`==`): two different objects are never equal. Override it when two objects with the same **state** should count as equal, for example two `Employee` objects with the same id.

The contract: `equals()` must be reflexive, symmetric, transitive and consistent, and `x.equals(null)` must be false. Whenever you override it, also override `hashCode()` with the same fields.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    Employee other = (Employee) obj;
    return this.id == other.id;                // compare identifying fields, not just salary
}

@Override
public int hashCode() { return Integer.hashCode(id); }
```

Without a matching `hashCode()`, equal objects can land in different buckets and a `HashSet` will treat them as different.

20) How to synchronize List, Set, and Map elements?

Updated

UPDATED Added what the original leaves out: the wrappers do not make iteration safe, and better options exist.

Checked against: ArrayList.

Yes, Collections class provides methods to make List, Set or Map elements as synchronized:

```
public static List synchronizedList(List l)
public static Set synchronizedSet(Set s)
public static SortedSet synchronizedSortedSet(SortedSet s)
public static Map synchronizedMap(Map m)
public static SortedMap synchronizedSortedMap(SortedMap m)
```

These methods provide thread-safe collection operations by enclosing the supplied collection (List, Set, or Map) in a synchronised wrapper. This avoids dangerous race circumstances and ensures data integrity, especially in situations where multiple threads may access or modify the collection concurrently.

Important: the wrapper locks each individual method call, but **iterating** is several calls, so you must lock the collection yourself:

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
synchronized (list) {              // required while iterating
    for (String s : list) { ... }
}
```

For better concurrency, prefer `ConcurrentHashMap` and `CopyOnWriteArrayList` (or `CopyOnWriteArraySet`), whose iterators never throw `ConcurrentModificationException`.

21) What are the advantages of the generic collection?

Generic collections in Java provide several advantages over non-generic collections:

1. Elimination of Type Casting

A major benefit of utilising generic collections is not requiring explicit type casting. When retrieving elements from non-generic collections, we must cast them to the correct data type; if this is done incorrectly, runtime issues may result. By using generics, we can work directly with the proper data type without casting because the compiler guarantees type safety at compilation time.

2. Type Safety

The compiler guarantees that only the proper data types are utilised within the collection by using generic collections, that offer type safety. By doing this, runtime issues caused by mismatched data types are avoided. When constructing a generic collection, we can reduce the possibility of errors resulting from erroneous data types by clearly defining the types of components that the collection can include by using the type argument.

3. Compile-Time Checking:

Compile-time checking is an additional benefit of generic collections. Type checks and problems relating to types can be found by the compiler prior to code execution because the type information is available at compile time. This enhances code stability and reliability by spotting any flaws early in the development process. Coders can lessen the chance of runtime mistakes and strengthen their code by proactively addressing errors found during compilation.

22) What is hash-collision in Hashtable and how it is handled in Java? Corrected

CORRECTED The source mixed several techniques together and described 'open addressing', which Java's HashMap and Hashtable do not use.

Checked against: HashMap, Hashtable, IdentityHashMap, HashMap.java (JDK 21 source).

A **hash collision** happens when two different keys map to the same bucket.

Hashtable and **HashMap** both handle it with **separate chaining**: each bucket holds a chain of entries, and keys inside a bucket are compared with `equals()`.

- **HashMap, Java 8 and later:** when a bucket gets **more than 8 entries** (adding the 9th triggers it) and the table has at least 64 buckets, the chain is converted to a **red-black tree**, so lookups in a crowded bucket cost O(log n) instead of O(n). If the table is still smaller than 64 buckets, it is resized instead. When keys are `Comparable`, HashMap uses their order to break ties.
- **Hashtable** keeps plain linked chains (no trees).
- **Open addressing** (linear probing and similar) is a different technique, in which a colliding entry is stored in another slot of the array. Some tables use it (for example `IdentityHashMap`, which its Javadoc calls a simple linear-probe hash table), but not HashMap or Hashtable.

Fewer collisions come from a good `hashCode()` and from resizing when the load factor is exceeded. The thresholds are implementation details, not part of HashMap's documented contract. Confirmed in the JDK 21 source (`TREEIFY_THRESHOLD = 8`, `UNTREEIFY_THRESHOLD = 6`, `MIN_TREEIFY_CAPACITY = 64`) and by running it: with 64 buckets a bin became a tree when the 9th colliding key was added, while a default 16-bucket map resized instead.

23) What is the Dictionary class? Corrected

CORRECTED The source said Hashtable 'replaced' Dictionary, and that elements() gives a 'list'. Hashtable is a **SUBCLASS** of Dictionary; the Map interface is what replaced it. elements() returns an Enumeration.

Checked against: Dictionary, Hashtable.

`Dictionary` is an **abstract** class from Java 1.0 that maps keys to values. The Javadoc marks it **obsolete**: "New implementations should implement the Map interface, rather than extending this class."

- `put(key, value)`, `get(key)`, `remove(key)`, `size()`, `isEmpty()`
- `keys()` returns an `Enumeration` of the keys; `elements()` returns an `Enumeration` of the values

`Hashtable` extends `Dictionary` (and, since Java 1.2, implements `Map`). In new code, use `Map` implementations.

24) What is the default size of load factor in hashing based collection? Corrected

CORRECTED The source called 12 the 'default capacity of Map'. The default **CAPACITY** is 16 buckets; 12 is the **RESIZE THRESHOLD**.

Checked against: HashMap, Hashtable.

The default **load factor** is **0.75** for `HashMap`, `HashSet` and `LinkedHashMap`, and also for `Hashtable`.

- **Capacity** = number of buckets. A new `HashMap` has a default capacity of **16** (a new `Hashtable`: **11**).
- **Threshold** = capacity x load factor = 16 x 0.75 = **12**.
- When the number of entries **exceeds** the threshold, the table is rehashed to about twice the number of buckets (16 becomes 32, and the new threshold is 24).

Trade-off: a higher load factor saves memory but causes more collisions; a lower one is faster but wastes memory. Confirmed on JDK 21: after the first put a HashMap has 16 buckets and threshold 12; with 13 entries it has 32 buckets and threshold 24.

25) What do you understand by fail-fast?

Corrected

CORRECTED The source was vague about what counts as a modification and implied the check is guaranteed. It is not.
Checked against: ArrayList, HashMap.

A **fail-fast** iterator throws `ConcurrentModificationException` if the collection is **structurally modified** after the iterator was created, by anything other than the iterator's own `remove()` (or `add()` for a ListIterator). A structural modification adds or removes elements; just calling `set()` on a list is not one.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
for (Integer i : list) {
    if (i == 1) list.remove(i);      // ConcurrentModificationException on the next iteration
}
// Safe: list.removeIf(i -> i == 1);  or use the iterator's own remove()
```

- It is **best effort**: the Javadoc says it cannot be guaranteed and should be used only to detect bugs, never relied on for correctness. For example, removing the **second-to-last** element of an ArrayList inside a for-each loop throws nothing, because the loop ends before the check happens (confirmed by running it).
- It is used by ArrayList, LinkedList, HashMap, TreeMap, Vector and others (internally a modification counter is compared).
- The concurrent collections (`ConcurrentHashMap`, `CopyOnWriteArrayList`) never throw it: their iterators are **weakly consistent** or work on a snapshot.

"Fail-safe" is a common informal term, but it is not an official Java term. The Javadoc uses 'fail-fast' and 'weakly consistent'.

26) What are the differences between Array and ArrayList?

Corrected

CORRECTED The source called arrays 'static type' and ArrayList 'dynamic type' (which means nothing here), and its declaration example lost the generic type because the page's HTML swallowed `<String>`.
Checked against: ArrayList.

Aspect	Array	ArrayList
Size	Fixed	Resizable
Element types	Primitives and objects (`int[]`)	Objects only (`ArrayList<Integer>` boxes values)
Generics	Not generic; checks the element type at runtime (`ArrayStoreException`)	Generic: `ArrayList<String>` is checked at compile time
Declaration	`int[] a = new int[5];`	`ArrayList<String> list = new ArrayList<>();`
Access	`a[i]`, `a.length`	`get(i)`, `set(i, e)`, `size()`
Dimensions	Multi-dimensional arrays are supported	One-dimensional (use a list of lists)
Framework	Part of the language, not the Collections Framework	Part of the Collections Framework (and backed by an array internally)

27) What are the differences between the length of an Array and the size of an ArrayList?

The length of an array can be obtained using the property of length whereas ArrayList does not support length property, but we can use size() method to get the number of objects in the list.

Finding the length of the array

```
int[] array = new int[4];
System.out.println("The size of the array is " + array.length);
```

Finding the size of the ArrayList

```
ArrayList<String> list=new ArrayList<String>();
list.add("ankit");
list.add("nippun");
System.out.println(list.size());
```

28) How to convert ArrayList to Array and Array to ArrayList?

Corrected

CORRECTED The source said asList() 'accepts the List object' and its toArray() text was garbled. It also did not warn that Arrays.asList() returns a fixed-size list backed by the array.

Checked against: Arrays, List.

Array to ArrayList:

```
String[] arr = {"a", "b", "c"};
List<String> fixedSize = Arrays.asList(arr);             // fixed-size view of the array: add/remove throw
List<String> list = new ArrayList<>(Arrays.asList(arr)); // a real, resizable ArrayList
List<String> immutable = List.of(arr);                   // Java 9+: immutable, no nulls
```

- `Arrays.asList()` returns a **fixed-size list backed by the array**: `set()` writes through to the array, but `add()` and `remove()` throw `UnsupportedOperationException`.
- For a primitive array such as `int[]`, `Arrays.asList(intArray)` gives a one-element `List<int[]>`. Use `Arrays.stream(a).boxed().toList()` instead.

ArrayList to array:

```
String[] back = list.toArray(new String[0]);   // preferred form; new String[list.size()] also works
```

The fixed-size behaviour was confirmed by running it on JDK 21.

29) How to make Java ArrayList Read-Only?

Corrected

CORRECTED The source said to use `unmodifiableCollection()` but its code used `unmodifiableList()`, and it did not explain that the result is only a **VIEW**.

Checked against: List.

```
List<String> original = new ArrayList<>(List.of("Apple", "Banana"));

List<String> view = Collections.unmodifiableList(original);   // read-only VIEW
view.add("Grape");                                            // UnsupportedOperationException
original.add("Grape");                                        // allowed, and the view now shows Grape too

List<String> copy = List.copyOf(original);                    // Java 10+: independent immutable COPY
```

- `Collections.unmodifiableList(list)` blocks `add`, `remove` and `set` through the wrapper, but changes to the original list show through.
- `List.copyOf(list)` (Java 10) and `List.of(...)` (Java 9) are truly immutable and disallow `null` elements.
- `Collections.unmodifiableCollection()` is the same idea for a general `Collection`.

View versus copy was confirmed by running it on JDK 21.

30) How to remove duplicates from ArrayList?

There are two ways to remove duplicates from the ArrayList.

- **Using HashSet:** By using HashSet we can remove the duplicate element from the ArrayList, but it will not then preserve the insertion order.
- **Using LinkedHashSet:** We can also maintain the insertion order by using LinkedHashSet instead of HashSet.

The Process to remove duplicate elements from ArrayList using the LinkedHashSet:

- Copy all the elements of ArrayList to LinkedHashSet.
- Empty the ArrayList using clear() method, which will remove all the elements from the list.
- Now copy all the elements of LinkedHashset to ArrayList.

31) How to reverse ArrayList?

To reverse an ArrayList, we can use reverse() method of Collections class. Consider the following example.

ReverseArrayList.java

```
import java.util.*;
public class ReverseArrayList {
public static void main(String[] args) {
     List list = new ArrayList<>();
     list.add(10);
     list.add(50);
     list.add(30);
     Iterator i = list.iterator();
     System.out.println("printing the list....");
     while(i.hasNext())
     {
         System.out.println(i.next());
     }
     Iterator i2 = list.iterator();
     Collections.reverse(list);
     System.out.println("printing list in reverse order....");
     while(i2.hasNext())
     {
         System.out.println(i2.next());
     }
    }
}
```

Output

```
printing the list....
10
50
30
printing list in reverse order....
30
50
10
```

32) How to sort ArrayList in descending order?

To sort the ArrayList in descending order, we can use the reverseOrder() method of Collections class. Consider the following example.

ReverseArrayList.java

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.Iterator;
import java.util.List;

public class ReverseArrayList {
public static void main(String[] args) {
         List list = new ArrayList<>();
         list.add(10);
         list.add(50);
         list.add(30);
         list.add(60);
         list.add(20);
         list.add(90);

         Iterator i = list.iterator();
         System.out.println("printing the list....");
         while(i.hasNext())
         {
                 System.out.println(i.next());
         }

        Comparator cmp = Collections.reverseOrder();
        Collections.sort(list,cmp);
         System.out.println("printing list in descending order....");
         Iterator i2 = list.iterator();
         while(i2.hasNext())
         {
                 System.out.println(i2.next());
         }

}
}
```

Output

```
printing the list....
10
50
30
60
20
90
printing list in descending order....
90
60
50
30
20
10
```

33) How to synchronize ArrayList?

Corrected

CORRECTED The source's description of CopyOnWriteArrayList was garbled ('a customised version of Java's built-in synchronization-supporting List interface') and it left out that iterating a synchronized list still needs a manual lock.

Checked against: ArrayList.

Three common ways to get a thread-safe list:

- **`Collections.synchronizedList(new ArrayList<>())`**: a wrapper that locks every method call. Iterating is several calls, so you must wrap the loop in `synchronized (list) { ... }`.
- **`CopyOnWriteArrayList`**: every change (add, set, remove) copies the underlying array, and iterators work on a snapshot, so they never throw `ConcurrentModificationException`. It is good for lists that are read often and changed rarely, and expensive when writes are frequent.
- **`Vector`**: legacy; each method is synchronized. Not recommended for new code.

The `ArrayList` Javadoc itself says to wrap the list with `Collections.synchronizedList` at creation time if multiple threads may modify it.

34) When to use ArrayList and LinkedList? Corrected

CORRECTED The source said LinkedList gives constant-time insertion and deletion 'at arbitrary positions'. That is only true once you already hold the position; finding the position is O(n).

Checked against: LinkedList, ArrayList.

Default to `ArrayList`.

- **ArrayList:** fast random access (`get(i)` is O(1)), fast iteration and appending, smaller memory footprint, good cache behaviour. Insertion or removal in the middle shifts elements, which costs O(n) but is a fast memory copy.
- **LinkedList:** consider it when you add or remove at **both ends** (although `ArrayDeque` is usually faster for queues and stacks), or when you repeatedly insert or remove at the iterator's current position so you never search for the index. `get(i)`, and finding where to insert, is O(n).

In most real programs `ArrayList` wins, so choose a `LinkedList` only when measurements show a benefit.

JDBC

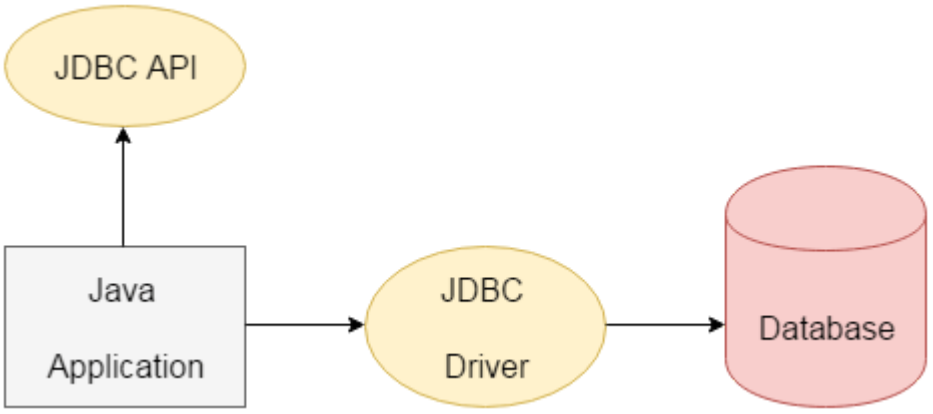
JDBC Interview Questions and Answers

Questions in this topic (30)

1) What is JDBC?
2) What is JDBC Driver?
3) What are the steps to connect to the database in java?
4) What are the JDBC API components?
5) What are the JDBC statements?
6) What is the return type of Class.forName() method?
7) What are the differences between Statement and PreparedStatement interface?
8) How can we set null value in JDBC PreparedStatement?
9) What are the benefits of PreparedStatement over Statement?
10) What are the differences between execute, executeQuery, and executeUpdate?
11) What are the different types of ResultSet?
12) What are the differences between ResultSet and RowSet?
13) How can we execute stored procedures using CallableStatement?
14) What is the role of the JDBC DriverManager class?
15) What are the functions of the JDBC Connection interface?
16) What does the JDBC ResultSet interface?
17) What does the JDBC ResultSetMetaData interface?
18) What does the JDBC DatabaseMetaData interface?
19) Which interface is responsible for transaction management in JDBC?
20) What is batch processing and how to perform batch processing in JDBC?
21) What are CLOB and BLOB data types in JDBC?
22) What are the different types of lockings in JDBC?
23) How can we store and retrieve images from the database?
24) How can we store the file in the Oracle database?
25) How can we retrieve the file in the Oracle database?
26) What are the differences between stored procedure and functions?
27) How can we maintain the integrity of a database by using JDBC?
28) What is the JDBC Rowset?
29) What is the major difference between java.util.Date and java.sql.Date data type?
30) What does JDBC setMaxRows method do?

1) What is JDBC?

JDBC is a Java API that is used to connect and execute the query to the database. JDBC API uses JDBC drivers to connect to the database. JDBC API can be used to access tabular data stored into any relational database.



More details.

2) What is JDBC Driver?

Corrected

CORRECTED The source said the JDBC-ODBC bridge is only 'discouraged'. Oracle removed it from the JDK in Java 8.

Checked against: Oracle: JDBC-ODBC Bridge (JDK 7 guide).

JDBC Driver is a software component that enables Java application to interact with the database. There are 4 types of JDBC drivers:

1. **JDBC-ODBC bridge driver:** The JDBC-ODBC bridge driver uses the ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. It was removed from the JDK in Java 8, so it cannot be used on any current Java; use the database vendor's Type 4 (thin) driver instead. It is easy to use and can be easily connected to any database.
2. **Native-API driver (partially java driver):** The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in Java. Its performance is better than JDBC-ODBC bridge driver. However, the native driver must be installed on each client machine.
3. **Network Protocol driver (fully java driver):** The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is entirely written in Java. There is no requirement of the client-side library because of the application server that can perform many tasks like auditing, load balancing, logging, etc.
4. **Thin driver (fully java driver):** The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as the thin driver. It is entirely written in Java language. Its performance is better than all other drivers however these drivers depend upon the database.

More details.

3) What are the steps to connect to the database in java?

Updated

UPDATED The steps are the classic JDBC 3 style. Since JDBC 4 (Java 6) the driver is found automatically, and connections must always be closed.

Checked against: java.sql.DriverManager, java.sql.Statement.

The following steps are used in database connectivity.

The forName() method of the Class class is used to register the driver class. This method is used to load the driver class dynamically. Consider the following example to register OracleDriver class.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

The getConnection() method of DriverManager class is used to establish the connection with the database. The syntax of the getConnection() method is given below.

```
1) public static Connection getConnection(String url)throws SQLException
2) public static Connection getConnection(String url,String name,String password)
throws SQLException
```

Consider the following example to establish the connection with the Oracle database.

```
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","password");
```

The createStatement() method of Connection interface is used to create the Statement. The object of the Statement is responsible for executing queries with the database.

```
public Statement createStatement()throws SQLException
```

consider the following example to create the statement object

```
Statement stmt=con.createStatement();
```

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

Syntax of executeQuery() method is given below.

```
public ResultSet executeQuery(String sql)throws SQLException
```

Example to execute the query

```
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next()){
System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

However, to perform the insert and update operations in the database, executeUpdate() method is used which returns the boolean value to indicate the successful completion of the operation.

By closing connection, object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax of close() method is given below.

```
public void close()throws SQLException
```

Consider the following example to close the connection.

```
con.close();
```

- **Registering the driver class:**
- **Creating connection:**
- **Creating the statement:**
- **Executing the queries:**
- **Closing connection:**

More details.

Modern version of these steps:

```
// No Class.forName(): a JDBC 4+ driver on the classpath registers itself
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement("select name from emp where id = ?")) {
    ps.setInt(1, 101);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) System.out.println(rs.getString("name"));
    }
}   // ResultSet, PreparedStatement and Connection are all closed, even if an exception is thrown
```

- `DriverManager` loads the drivers it finds through the service-provider mechanism, so `Class.forName(...)` is normally unnecessary.
- `Connection`, `Statement` and `ResultSet` are `AutoCloseable`: use **try-with-resources** so they are closed.
- The Javadoc says a `DataSource` is the **preferred** way to get a connection (it enables connection pooling).
- Do not hard-code the URL and password; read them from configuration.

4) What are the JDBC API components?

The java.sql package contains following interfaces and classes for JDBC API.

Interfaces:

- **Connection:** The Connection object is created by using getConnection() method of DriverManager class. DriverManager is the factory for connection.

- **Statement:** The Statement object is created by using createStatement() method of Connection class. The Connection interface is the factory for Statement.

- **PreparedStatement:** The PrepareStatement object is created by using prepareStatement() method of Connection class. It is used to execute the parameterized query.

- **ResultSet:** The object of ResultSet maintains a cursor pointing to a row of a table. Initially, cursor points before the first row. The executeQuery() method of Statement interface returns the ResultSet object.

- **ResultSetMetaData:** The object of ResultSetMetaData interface cotains the information about the data (table) such as numer of columns, column name, column type, etc. The getMetaData() method of ResultSet returns the object of ResultSetMetaData.

- **DatabaseMetaData:** DatabaseMetaData interface provides methods to get metadata of a database such as the database product name, database product version, driver name, name of the total number of tables, the name of the total number of views, etc. The getMetaData() method of Connection interface returns the object of DatabaseMetaData.

- **CallableStatement:** CallableStatement interface is used to call the stored procedures and functions. We can have business logic on the database through the use of stored procedures and functions that will make the performance better because these are precompiled. The prepareCall() method of Connection interface returns the instance of CallableStatement.

-
-
-

Classes:

- **DriverManager:** The DriverManager class acts as an interface between the user and drivers. It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver. It contains several methods to keep the interaction between the user and drivers.

- **Blob:** Blob stands for the binary large object. It represents a collection of binary data stored as a single entity in the database management system.

- **Clob:** Clob stands for Character large object. It is a data type that is used by various database management systems to store character files. It is similar to Blob except for the difference that BLOB represent binary data such as images, audio and video files, etc. whereas Clob represents character stream data such as character files, etc.

- **SQLException** It is an Exception class which provides information on database access errors.

5) What are the JDBC statements?

In JDBC, Statements are used to send SQL commands to the database and receive data from the database. There are various methods provided by JDBC statements such as execute(), executeUpdate(), executeQuery, etc. which helps you to interact with the database.

There is three type of JDBC statements given in the following table.

Statements	Explanation
Statement	Statement is the factory for resultset. It is used for general purpose access to the database. It executes a static SQL query at runtime.
PreparedStatement	The PreparedStatement is used when we need to provide input parameters to the query at runtime.
CallableStatement	CallableStatement is used when we need to access the database stored procedures. It can also accept runtime parameters.

6) What is the return type of Class.forName() method?

The Class.forName() method returns the object of java.lang.Class object.

7) What are the differences between Statement and PreparedStatement interface? Updated

> **UPDATED** The source gives only the performance reason. The most important reason to use PreparedStatement is protection against SQL injection.
> Checked against: OWASP SQL Injection Prevention.

Statement	PreparedStatement
The Statement interface provides methods to execute queries with the database. The statement interface is a factory of ResultSet; i.e., it provides the factory method to get the object of ResultSet.	The PreparedStatement interface is a subinterface of Statement. It is used to execute the parameterized query.
In the case of Statement, the query is compiled each time we run the program.	In the case of PreparedStatement, the query is compiled only once.
The Statement is mainly used in the case when we need to run the static query at runtime.	PreparedStatement is used when we need to provide input parameters to the query at runtime.

More details.

The most important difference is security. A `Statement` that builds SQL by concatenating user input can be attacked: input such as `' OR '1'='1` changes the meaning of the query. With a `PreparedStatement` the SQL is fixed and the values are sent as **data**, so the database can never run them as SQL.

```
// Vulnerable
stmt.executeQuery("select * from users where name = '" + name + "'");

// Safe
PreparedStatement ps = con.prepareStatement("select * from users where name = ?");
ps.setString(1, name);
```

OWASP lists parameterized queries as the first and main defence. Also, 'compiled only once' is a possible optimization (the database or driver may cache the plan), not something the JDBC specification guarantees.

8) How can we set null value in JDBC PreparedStatement?

By using setNull() method of PreparedStatement interface, we can set the null value to an index. The syntax of the method is given below.

```
void setNull(int parameterIndex, int sqlType) throws SQLException
```

9) What are the benefits of PreparedStatement over Statement?

Updated

UPDATED Same gap as the previous question: the security benefit is the main one.

Checked against: OWASP SQL Injection Prevention.

The benefits of using PreparedStatement over Statement interface is given below.

- The PreparedStatement performs faster as compare to Statement because the Statement needs to be compiled everytime we run the code whereas the PreparedStatement compiled once and then execute only on runtime.
- PreparedStatement can execute Parameterized query whereas Statement can only run static queries.
- The query used in PreparedStatement is appeared to be similar every time. Therefore, the database can reuse the previous access plan whereas, Statement inline the parameters into the String, therefore, the query doesn't appear to be same everytime which prevents cache reusage.

Security: parameters are passed separately from the SQL text, so a `PreparedStatement` protects against SQL injection, while a `Statement` built by string concatenation does not. This is the benefit interviewers expect you to mention first.

10) What are the differences between execute, executeQuery, and executeUpdate?

execute	executeQuery	executeUpdate
The execute method can be used for any SQL statements(Select and Update both).	The executeQuery method can be used only with the select statement.	The executeUpdate method can be used to update/delete/insert operations in the database.
The execute method returns a boolean type value where true indicates that the ResultSet s returned which can later be extracted and false indicates that the integer or void value is returned.	The executeQuery() method returns a ResultSet object which contains the data retrieved by the select statement.	The executeUpdate() method returns an integer value representing the number of records affected where 0 indicates that query returns nothing.

11) What are the different types of ResultSet?

ResultSet is categorized by the direction of the reading head and sensitivity or insensitivity of the result provided by it. There are three general types of ResultSet.

Type	Description
ResultSet.TYPE_FORWARD_ONLY	The cursor can move in the forward direction only.
ResultSet.TYPE_SCROLL_INSENSITIVE	The cursor can move in both the direction (forward and backward). The ResultSet is not sensitive to the changes made by the others to the database.
ResultSet.TYPE_SCROLL_SENSITIVE	The cursor can move in both the direction. The ResultSet is sensitive to the changes made by the others to the database.

12) What are the differences between ResultSet and RowSet?

ResultSet	RowSet
ResultSet cannot be serialized as it maintains the connection with the database.	RowSet is disconnected from the database and can be serialized.
ResultSet object is not a JavaBean object	RowSet object is a JavaBean object.
ResultSet is returned by the executeQuery() method of Statement Interface.	Rowset Interface extends ResultSet Interface and returned by calling the RowSetProvider.newFactory().createJdbcRowSet() method.
ResultSet object is non-scrollable and non-updatable by default.	RowSet object is scrollable and updatable by default.

13) How can we execute stored procedures using CallableStatement?

Following are the steps to create and execute stored procedures. Here, we are creating a table user420 by using a stored procedure and inserting values into it.

To call the stored procedure, you need to create it in the database. Here, we are assuming that the stored procedure looks like this.

```
create or replace procedure "INSERTR"
(id IN NUMBER,
name IN VARCHAR2)
is
begin
insert into user420 values(id,name);
end;
/
```

The table structure is given below:

```
create table user420(id number(10), name varchar2(200));
```

```
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
```

```
CallableStatement stmt=con.prepareCall("{call insertR(?,?)}");
```

```
stmt.setInt(1,1011);
stmt.setString(2,"Amit");
stmt.execute();
```

```
import java.sql.*;
public class Proc {
public static void main(String[] args) throws Exception{

Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

CallableStatement stmt=con.prepareCall("{call insertR(?,?)}");
stmt.setInt(1,1011);
stmt.setString(2,"Amit");
stmt.execute();

System.out.println("success");
}
}
```

- **Create the procedure in the database.**
- **Establish a network connection.**
- **Create the Object of CallableStatement.**
- **Provide the values and execute the query by using the following syntax.**
- **Check the database; the values will be found there. However, the complete code will look like the following.**

14) What is the role of the JDBC DriverManager class?

The DriverManager class acts as an interface between user and drivers. It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver. The DriverManager class maintains a list of Driver classes that have registered themselves by calling the method DriverManager.registerDriver().

More details.

15) What are the functions of the JDBC Connection interface?

The **Connection interface** maintains a session with the database. It can be used for transaction management. It provides factory methods that return the instance of Statement, PreparedStatement, CallableStatement, and DatabaseMetaData.

More details.

16) What does the JDBC ResultSet interface?

The ResultSet object represents a row of a table. It can be used to change the cursor pointer and get the information from the database. By default, ResultSet object can move in the forward direction only and is not updatable. However, we can make this object to move the forward and backward direction by passing either TYPE_SCROLL_INSENSITIVE or TYPE_SCROLL_SENSITIVE in createStatement(int, int) method.

More details.

17) What does the JDBC ResultSetMetaData interface?

The ResultSetMetaData interface returns the information of table such as the total number of columns, column name, column type, etc.

More details.

18) What does the JDBC DatabaseMetaData interface?

The DatabaseMetaData interface returns the information of the database such as username, driver name, driver version, number of tables, number of views, etc. Consider the following example.

```
import java.sql.*;
class Dbmd{
public static void main(String args[]){
try{
Class.forName("oracle.jdbc.driver.OracleDriver");

Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
DatabaseMetaData dbmd=con.getMetaData();

System.out.println("Driver Name: "+dbmd.getDriverName());
System.out.println("Driver Version: "+dbmd.getDriverVersion());
System.out.println("UserName: "+dbmd.getUserName());
System.out.println("Database Product Name: "+dbmd.getDatabaseProductName());
System.out.println("Database Product Version: "+dbmd.getDatabaseProductVersion());

con.close();
}catch(Exception e){ System.out.println(e);}
}
}
```

Output

```
Driver Name: Oracle JDBC Driver
Driver Version: 10.2.0.1.0XE
Database Product Name: Oracle
Database Product Version: Oracle Database 10g Express Edition Release 10.2.0.1.0 -Production
```

More details.

19) Which interface is responsible for transaction management in JDBC?

The **Connection interface** provides methods for transaction management such as commit(), rollback() etc.

More details.

20) What is batch processing and how to perform batch processing in JDBC?

By using the batch processing technique in JDBC, we can execute multiple queries. It makes the performance fast. The java.sql.Statement and java.sql.PreparedStatement interfaces provide methods for batch processing. The batch processing in JDBC requires the following steps.

- Load the driver class
- Create Connection
- Create Statement
- Add query in the batch
- Execute the Batch
- Close Connection

Consider the following example to perform batch processing using the Statement interface.

```
import java.sql.*;
class FetchRecords{
public static void main(String args[])throws Exception{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
con.setAutoCommit(false);

Statement stmt=con.createStatement();
stmt.addBatch("insert into user420 values(190,'abhi',40000)");
stmt.addBatch("insert into user420 values(191,'umesh',50000)");

stmt.executeBatch();//executing the batch

con.commit();
con.close();
}}
```

More details.

21) What are CLOB and BLOB data types in JDBC? Updated

> **UPDATED** The source gave maximum sizes (2 GB on MySQL, 128 GB on Oracle) that are not right. Limits depend on the database and the exact type.
>
> Checked against: MySQL storage requirements, Oracle LOB documentation.

BLOB: Blob can be defined as the variable-length, binary large object which is used to hold the group of Binary data such as voice, images, and mixed media. It can hold up to 2GB data on MySQL database and 128 GB on Oracle database. BLOB is supported by many databases such as MySQL, Oracle, and DB2 to store the binary data (images, video, audio, and mixed media).

CLOB: Clob can be defined as the variable-length, character-large object which is used to hold the character-based data such as files in many databases. It can hold up to 2 GB on MySQL database, and 128 GB on Oracle Database. A CLOB is considered as a character string.

Maximum sizes depend on the database and the type. In MySQL, for example, `BLOB` holds up to 64 KB, `MEDIUMBLOB` up to 16 MB and `LONGBLOB` up to 4 GB (the same limits apply to `TEXT`, `MEDIUMTEXT` and `LONGTEXT`). Oracle's documentation says a single LOB can be 8 to 128 terabytes, depending on how the database is configured. Check your database's documentation instead of relying on a single number.

22) What are the different types of lockings in JDBC?

A lock is a certain type of software mechanism by using which, we can restrict other users from using the data resource. There are four type of locks given in JDBC that are described below.

- **Row and Key Locks:** These type of locks are used when we update the rows.
- **Page Locks:** These type of locks are applied to a page. They are used in the case, where a transaction remains in the process and is being updated, deleting, or inserting some data in a row of the table. The database server locks the entire page that contains the row. The page lock can be applied once by the database server.
- **Table locks:** Table locks are applied to the table. It can be applied in two ways, i.e., shared and exclusive. Shared lock lets the other transactions to read the table but not update it. However, The exclusive lock prevents others from reading and writing the table.
- **Database locks:** The Database lock is used to prevent the read and update access from other transactions when the database is open.

23) How can we store and retrieve images from the database?

By using the PreparedStatement interface, we can store and retrieve images. Create a table which contains two columns namely NAME and PHOTO.

```
CREATE TABLE  "IMGTABLE"
   (    "NAME" VARCHAR2(4000),
        "PHOTO" BLOB
   )
```

Consider the following example to store the image in the database.

```
import java.sql.*;
import java.io.*;
public class InsertImage {
public static void main(String[] args) {
try{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

PreparedStatement ps=con.prepareStatement("insert into imgtable values(?,?)");
ps.setString(1,"sonoo");

FileInputStream fin=new FileInputStream("d:\\g.jpg");
ps.setBinaryStream(2,fin,fin.available());
int i=ps.executeUpdate();
System.out.println(i+" records affected");

con.close();
}catch (Exception e) {e.printStackTrace();}
}
}
```

Consider the following example to retrieve the image from the table.

```
import java.sql.*;
import java.io.*;
public class RetrieveImage {
public static void main(String[] args) {
try{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

PreparedStatement ps=con.prepareStatement("select * from imgtable");
ResultSet rs=ps.executeQuery();
if(rs.next()){//now on 1st row

Blob b=rs.getBlob(2);//2 means 2nd column data
byte barr[]=b.getBytes(1,(int)b.length());//1 means first image

FileOutputStream fout=new FileOutputStream("d:\\sonoo.jpg");
fout.write(barr);

fout.close();
}//end of if
System.out.println("ok");

con.close();
}catch (Exception e) {e.printStackTrace();      }
}
}
```

More details.

24) How can we store the file in the Oracle database?

The setCharacterStream() method of PreparedStatement interface is used to set character information into the parameterIndex. For storing the file into the database, CLOB (Character Large Object) datatype is used in the table. For example:

```
CREATE TABLE  "FILETABLE"
   (    "ID" NUMBER,
        "NAME" CLOB
   )
```

Java Code

```
import java.io.*;
import java.sql.*;

public class StoreFile {
public static void main(String[] args) {
try{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

PreparedStatement ps=con.prepareStatement(
"insert into filetable values(?,?)");

File f=new File("d:\\myfile.txt");
FileReader fr=new FileReader(f);

ps.setInt(1,101);
ps.setCharacterStream(2,fr,(int)f.length());
int i=ps.executeUpdate();
System.out.println(i+" records affected");

con.close();

}catch (Exception e) {e.printStackTrace();}
}
}
```

25) How can we retrieve the file in the Oracle database?

The getClob() method of PreparedStatement is used to get file information from the database. Let's see the table structure of the example to retrieve the file.

```
CREATE TABLE  "FILETABLE"
   (    "ID" NUMBER,
        "NAME" CLOB
   )
```

The example to retrieve the file from the Oracle database is given below.

```
import java.io.*;
import java.sql.*;

public class RetrieveFile {
public static void main(String[] args) {
try{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

PreparedStatement ps=con.prepareStatement("select * from filetable");
ResultSet rs=ps.executeQuery();
rs.next();//now on 1st row

Clob c=rs.getClob(2);
Reader r=c.getCharacterStream();

FileWriter fw=new FileWriter("d:\\retrivefile.txt");

int i;
while((i=r.read())!=-1)
fw.write((char)i);

fw.close();
con.close();

System.out.println("success");
}catch (Exception e) {e.printStackTrace();      }
}
}
```

26) What are the differences between stored procedure and functions?

Updated

UPDATED Added: some of the stored procedure vs function differences depend on the database.
Checked against: SQL Server: CREATE FUNCTION.

The differences between stored procedures and functions are given below:

Stored Procedure	Function
Is used to perform business logic.	Is used to perform the calculation.
Must not have the return type.	Must have the return type.
May return 0 or more values.	May return only one value.
The procedure supports input and output parameters.	The function supports only input parameter.
Exception handling using try/catch block can be used in stored procedures.	Exception handling using try/catch can't be used in user-defined functions.

The rules in the table are general, and several are **database specific**. For example, Microsoft's documentation lists control-of-flow statements *except* `TRY...CATCH` as valid inside a SQL Server user-defined function, so a SQL Server function cannot use TRY...CATCH. Other databases differ.

27) How can we maintain the integrity of a database by using JDBC?

To maintain the integrity of a database, we need to ensure the ACID properties. ACID properties mean Atomicity, Consistency, Isolation, and durability. In JDBC, Connection interface provides methods like setAutoCommit(), commit(), and rollback() which can be used to manage transaction. Let's see an example of transaction management in JDBC.

```
import java.sql.*;
class FetchRecords{
public static void main(String args[])throws Exception{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
con.setAutoCommit(false);

Statement stmt=con.createStatement();
stmt.executeUpdate("insert into user420 values(190,'abhi',40000)");
stmt.executeUpdate("insert into user420 values(191,'umesh',50000)");

con.commit();
con.close();
}}
```

28) What is the JDBC Rowset?

JDBC Rowset is the wrapper of ResultSet. It holds tabular data like ResultSet, but it is easy and flexible to use. The implementation classes of RowSet interface are as follows:

- JdbcRowSet
- CachedRowSet
- WebRowSet
- JoinRowSet
- FilteredRowSet

29) What is the major difference between java.util.Date and java.sql.Date data type? Updated

> **UPDATED** Added: modern code uses the `java.time` types with JDBC.
> Checked against: PostgreSQL JDBC: date/time types, java.sql package.

The major difference between java.util.Date and java.sql.Date is that, java.sql.Date represents date without time information whereas, java.util.Date represents both date and time information.

Since JDBC 4.2 (Java 8) you can read and write `java.time` types directly, which avoids the old `java.util.Date` and `java.sql.Date` mix-up:

```
ps.setObject(1, LocalDate.of(2026, 1, 15));                          // DATE
LocalDate d = rs.getObject("hire_date", LocalDate.class);
LocalDateTime t = rs.getObject("created_at", LocalDateTime.class);   // TIMESTAMP
java.sql.Date legacy = java.sql.Date.valueOf(d);                     // bridge to the old API
LocalDate back = legacy.toLocalDate();
```

Support depends on the driver. For example, the PostgreSQL JDBC driver maps DATE to `LocalDate`, TIME to `LocalTime`, TIMESTAMP to `LocalDateTime` and TIMESTAMP WITH TIME ZONE to `OffsetDateTime`, but does **not** support `ZonedDateTime` or `Instant`. Read your driver's documentation.

30) What does JDBC setMaxRows method do?

The setMaxRows(int i) method limits the number of rows the database can return by using the query. This can also be done within the query as we can use the LIMIT clause in MySQL.

Java Basics Interview Questions	Java OOPs Interview Questions
Java Multithreading Interview Questions	Java String & Exception Interview Questions
Java Collection Interview Questions	JDBC Interview Questions
Servlet Interview Questions	JSP Interview Questions

Spring Interview Questions	Hibernate Interview Questions
PL/SQL Interview Questions	SQL Interview Questions
Oracle Interview Questions	Android Interview Questions
SQL Server Interview Questions	MySQL Interview Questions

Java 8 Features

Overview & General

Questions in this topic (5)

1) What are the major features introduced in Java 8?
12) How does Java 8 improve performance in large collections?
17) What are the differences between Collections.synchronizedList() and CopyOnWriteArrayList()?
41) What are the advantages of Java 8?
48) What is the function of the Type interface?

1) What are the major features introduced in Java 8?

Java 8 introduced several new features, including:

- **Lambda Expressions**: It is used to implement functional programming.
- **Functional Interfaces**: Interfaces with a single abstract method (SAM).
- **Stream API**: For processing collections in a functional style.
- **Optional Class**: It helps to avoid NullPointerException.
- **Default and Static Methods in Interfaces**: It allows interfaces to have method implementations.
- **New Date-Time API**: It provides better handling of dates and times.
- **Collectors API**: It helps in performing reduction operations on collections.

12) How does Java 8 improve performance in large collections?

- Parallel Streams – Divides data for faster execution.
- **Lazy Evaluation:** Processes only necessary elements.
- **Better Garbage Collection:** Uses Metaspace instead of PermGen.
- **Efficient Data Structures:** Example: ConcurrentHashMap.computeIfAbsent().

17) What are the differences between Collections.synchronizedList() and CopyOnWriteArrayList()?

Collections. synchronized list (): It wraps any List implementation and makes it thread-safe by synchronizing the methods. However, it can cause performance overhead due to synchronization.

Example

```
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
```

CopyOnWriteArrayList(): It is a thread-safe variant of ArrayList that allows read operations to occur concurrently without synchronization. However, write operations (add, remove) are slow because the list is copied each time a modification is made.

Example

```
List<String> copyOnWriteList = new CopyOnWriteArrayList<>();
```

Feature	Collections.synchronizedList()	CopyOnWriteArrayList()
Thread Safety	It provides thread safety through synchronizing all operations on the list.	It provides thread safety by creating a new copy of the list during a write operation.
Performance	Slower for frequent reads due to synchronization overhead.	Optimized for scenarios with frequent reads and infrequent writes.

Write Operation	It modifies the original list directly.	It modifies the list but creates a new list for every write operation.
Use Cases	It is suitable for general-purpose thread-safe lists.	It is ideal for lists with a high read-to-write ratio, such as event listener lists.
Iteration	It requires manual synchronization during iteration to avoid ConcurrentModificationException.	It allows safe iteration without manual synchronization, as it works on a copy of the list.

41) What are the advantages of Java 8?

- It is Compact, readable, and reusable code.
- It avoids boilerplate code.
- It allows Parallel operations and execution, simultaneously.
- It can be ported across operating systems.
- It is highly stable.
- It has a Stable environment.

48) What is the function of the Type interface?

In Java, Type is an interface that belongs to the java.lang.reflect package. It is used to represent more specific Java types, such as classes, interfaces, arrays, and even parameterized types. Since Java 8, it defines a method getTypeName(). The method returns a string describing this type, including information about any type parameters. We should use the Type interface and its subinterfaces when working with reflection.

Lambdas & Functional Interfaces

Questions in this topic (10)

2) What is a lambda expression in Java 8?
3) What is a functional interface?
13) How do you handle checked exceptions inside a Lambda?
16) What is a method reference in Java 8?
24) What is the difference between a predicate and a function in Java 8?
29) Can we use lambda expressions in switch statements in Java 8?
30) Why does Java 8 not allow lambda expressions in switch statements?
42) Give an example of a reference to a static method with syntax.
44) What is the meaning of String::Valueof expression?
50) What is the SAM interface in Java 8?

2) What is a lambda expression in Java 8?

A lambda expression is a short way to write anonymous functions. It helps in writing cleaner and more readable code.

Example

```
// Traditional way (before Java 8)
Runnable r1 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello, World!");
    }
};
// Lambda Expression
Runnable r2 = () -> System.out.println("Hello, World!");
```

Lambda expressions reduce boilerplate code and are mainly used with functional interfaces.

3) What is a functional interface?

A functional interface is an interface with only one abstract method. Java 8 introduced the @FunctionalInterface annotation to enforce this rule.

Example

```
@FunctionalInterface
interface MyFunctionalInterface {
    void display();  // Only one abstract method
}
```

Some built-in functional interfaces are:

1. Runnable
2. Callable
3. Predicate
4. Function
5. Consumer
6. Supplier

13) How do you handle checked exceptions inside a Lambda?

Since lambda expressions cannot throw checked exceptions, we must wrap them inside a try-catch or use a wrapper function.

Example

```
Function<String, Integer> parseInt = str -> {
    try {
        return Integer.parseInt(str);
    } catch (NumberFormatException e) {
        return 0;
    }
};
System.out.println(parseInt.apply("123"));  // 123
System.out.println(parseInt.apply("ABC"));  // 0
```

16) What is a method reference in Java 8?

A method reference in Java 8 is a shorthand notation of a lambda expression to call a method. It refers to a method of an object or a class. It can be used with built-in functional interfaces. Four types of method references are listed below:

- Static Method Reference
- Instance Method Reference of a particular object
- Referencing an instance method of an unspecified object belonging to a specific class.
- Constructor Reference.

24) What is the difference between a predicate and a function in Java 8?

Feature	Predicate	Function
Purpose	The predicate is used to test if something is true or false. It returns a Boolean result. It's commonly used in situations like filtering or checking conditions.	Function is used to transform or convert an object into another type of object, and it returns a result that may be different from the input.
Input	It takes an argument of type T.	It takes an argument of type T.
Output	Returns a boolean value.	Returns a value of type R.
Abstract Method	test(T t)	apply(T t)
Use Case	Used for filtering or conditional checks.	Used for transforming data.

29) Can we use lambda expressions in switch statements in Java 8?

Java 8 does not support using lambda expressions inside a switch statement. The switch statement is designed to work with specific values like numbers, characters, strings, and enums.

30) Why does Java 8 not allow lambda expressions in switch statements?

A switch statement checks a value and matches it against different case labels. Since lambda is not a value, it cannot be used as a case label. That’s why Java 8 does not allow it.

Even though we cannot use a lambda inside a switch, we can achieve the same effect using a Map with functional interfaces. It allows us to execute different pieces of code based on a key, similar to a switch.

Example: Using a Map Instead of Switch

Example

```
import java.util.HashMap;
import java.util.Map;
import java.util.function.Supplier;
public class LambdaSwitchExample {
    public static void main(String[] args) {
        String command = "GREET"; // This simulates an input value
// Creating a map where keys are commands and values are lambda expressions
        Map<String, Supplier<String>> actionMap = new HashMap<>();
        actionMap.put("GREET", () -> "Hello, how are you?");
        actionMap.put("BYE", () -> "Goodbye, see you later!");
        actionMap.put("THANKS", () -> "You're welcome!");
// Get the output based on the command, or return a default message
  String result = actionMap.getOrDefault(command, () -> "Unknown command").get();
// Print the result
        System.out.println(result);
    }
}
```

Output:

```
Hello, how are you?
```

42) Give an example of a reference to a static method with syntax.

In Java 8, method references are a shorthand for lambda expressions that refer to methods directly.

Syntax:

```
ClassName::staticMethodName
```

Example:

```
Math::sqrt (refers to the sqrt() method of the Math class)
```

44) What is the meaning of String::Valueof expression?

String::Valueof() expression is a static method reference to the valueOf() method of the String class.

50) What is the SAM interface in Java 8?

In Java 8, SAM stands for Single Abstract Method interface. It is also known as a functional interface. It contains only one abstract method but multiple static and default methods. It serves as a foundation for a lambda expression and method reference. If we want to make an interface a functional interface, mark it with the annotation @FunctionalInterface. It provides a functional programming paradigm.

Some examples of SAM interfaces in Java are Runnable, Callable, Comparator, and ActionListener.

Default & Static Methods

Questions in this topic (5)

6) What are the default methods in Java 8 interfaces?

9) Can we override default methods in Java 8?

10) How does Java 8 handle multiple default methods in interfaces?

18) What is the default keyword in Java 8?

43) What are static methods in interfaces?

6) What are the default methods in Java 8 interfaces?

Before Java 8, interfaces could only have abstract methods. With Java 8, we can now have default methods inside interfaces.

Example

```
interface MyInterface {
    default void show() {
        System.out.println("This is a default method.");
    }
}
```

- Allows backward compatibility without breaking existing code.
- Helps in providing common functionality to multiple classes.

9) Can we override default methods in Java 8?

- Yes, default methods can be overridden in implementing classes.
- If two interfaces have the same default method, the implementing class must override it explicitly.

10) How does Java 8 handle multiple default methods in interfaces?

If a class implements multiple interfaces that contain the same default method, how does Java resolve the conflict?

Example

```
interface A {
    default void show() {
        System.out.println("Interface A");
    }
}
interface B {
    default void show() {
        System.out.println("Interface B");
    }
}
class C implements A, B {
    @Override
    public void show() {
        A.super.show();  // Resolving conflict by calling A's method
    }
}
```

18) What is the default keyword in Java 8?

The default keyword allows us to define methods with a body in an interface. It is useful for providing a default implementation for methods in interfaces, making it easier to extend interfaces without breaking existing classes that implement the interface.

Example

```
interface MyInterface {
    default void display() {
        System.out.println("This is a default method in the interface.");
    }
}
class Main implements MyInterface {
    // No need to implement the display() method unless required
}
```

43) What are static methods in interfaces?

Static methods contain method implementations that are owned by the interface. These methods are invoked using the name of the interface. It is suitable for defining the utility methods. These methods cannot be overridden.

Streams

Questions in this topic (19)

4) What is the Stream API in Java 8?
11) What are the characteristics of a Stream?
14) How do we create an infinite Stream in Java 8?
15) What is the difference between map() and flatMap() in Java 8?
19) What are the differences between reduce() and collect() in Java 8?
20) What is the peek() method in Java 8 Streams?
21) What is the use of forEach() method in Java 8?
22) What is the purpose of the findFirst() and findAny() methods in Java 8.
23) What are the differences between forEach() and forEachOrdered() in Java 8?
25) What are the differences between Stream and Parallel Stream in Java 8?
26) What are the differences between limit() and skip() in Java Streams?
27) How do we remove duplicates from a list in Java 8?
28) What is the removeIf() method in Java 8?
31) How to convert a stream to an array in Java 8?
32) What is Stream.concat() in Java 8?
39) What are the differences between intermediate and terminal operations?
40) What is stream pipelining used for?
47) In which situation will you use the Java 8 Stream API?
49) Name the main components of a Stream.

4) What is the Stream API in Java 8?

The Stream API helps in processing collections efficiently. It provides functional-style operations such as filter, map, reduce, collect, and forEach.

Example

```
List<String> names = Arrays.asList("John", "Jane", "Jack", "Jill");
names.stream()
     .filter(name -> name.startsWith("J"))
     .forEach(System.out::println);
```

Key Features of Streams

1. Does not modify the original collection.
2. Supports parallel processing for better performance.
3. Uses lazy evaluation (operations are executed only when needed).

11) What are the characteristics of a Stream?

Java Streams have four key characteristics

- **Lazy Execution:** Operations are executed only when a terminal operation is called.
- **Pipelining:** Intermediate operations return another stream, allowing method chaining.
- **Functional Nature:** Works with functional programming concepts like map(), filter().
- **Can be Parallelized:** Supports parallel execution using parallelStream().

14) How do we create an infinite Stream in Java 8?

Using Stream.iterate() or Stream.generate().

Example (Generating Fibonacci Sequence)

```
Stream.iterate(new int[]{0, 1}, fib -> new int[]{fib[1], fib[0] + fib[1]})
      .limit(10)
      .map(fib -> fib[0])
      .forEach(System.out::println);
```

It generates the first 10 terms of the Fibonacci series.

15) What is the difference between map() and flatMap() in Java 8?

Features	map()	flatMap()
Input	It takes a stream of values as input.	It takes a stream of streams (nested data) as input.
Output	It transforms each element into a single value.	It flattens nested streams into a single stream.
Mapping Type	It uses one-to-one mapping.	It uses one-to-many mapping.
Uses Cases	It is used when each input produces a single output.	It is used when each input produces multiple outputs.
Return Type	It returns a stream of the transformed elements.	It returns a flattened stream, which could have more or fewer elements than the original stream.

19) What are the differences between reduce() and collect() in Java 8?

reduce(): reduce() is a terminal operation that combines elements of a stream into a single result. It takes a binary operator that accumulates values.

Example

```
int sum = Stream.of(1, 2, 3, 4).reduce(0, Integer::sum);
System.out.println(sum);  // Output: 10
```

collect(): collect() is also a terminal operation, but is used to accumulate elements into a collection like a List, Set, or Map. It provides more flexibility with different types of collection operations.

Example

```
List<String> names = Stream.of("Dini", "Andrew", "Peter")
                           .collect(Collectors.toList());
System.out.println(names);  // Output: [Dini, Andrew, Peter]
```

Feature	reduce()	collect()
Purpose	It combines all elements of the stream into one result.	Gathers elements into a container like a List, Set, or Map. It's more about collecting items than combining them.
Output	It produces a single value.	It produces a collection or complex result.
Flexibility	It is limited to simple reduction operations.	It is highly flexible, supports mutable reductions.
Use case	It should be used when we want to combine or aggregate elements, like adding numbers or finding the max.	It should be used when we need to gather elements into a collection, like collecting names into a list.

20) What is the peek() method in Java 8 Streams?

The peek() method is an intermediate operation that is used for debugging purposes or for inspecting elements as they pass through a stream. It does not modify the stream but allows us to perform some action on the elements as they are processed.

Example

```
List<String> names = Arrays.asList("John", "Jane", "Jack");
names.stream()
     .peek(name -> System.out.println("Processing: " + name))
     .forEach(System.out::println);
```

21) What is the use of forEach() method in Java 8?

The forEach() method in Java 8 is used to iterate over collections using a Consumer functional interface. It simplifies iteration by replacing traditional loops. It is commonly used with List and Stream for performing operations on elements.

Example

```
List<String> names = Arrays.asList("John", "Peter", "Lucy");
names.forEach(System.out::println);
```

22) What is the purpose of the findFirst() and findAny() methods in Java 8.

Two common methods used in streams are findFirst() and findAny().

findFirst(): The method always returns the first element from the stream. It ensures order is preserved, meaning the element picked is the first one in the list. It is useful when working with sequential streams, where order matters.

findAny(): The method returns any one element from the stream. It does not follow order, meaning the element picked could be any available one. It is optimized for parallel streams, making it faster in some cases.

Example

```
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(3, 6, 8, 10);
        // Always returns the first element (3)
        System.out.println(numbers.stream().findFirst().get()); // Output: 3
// May return any element when using parallelStream()
        System.out.println(numbers.parallelStream().findAny().get());
    }
}
```

Output:

```
3
8
```

23) What are the differences between forEach() and forEachOrdered() in Java 8?

Aspect	forEach()	forEachOrdered()
Order	It does not guarantee the order of processing.	It guarantees processing in the encounter order.
Parallel Stream	It may process elements in an unordered fashion.	It processes elements in the order of the source.
Performance	It is faster in parallel streams due to the lack of order.	It is slightly slower in parallel streams as it enforces order.
Use Case	It should be used when order is not important.	It should be used when maintaining order is critical.

25) What are the differences between Stream and Parallel Stream in Java 8?

Aspect	Stream	Parallel Stream

Processing	Its processing is sequential (single thread).	Its processing is parallel (multiple threads).
Performance	It is suitable for smaller datasets.	It is suitable for larger datasets.
Order	It maintains order during processing.	It does not preserve order unless handle.
Overhead	Minimal thread management overhead.	Overhead due to thread management.
Thread Usage	It uses the main thread.	It uses ForkJoinPool for multiple threads.
Creation	It uses .stream().	It uses .parallelStream() or .paralle().
Use Case	It is used when order is crucial.	It is used when performance outweighs order importance.

26) What are the differences between limit() and skip() in Java Streams?

In Java Streams, limit(n) and skip(n) are used to help control how many elements are processed.

limit(n): Get Only the First N Elements

- It keeps only the first n elements from the stream.
- If the stream has fewer than n elements, it returns all of them.
- Useful when you need to fetch a fixed number of results, like in pagination.

skip(n): Ignore the First N Elements

- It removes the first n elements and returns the rest.
- If n equals or exceeds the total elements, it gives an empty stream.
- Often used to skip past unwanted results, such as when implementing pagination.

Feature	limit()	skip()
Functionality	Limits the stream to the first n elements.	It skips the first n elements and processes the rest.
Use Case	It is used when we want to take only the first few elements.	It is used when we want to skip the initial elements and process the remaining ones
Order of Elements	It keeps the order of the first n elements.	It keeps the order of the remaining elements after skipping.
Behavior	It truncates the stream after processing the specified number of elements.	It discards the specified number of elements from the beginning of the stream.
Short-circuting	It stops processing once the limit is reached.	It processes all elements after skipping the specified number.

27) How do we remove duplicates from a list in Java 8?

In Java 8, we can use the distinct() method from the Stream API to remove duplicate elements from a list.

Example

```
import java.util.*;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 4, 4, 5);
// Removing duplicates using distinct()
List<Integer> uniqueNumbers = numbers.stream().distinct().collect(Collectors.toList());
System.out.println(uniqueNumbers); // Output: [1, 2, 3, 4, 5]
    }
}
```

Output:

```
[1, 2, 3, 4, 5]
```

28) What is the removeIf() method in Java 8?

The removeIf() method in Java 8 allows us to remove elements from a collection based on a condition. It is a default method in the Collection interface, making it easy to remove elements without using an explicit iterator or loop.

Example

Imagine we have a list of numbers and want to remove all even numbers. Instead of using a for loop, we can use removeIf() with a lambda expression:

Example

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        // Creating a list of numbers
        List<Integer> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
// Removing even numbers
        numbers.removeIf(n -> n % 2 == 0);
// Printing the updated list
        System.out.println(numbers); // Output: [1, 3, 5]
    }
}
```

Output:

```
[1, 3, 5]
```

31) How to convert a stream to an array in Java 8?

In Java 8, we can convert a Stream into an array using the toArray() method. It is useful when we want to process elements as a Stream but store the result in an array.

Example

We have a Stream containing some string values, and we want to convert it into an array of Strings.

Example

```
import java.util.Arrays;
import java.util.stream.Stream;
public class Main {
    public static void main(String[] args) {
        // Creating a Stream with some elements
        Stream<String> stream = Stream.of("A", "B", "C");
// Converting Stream to an Array
        String[] array = stream.toArray(String[]::new);
// Printing the array
        System.out.println(Arrays.toString(array));  // Output: [A, B, C]
    }
}
```

Output:

```
[A, B, C]
```

32) What is Stream.concat() in Java 8?

In Java 8, Stream.concat() is used to merge two streams into a single stream. Instead of manually iterating and combining two collections, we can use this method to process elements from both streams as one continuous stream.

Example: Merging Two Streams

Example

```
import java.util.stream.Stream;
public class Main {
    public static void main(String[] args) {
        // Creating two separate streams
        Stream<String> stream1 = Stream.of("A", "B");
        Stream<String> stream2 = Stream.of("C", "D");
// Merging both streams using Stream.concat()
        Stream<String> combinedStream = Stream.concat(stream1, stream2);
// Printing the merged stream elements
        combinedStream.forEach(System.out::print);  // Output: ABCD
    }
}
```

Output:

```
ABCD
```

39) What are the differences between intermediate and terminal operations?

Intermediate Operations: It modifies or filters the stream but does not produce a final result immediately. They are lazy, meaning that they would not execute until a terminal operation is called. Here some common examples include map(), filter(), sorted(), and distinct().

Example

```
Stream<Integer> stream = Stream.of(1, 2, 3, 4, 5)
                              .filter(n -> n % 2 == 0); // No execution yet
```

Here, the filter() method is an intermediate operation, but nothing happens until we use a terminal operation.

Terminal Operations: These trigger the execution of the stream pipeline and produce a result or perform an action. Once a terminal operation is used, the stream cannot be reused. It includes collect(), forEach(), count(), and reduce() methods.

Example

```
long count = Stream.of(1, 2, 3, 4, 5)
                   .filter(n -> n % 2 == 0)  // Intermediate
                   .count();                 // Terminal (executes the pipeline)
System.out.println(count);
```

Here, the count() method is a terminal operation, which triggers the execution of the filter() method and counts the even numbers.

Feature	Intermediate Operations	Terminal Operations
Purpose	It transforms a stream into another stream.	It produces a result or a side effect.
Execution	It is not executed until a terminal operation is invoked.	It triggers the processing of the stream.
Return Type	It always returns a new Stream, allowing further stream operations.	It returns a non-stream value (like, list, int) or no value.
Chaining	It can be chained together to form a pipeline.	It cannot be chained; it marks the end of the pipeline.

Examples	map(), filter(), dictinct(), sorted() are intermediate operations.	forEach(), collect(), reduce(), count() are terminal operations.

40) What is stream pipelining used for?

Stream pipelining in Java 8 allows us to chain multiple operations together on a stream of data. Instead of processing elements one by one manually, we can perform a sequence of transformations and computations in a single flow.

The concept is based on two types of operations:

- Intermediate Operations
- Terminal Operations

Example of Stream Pipelining

```
List<String> names = List.of("Alice", "Bob", "Charlie", "David");
long count = names.stream() // Creates a stream
                  .filter(name -> name.startsWith("A"))  // Intermediate operation
                  .count();// Terminal operation (triggers execution)
System.out.println(count); // Output: 1
```

Here, the filter() method processes the data but does not execute immediately. The count() method is a terminal operation that triggers the entire pipeline and returns the result.

In simple terms, stream pipelining writes cleaner, more efficient code by chaining multiple operations in a single flow instead of using loops.

47) In which situation will you use the Java 8 Stream API?

The Java 8 Stream API is ideal for situations where we need to process collections of data functionally and efficiently. There are some common scenarios:

- For executing lazy operations
- To perform database operations
- Use for internal iterations
- For writing functional-style programming
- Using pipeline operations

49) Name the main components of a Stream.

- A data source.
- Set of Intermediate Operations to process the data source.
- Single Terminal Operation that produces the result.

Collectors

Questions in this topic (4)

7) What are Collectors in Java 8?
33) What is Collectors.groupingBy() in Java 8?
34) What is Collectors.joining() in Java 8?
35) What is Collectors.partitioningBy() in Java 8?

7) What are Collectors in Java 8?

Collectors help in reducing the stream elements into a collection or a single value.

Example

```
List<String> names = Arrays.asList("John", "Jane", "Jack");
List<String> upperNames = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
System.out.println(upperNames);  // [JOHN, JANE, JACK]
```

Some common collectors are

1. **toList():** Collects elements into a List.
2. **toSet():** Collects elements into a Set.
3. **joining():** Concatenates elements into a String.
4. **groupingBy():** Groups elements based on a condition.

33) What is Collectors.groupingBy() in Java 8?

The Collectors. The groupingBy() method in Java 8 groups the elements of a stream based on a classification function. It is useful when we want to categorize data based on some property, like grouping words by their length or numbers by even and odd values.

Example: Grouping Words by Length

Example

```
import java.util.*;
import java.util.stream.Collectors;
import java.util.stream.Stream;
public class Main {
    public static void main(String[] args) {
        // Creating a stream of words
        Stream<String> words = Stream.of("lion", "dog", "elephant", "tiger");
// Grouping words based on their length
Map<Integer,List<String>>groupedWords=words.collect(Collectors.groupingBy(String::length));
// Printing the grouped result
        System.out.println(groupedWords);
    }
}
```

Output:

```
{3=[lion, dog, tiger], 8=[elephant]}
```

34) What is Collectors.joining() in Java 8?

The Collectors.joining() method is used to concatenate (combine) elements of a Stream into a single String. It is particularly useful when we want to convert a list of words or values into a single formatted string.

Example: Joining Words with a Space

Example

```
import java.util.stream.Collectors;
import java.util.stream.Stream;
public class Main {
    public static void main(String[] args) {
        // Creating a Stream of words
        Stream<String> words = Stream.of("Java", "8", "Features");
// Joining elements into a single string with spaces
        String result = words.collect(Collectors.joining(" "));
// Printing the final result
        System.out.println(result);
    }
}
```

Output:

```
Java 8 Features
```

35) What is Collectors.partitioningBy() in Java 8?

The Collectors.partitioningBy() method is used to split a collection into two groups based on a given condition. It creates a Map with two lists, where:

- **Key true:** Elements that satisfy the condition.
- **Key false:** Elements that don't satisfy the condition.

It is useful when we want to separate data into two categories, such as even vs. odd numbers, passing vs. failing students, etc.

Example: Partitioning Numbers into Even and Odd Groups

Example

```
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.Stream;
public class Main {
    public static void main(String[] args) {
        // Creating a Stream of numbers
        Stream<Integer> numbers = Stream.of(1, 2, 3, 4, 5);
// Partitioning numbers into even and odd
        Map<Boolean, List<Integer>> partitioned = numbers
                .collect(Collectors.partitioningBy(n -> n % 2 == 0));
// Printing the result
        System.out.println(partitioned);
    }
}
```

Output:

```
{false=[1, 3, 5], true=[2, 4]}
```

Optional

Questions in this topic (1)

5) What is the purpose of the Optional class?

5) What is the purpose of the Optional class?

The Optional class helps avoid NullPointerException by wrapping values inside a container.

Example

```
Optional<String> name = Optional.ofNullable(null);
System.out.println(name.orElse("Default Name"));  // Output: Default Name
```

Methods in Optional

1. **of():** Creates an Optional for a non-null value.
2. **ofNullable():** Creates an Optional for a nullable value.
3. **isPresent():** Checks if a value is present.
4. **orElse():** Returns a default value if the Optional is empty.
5. **orElseGet():** Returns a default value using a Supplier function.

Date & Time API

Questions in this topic (2)

36) What is the easiest way to print the current date and time using Java 8?

37) What issues have been fixed in Java 8's new Date and Time API?

36) What is the easiest way to print the current date and time using Java 8?

The now method, which is a part of LocalDate, can be used to get the current date as shown below:

```
LocalDate currentDate = LocalDate.now();
System.out.println(currentDate);
```

Similarly, it can also be used to get the current time:

```
LocalTime currentTime = LocalTime.now();
System.out.println(currentTime);
```

To get both the current date and time together, we can use LocalDateTime:

```
LocalDateTime currentDateTime = LocalDateTime.now();
System.out.println(currentDateTime);
```

If we want the date and time in a specific format, we can use DateTimeFormatter:

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
String formattedDateTime = LocalDateTime.now().format(formatter);
System.out.println(formattedDateTime);
```

37) What issues have been fixed in Java 8's new Date and Time API?

It wasn't easy to handle dates and timings in Java prior to Java 8. Working with the Date and Calendar classes became difficult due to a number of problems. Java 8 added the java.time package to solve these issues.

Issue of Mutability

- Unexpected issues in multi-threaded systems were caused by the old Date class's allowing changes after creation.
- It was fixed in Java 8 by making LocalDate and LocalTime immutable, which means that once they are created, they cannot be changed.

Issues with Thread Safety

- Using SimpleDateFormat in several threads could result in issues because it is not thread-safe.
- DateTimeFormatter, which is safe to use in many threads, was introduced in Java 8.

Unclear API

- Uncomfortable methods like getYear() in the previous API returned

JVM & Tools

Questions in this topic (4)

8) How does Java 8 improve memory management?
38) What are PermGen and Metaspace in Java 8?
45) What is Nashorn in Java 8?
46) What is JJS in Java 8?

8) How does Java 8 improve memory management?

- Introduction of Metaspace instead of PermGen (removes memory leaks).
- Improved Garbage Collection (G1 GC, Parallel GC, ZGC).
- Compact Strings for memory efficiency.

38) What are PermGen and Metaspace in Java 8?

Before Java 8, Java relied on PermGen to store information about things like classes, methods, and metadata. The problem with PermGen was its fixed size, set when the JVM started. If an application loads too many classes, it could easily run out of space, causing annoying crashes and errors. It made it difficult to handle applications that loaded and unloaded classes frequently, like web applications.

Java 8 introduced Metaspace, which solved these issues. Instead of being stuck with a fixed-size storage system, Metaspace uses native memory and adjusts its size automatically based on how many classes are loaded. It’s like upgrading from a cramped closet to a stretchable storage unit, it grows or shrinks depending on what we need. The flexibility not only makes memory management easier but also reduces the chances of our application running into memory problems.

45) What is Nashorn in Java 8?

Java 8 provides a new JavaScript processing engine named Nashorn. It provides better compliance with the ECMA-normalized JavaScript specification and better runtime performance than its predecessor.

Prior to Java 8, version Java platform used Mozilla Rhino as a JavaScript processing engine for the same purpose.

Nashorn

46) What is JJS in Java 8?

It is the new executable or command-line tool for executing JavaScript code at the console.

Java 21 Concurrency

Virtual Threads (Java 21)

Questions in this topic (12)

1) What is a virtual thread, and how is it different from a platform thread?
2) Why were virtual threads introduced? What problem do they solve?
3) How are virtual threads scheduled? What is a carrier thread?
4) How do you create virtual threads in Java 21?
5) Should you pool virtual threads? How do you limit concurrency then?
6) When should you NOT use virtual threads?
7) What is virtual thread pinning, and how do you avoid it?
8) How do ThreadLocal variables behave with virtual threads?
9) How do virtual threads compare with reactive programming?
10) How do you observe and debug virtual threads?
11) What other behavioural differences do virtual threads have?
12) How do you enable virtual threads in a Spring Boot application?

1) What is a virtual thread, and how is it different from a platform thread?

A **platform thread** is a thin wrapper around an OS thread (1:1). It is expensive to create and keep, so a JVM can only run a few thousand of them. A **virtual thread** is still a `java.lang.Thread`, but it is not tied to one OS thread: the JVM runs many virtual threads on a small number of platform threads (M:N).

	Platform thread	Virtual thread
Managed by	OS	JVM
Cost to create / keep	High (OS thread plus a large reserved stack)	Very low (stack lives on the heap and grows as needed)
How many	Thousands	Hundreds of thousands to millions
Scheduling	OS scheduler	JVM scheduler (a ForkJoinPool)
Daemon	Either	Always daemon

Virtual threads are **not faster**: they run code at the same speed. They give you more **throughput** for workloads that spend most of their time waiting.

Source: JEP 444

Also checked against: Thread

2) Why were virtual threads introduced? What problem do they solve?

Servers traditionally use one thread per request. That style is simple, but the number of OS threads limits how many requests you can serve at once, and most of the time a request is just waiting on a database or network call. To scale, people moved to asynchronous or reactive code, which is harder to write, debug and profile (stack traces lose the context of the request).

Virtual threads remove the limit: a blocked virtual thread releases its OS thread, so you can create one per task and keep the plain blocking style. Existing code and tools (debuggers, thread dumps) keep working.

Source: JEP 444

3) How are virtual threads scheduled? What is a carrier thread?

The JVM schedules virtual threads on platform threads called **carriers**. The scheduler is a work-stealing `ForkJoinPool` in FIFO mode, and its parallelism defaults to the number of CPU cores (`-Djdk.virtualThreadScheduler.parallelism`).

- When a virtual thread runs, it is **mounted** on a carrier.
- When it blocks (for example on I/O, `sleep`, a `java.util.concurrent` lock or a blocking queue), it is **unmounted**: its stack is saved to the heap and the carrier is free to run another virtual thread.
- When it can continue, it is mounted again, possibly on a different carrier.

Application code never sees the carrier.

Source: JEP 444

4) How do you create virtual threads in Java 21?

```
// 1. Builder
Thread t = Thread.ofVirtual().name("worker-1").unstarted(() -> System.out.println("hi"));
t.start();

// 2. Shortcut
Thread.startVirtualThread(() -> System.out.println("hi"));

// 3. Executor: one new virtual thread per task (recommended for many tasks)
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> callRemoteService());
    }
} // close() waits for all tasks (ExecutorService is AutoCloseable since Java 19)
```

`Thread.isVirtual()` tells you whether a thread is virtual.

Source: JEP 444

Also checked against: Executors, ExecutorService

5) Should you pool virtual threads? How do you limit concurrency then?

No. Virtual threads are cheap, so the pooling reason (thread creation is expensive) disappears. `newVirtualThreadPerTaskExecutor()` is **not** a pool: it starts a new virtual thread for every task.

If you need to limit concurrency, for example to protect a database or an API with a rate limit, limit access to the **resource**, not the number of threads:

```
Semaphore dbPermits = new Semaphore(20);

executor.submit(() -> {
    dbPermits.acquire();
    try {
        return queryDatabase();
    } finally {
        dbPermits.release();
    }
});
```

Source: JEP 444

Also checked against: Executors

6) When should you NOT use virtual threads?

- **CPU-bound work**: virtual threads do not make computation faster, and more threads than cores will not help. Use a fixed pool or parallel streams.
- **Few concurrent tasks**: the benefit shows up with thousands or more concurrent tasks.
- Code that **pins** the carrier with long blocking calls (see the pinning question).
- Code that depends on thread identity, thread priority or heavy `ThreadLocal` use.

Rule of thumb: virtual threads help when many tasks spend most of their time **waiting**.

Source: JEP 444

7) What is virtual thread pinning, and how do you avoid it?

A virtual thread is **pinned** when it cannot unmount while blocked, so it keeps its carrier thread busy and you lose the scalability benefit.

- **Java 21 to 23:** a virtual thread blocking inside a `synchronized` block or method is pinned. Native method or foreign-function calls that block also pin.
- **Java 24 and later (JEP 491):** `synchronized` no longer pins. What remains: native frames, and blocking during class loading or class initialization.

How to avoid it (Java 21): use `ReentrantLock` instead of `synchronized` around blocking calls, keep synchronized sections short and in-memory, or move to Java 24+.

How to detect it: the JFR event `jdk.VirtualThreadPinned` (reported when a pinned thread blocks for longer than 20 ms). In Java 21 to 23 you can also use `-Djdk.tracePinnedThreads=full`; that option was removed in Java 24.

Source: JEP 444, JEP 491

8) How do ThreadLocal variables behave with virtual threads?

Virtual threads fully support `ThreadLocal` and `InheritableThreadLocal`. The catch is scale: every thread has its own copy, and there can be millions of virtual threads.

- Do not use thread locals to cache expensive objects (for example a `SimpleDateFormat` or a connection) the way people did with thread pools; each virtual thread would create its own.
- Use them sparingly for small per-request data.
- **Scoped values** (`ScopedValue`) are the designed alternative for sharing immutable data (preview in Java 21).

Source: JEP 444

9) How do virtual threads compare with reactive programming?

Both let a small number of OS threads serve many concurrent requests. The difference is the programming model:

	Virtual threads	Reactive (e.g. WebFlux)
Code style	Plain sequential blocking code	Callbacks / Mono and Flux pipelines
Debugging	Normal stack traces and debugger	Harder: the stack shows the framework, not your logic
Backpressure and streaming	Not built in (use queues, semaphores)	Built in
Learning curve	Low	High

For ordinary request/response services, virtual threads are usually simpler. Reactive still fits streaming and event pipelines where backpressure matters. This comparison is a design judgement; JEP 444 explains why virtual threads were added as an alternative to the asynchronous style.

Source: JEP 444

10) How do you observe and debug virtual threads?

- **Thread dumps:** `jcmd <pid> Thread.dump_to_file -format=json file.json` lists platform and virtual threads, grouped sensibly, without pausing the application.
- **JFR events:** `jdk.VirtualThreadPinned` and `jdk.VirtualThreadSubmitFailed` are enabled by default; `jdk.VirtualThreadStart` and `jdk.VirtualThreadEnd` are off by default.

- **Debuggers** step through virtual threads normally and stack traces are complete.
- `Thread.getAllStackTraces()` returns only platform threads.

Source: JEP 444

11) What other behavioural differences do virtual threads have?

- Always **daemon** threads, so they do not keep the JVM alive: `setDaemon(false)` throws `IllegalArgumentException`.
- **Priority** is always `NORM_PRIORITY`; `setPriority()` is ignored.
- No thread group of their own: `getThreadGroup()` returns a placeholder group called "VirtualThreads".
- Blocking socket operations are interruptible when called from a virtual thread.
- `ThreadMXBean` deadlock detection only covers platform threads.

Source: JEP 444

Also checked against: Thread

12) How do you enable virtual threads in a Spring Boot application?

With Spring Boot 3.2+ on Java 21, add this property:

```
spring.threads.virtual.enabled=true
```

Spring then runs request handling (embedded Tomcat or Jetty), `@Async` and scheduled tasks on virtual threads. You still need to watch for pinning in libraries (for example older JDBC drivers) and limit access to the database with a connection pool or a semaphore.

Also checked against: Spring Boot reference: task execution, Spring Boot 3.2 release notes

Structured Concurrency & Scoped Values

Questions in this topic (4)

13) What is structured concurrency, and what problem does it solve?

14) How do you use StructuredTaskScope? What are ShutdownOnFailure and ShutdownOnSuccess?

15) What are Scoped Values, and how do they differ from ThreadLocal?

16) Which of the Java 21 concurrency features are final and which are preview?

13) What is structured concurrency, and what problem does it solve?

With `ExecutorService` and `Future`, the relationship between a task and its subtasks exists only in the programmer's head. If one subtask fails, the others keep running (thread leaks), cancellation does not propagate, and thread dumps do not show who started what.

Structured concurrency ties the lifetime of subtasks to a code block: all subtasks must finish (or be cancelled) before the block exits. Errors and cancellation are handled in one place, like a normal `try` block.

Status: a preview API in Java 21 (JEP 453), so you need `--enable-preview`; it is still a preview feature in later releases, and the API changed after 21.

Source: JEP 453

14) How do you use StructuredTaskScope? What are ShutdownOnFailure and ShutdownOnSuccess?

```
// Java 21 preview API (run with --enable-preview)
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Subtask<User>  user  = scope.fork(() -> findUser(id));
    Subtask<Order> order = scope.fork(() -> fetchOrder(id));

    scope.join().throwIfFailed();          // wait for both; rethrow the first failure

    return new Response(user.get(), order.get());
}
```

- `ShutdownOnFailure`: if any subtask fails, the remaining subtasks are cancelled; `throwIfFailed()` reports the error.
- `ShutdownOnSuccess`: the first subtask to succeed wins and the rest are cancelled; `result()` returns it (a "race" pattern).
- `fork()` returns a `Subtask` (not a `Future`); call `get()` only after `join()`.
- Each forked subtask runs in its own (virtual) thread.

In Java 25 the API was reshaped (scopes are opened with `StructuredTaskScope.open()` and a `Joiner`), so check the version you target.

Source: JEP 453

15) What are Scoped Values, and how do they differ from ThreadLocal?

`ThreadLocal` has three weaknesses: any code can call `set()` (unclear data flow), the value lives until removed (leaks, especially in thread pools), and inheriting values into child threads copies data.

A `ScopedValue` is bound for the duration of a call and then disappears automatically. It is immutable (a method can pass it down but not change it for its callers) and child threads started in a `StructuredTaskScope` see it cheaply.

```
private static final ScopedValue<User> CURRENT_USER = ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, user)
           .run(() -> handleRequest());     // anywhere below: CURRENT_USER.get()
```

Status: preview in Java 21 (JEP 446); finalized in Java 25 (JEP 506).

Source: JEP 446, JEP 506

16) Which of the Java 21 concurrency features are final and which are preview?

Feature	Java 21	Later
Virtual threads (JEP 444)	Final	Java 24: synchronized no longer pins (JEP 491)
Structured concurrency (JEP 453)	Preview	Still preview in later releases; API reshaped in Java 25 (JEP 505)
Scoped values (JEP 446)	Preview	Final in Java 25 (JEP 506)
Sequenced collections (JEP 431)	Final	-

In an interview, say which Java version you are talking about, because the answer to "does synchronized pin?" changed between 21 and 24.

Source: JEP 444, JEP 491, JEP 453, JEP 446, JEP 506, JEP 431, JEP 533

Modern Concurrency Essentials

Questions in this topic (15)

17) Explain the parameters of ThreadPoolExecutor and how it handles a new task.
18) What are the rejection policies of ThreadPoolExecutor?
19) How do you size a thread pool?
20) Why are Executors.newFixedThreadPool() and newCachedThreadPool() considered risky in production?
21) What is the difference between shutdown(), shutdownNow() and awaitTermination()?
22) What is the difference between synchronized and ReentrantLock? When would you use each?
23) What is the happens-before relationship? Give the main rules.
24) What is CAS (compare-and-swap), and what is the ABA problem?
25) How does ConcurrentHashMap work internally (Java 8+)? How is it different from Hashtable and synchronizedMap?
26) What are deadlock, livelock and starvation? What are the conditions for deadlock?
27) What is double-checked locking, and why does the field need volatile?
28) What is the difference between thenApply() and thenCompose() in CompletableFuture? What do thenCombine(), allOf() and anyOf() do?
29) How do you handle exceptions and timeouts in CompletableFuture? Which executor does it use?
30) What problems can ThreadLocal cause in thread pools, and how do you avoid them?
31) How do you stop a thread safely?

17) Explain the parameters of ThreadPoolExecutor and how it handles a new task.

```
new ThreadPoolExecutor(
    corePoolSize,      // threads kept even when idle
    maximumPoolSize,   // upper limit of threads
    keepAliveTime, unit, // idle time before extra threads die
    workQueue,         // holds tasks waiting for a thread
    threadFactory,     // names / configures threads
    rejectedHandler);  // what to do when saturated
```

When a task is submitted:

- If fewer than `corePoolSize` threads are running, start a new thread for it.
- Otherwise put it in the queue.
- If the queue is full and fewer than `maximumPoolSize` threads are running, start another thread.
- If the queue is full and the maximum is reached, the task is **rejected**.

A consequence: with an **unbounded** queue the pool never grows past `corePoolSize`, so `maximumPoolSize` is ignored.

Also checked against: ThreadPoolExecutor

18) What are the rejection policies of ThreadPoolExecutor?

- `AbortPolicy` (default): throws `RejectedExecutionException`.
- `CallerRunsPolicy`: the submitting thread runs the task itself, which slows the producer down (simple backpressure).
- `DiscardPolicy`: silently drops the new task.
- `DiscardOldestPolicy`: drops the oldest queued task and retries the new one.
- You can implement `RejectedExecutionHandler` for custom behaviour (log, persist, retry).

Also checked against: ThreadPoolExecutor

19) How do you size a thread pool?

- **CPU-bound tasks:** about `N` or `N + 1` threads, where `N = Runtime.getRuntime().availableProcessors()`.

- **I/O-bound tasks:** `N * (1 + waitTime / computeTime)`, so a task that waits 9 times longer than it computes suggests about 10 threads per core.
- Measure under realistic load and adjust; also bound the queue.
- Use separate pools for CPU-bound and I/O-bound work so one cannot starve the other.
- For huge numbers of mostly-waiting tasks, virtual threads remove the need to size a pool.
- These formulas are widely used rules of thumb (popularised by *Java Concurrency in Practice*), not an official rule, so always measure.

20) Why are Executors.newFixedThreadPool() and newCachedThreadPool() considered risky in production?

- `newFixedThreadPool(n)` uses an **unbounded** `LinkedBlockingQueue`: if producers are faster than consumers, the queue grows until you run out of memory.
- `newCachedThreadPool()` has **no thread limit** (`maximumPoolSize = Integer.MAX_VALUE`) and a `SynchronousQueue`: a burst of tasks can create thousands of threads.

Prefer constructing a `ThreadPoolExecutor` with a **bounded queue**, a sensible maximum and an explicit rejection policy, and give threads meaningful names through a `ThreadFactory`.

Also checked against: Executors, ThreadPoolExecutor

21) What is the difference between shutdown(), shutdownNow() and awaitTermination()?

- `shutdown()`: no new tasks are accepted; tasks already submitted (running and queued) still complete. It does not wait.
- `shutdownNow()`: attempts to stop running tasks by **interrupting** them and returns the list of tasks that never started. Tasks must respond to interruption to stop.
- `awaitTermination(timeout, unit)`: blocks until all tasks finish or the timeout passes.
- From Java 19, `ExecutorService` is `AutoCloseable`: `close()` calls `shutdown()` and waits.

```
pool.shutdown();
if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
    pool.shutdownNow();   // give up waiting: interrupt what is still running
}
```

Also checked against: ExecutorService

22) What is the difference between synchronized and ReentrantLock? When would you use each?

	synchronized	ReentrantLock
Release	Automatic, even on exception	Manual: `unlock()` in `finally`
Try without blocking	No	`tryLock()`, `tryLock(timeout)`
Interruptible while waiting	No	`lockInterruptibly()`
Fairness option	No	`new ReentrantLock(true)`
Condition queues	One (`wait/notify`)	Many (`newCondition()`)

Use `synchronized` by default because it is simple and hard to misuse. Use `ReentrantLock` when you need one of its extra features. Before Java 24 it was also the way to avoid virtual-thread pinning around blocking code.

Also checked against: ReentrantLock

23) What is the happens-before relationship? Give the main rules.

Happens-before is the Java Memory Model's visibility guarantee: if A happens-before B, everything A did is visible to B. Without it, the compiler, CPU or caches may reorder or hide changes between threads.

- **Program order:** each action in a thread happens-before later actions in that thread.
- **Monitor lock:** unlocking a monitor happens-before every later lock of the same monitor.
- `volatile`**:** a write to a volatile field happens-before every later read of it.
- **Thread start:** `Thread.start()` happens-before any action in the started thread.
- **Thread join:** all actions in a thread happen-before another thread returns from `join()` on it.
- **Transitivity:** if A happens-before B and B happens-before C, then A happens-before C.

Also checked against: JLS 17.4.5 Happens-before

24) What is CAS (compare-and-swap), and what is the ABA problem?

CAS is a CPU-level atomic operation: "set the value to `new` only if it is currently `expected`". The `Atomic*` classes use it, so threads can update shared state without locks: they read, compute, CAS, and retry if another thread got there first.

ABA problem: a thread reads `A`, another thread changes it to `B` and back to `A`. The first thread's CAS succeeds because the value looks unchanged, even though the state around it may have changed (typical in lock-free stacks and queues).

Fix: add a version stamp with `AtomicStampedReference` (or `AtomicMarkableReference`) so the stamp must match too.

Also checked against: AtomicStampedReference

25) How does ConcurrentHashMap work internally (Java 8+)? How is it different from Hashtable and synchronizedMap?

- **Java 8+:** an array of bins. Adding to an **empty bin** uses a lock-free **CAS**; adding to a bin that already has nodes **synchronizes on that bin's first node only** (confirmed in the JDK 21 source).
- **Reads** are mostly lock-free, and iterators are **weakly consistent** (no `ConcurrentModificationException`).
- Long bins are converted to **trees** like in `HashMap`.
- `size()` is computed from a base count plus per-thread cells (like `LongAdder`), so counting does not become a bottleneck.
- **Null keys and null values are not allowed**.
- `Hashtable` and `Collections.synchronizedMap()` lock the **whole map** for every operation, so threads queue up.

Compound actions still need atomic methods such as `putIfAbsent`, `computeIfAbsent` or `merge`; a separate `get` then `put` is not atomic.

Also checked against: ConcurrentHashMap

26) What are deadlock, livelock and starvation? What are the conditions for deadlock?

- **Deadlock:** two or more threads each hold a lock the other needs, so none can continue.
- **Livelock:** threads are not blocked but keep changing state in response to each other (like two people stepping aside in a corridor), so no work is done.
- **Starvation:** a thread is ready to run but is repeatedly denied the CPU or a lock (for example by unfair locks or thread priorities).

Deadlock needs all four conditions: mutual exclusion, hold and wait, no preemption, circular wait.

Prevention: always take locks in the same global order, hold locks for a short time, use `tryLock` with a timeout, and avoid calling unknown code while holding a lock. Use a thread dump to diagnose.

Also checked against: Java Tutorial: Deadlock, Java Tutorial: Starvation and Livelock

27) What is double-checked locking, and why does the field need volatile?

```
class Singleton {
    private static volatile Singleton instance;   // volatile is essential

    static Singleton getInstance() {
        if (instance == null) {                   // 1st check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) {           // 2nd check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Without `volatile`, the assignment can be reordered so that `instance` becomes visible **before** the constructor has finished, and another thread could use a partly built object. `volatile` forbids that reordering.

Simpler alternatives: the initialization-on-demand **holder class** idiom, or an **enum** singleton.

Also checked against: Double-Checked Locking is Broken (JSR-133 explanation)

28) What is the difference between thenApply() and thenCompose() in CompletableFuture? What do thenCombine(), allOf() and anyOf() do?

- `thenApply(f)`: transforms the result with a normal function (like `map`).
- `thenCompose(f)`: `f` returns another `CompletableFuture` and the result is flattened (like `flatMap`); use it to chain async calls.
- `thenCombine(other, f)`: waits for two independent futures and combines their results.
- `allOf(...)`: completes when all futures complete (it returns `CompletableFuture<Void>`, so read each result yourself).
- `anyOf(...)`: completes when the first one completes.

```
CompletableFuture<Order> order =
    CompletableFuture.supplyAsync(() -> findUser(id))
        .thenCompose(user -> fetchOrderAsync(user))   // returns CompletableFuture<Order>
        .thenApply(Order::validated);                 // plain function
```

Also checked against: CompletableFuture

29) How do you handle exceptions and timeouts in CompletableFuture? Which executor does it use?

- `exceptionally(ex -> fallback)`: recover from an error with a fallback value.
- `handle((result, ex) -> ...)`: always runs, with either the result or the exception, and returns a new value.
- `whenComplete((result, ex) -> ...)`: side effect (for example logging); it does not change the result.
- `orTimeout(2, SECONDS)` fails the future with a `TimeoutException`; `completeOnTimeout(default, 2, SECONDS)` completes it with a default value (both since Java 9).
- **Executor:** the `*Async` methods use `ForkJoinPool.commonPool()` by default. Pass your own executor for blocking or I/O work, so you do not starve the common pool.

Also checked against: CompletableFuture

30) What problems can ThreadLocal cause in thread pools, and how do you avoid them?

- Threads in a pool are reused, so a value set during one task is still there for the next one: stale data can leak between requests (a security problem).
- The value stays reachable as long as the thread lives, which can cause memory leaks, including class loader leaks in application servers.

Avoid it: always call `remove()` in a `finally` block, store small objects only, and consider `ScopedValue` or passing the data explicitly.

Also checked against: ThreadLocal

31) How do you stop a thread safely?

Java has no safe way to force a thread to stop. Stopping is **cooperative**:

- Call `thread.interrupt()`. Blocking methods such as `sleep`, `wait` and `BlockingQueue.take` then throw `InterruptedException`.
- The task must check `Thread.currentThread().isInterrupted()` in loops and exit cleanly, and must not swallow `InterruptedException` (restore the flag with `Thread.currentThread().interrupt()` if you cannot rethrow).
- A `volatile boolean` stop flag also works for tasks that never block.
- `Thread.stop()` is unsafe (it can leave objects half-updated); it is deprecated for removal and since Java 20 it throws `UnsupportedOperationException`.

For executor tasks use `future.cancel(true)` or `shutdownNow()`, which also rely on interruption.

Also checked against: Thread

Modern JVM & JDBC

Garbage Collection & JVM Memory (Modern)

Questions in this topic (6)

1) How does the JVM decide that an object is garbage?
2) What is the generational heap, and what are minor and full GCs?
3) Which garbage collectors does the JVM offer, and which one is the default?
4) What replaced PermGen?
5) Java has garbage collection. Can it still leak memory? How do you find leaks?
6) How do you choose and tune a garbage collector?

1) How does the JVM decide that an object is garbage?

The collector starts from the **GC roots** and follows references. Everything it can reach is **live**; everything else is garbage, even if those objects still point to each other.

- **GC roots:** local variables and parameters on the stacks of live threads, static fields, active threads, and native (JNI) references.
- **No reference counting:** a cycle `A -> B -> A` that nothing else points to is collected.
- **Reference strength:** strong references keep an object alive; `SoftReference` objects are cleared when memory is tight; `WeakReference` objects are cleared as soon as the object is only weakly reachable.

So memory leaks in Java are objects that are **still reachable** but no longer needed.

Also checked against: JLS 12.6.1: reachable objects, java.lang.ref: reachability levels

2) What is the generational heap, and what are minor and full GCs?

Most objects live for a very short time (the **weak generational hypothesis**). The heap therefore has two main parts:

- **Young generation:** new objects go into **Eden**. A **minor GC** collects only the young generation: survivors are copied into survivor spaces, and objects that survive enough collections are **promoted** to the old generation.
- **Old generation:** long-lived objects. Collecting it is more expensive.
- **Full GC:** collects the whole heap, which is the most expensive and can cause long pauses; a well-tuned application has few of them.

G1 (the default collector) divides the heap into many equal-sized **regions**, each acting as Eden, survivor or old, and collects the regions with the most garbage first. Java 21 also added **Generational ZGC**.

Source: JEP 439

Also checked against: Oracle GC tuning guide: generations, Oracle GC tuning guide: G1

3) Which garbage collectors does the JVM offer, and which one is the default?

Collector	Flag	Notes
Serial	`-XX:+UseSerialGC`	One thread; small heaps and tiny containers
Parallel	`-XX:+UseParallelGC`	Throughput-oriented; it was the default until Java 8
G1	`-XX:+UseG1GC`	**Default since Java 9 (JEP 248)**: region based, balances pause time and throughput; pause goal `-XX:MaxGCPauseMillis` (default 200 ms)
ZGC	`-XX:+UseZGC`	Low latency, very large heaps; production since Java 15 (JEP 377). Generational mode arrived in Java 21 (`-XX:+ZGenerational` , JEP 439), with pauses meant to stay under 1

		ms; generational mode became the default in JDK 23 (JEP 474) and the non-generational mode was removed in JDK 24 (JEP 490)
Shenandoah	`-XX:+UseShenandoahGC`	Low pause; production since Java 15 (JEP 379). It is in many OpenJDK builds (for example Temurin) but is not listed in Oracle's GC tuning guide
CMS	-	**Removed in Java 14** (JEP 363)

Which one is picked when you set nothing: up to JDK 26 the JVM chooses G1 on a 'server-class' machine (2 or more CPUs and about 1792 MB of memory or more) and Serial on smaller ones; **from JDK 27 G1 is the default everywhere (JEP 523)**. Oracle's tuning guide says G1 is selected by default on most hardware. Check what is running with `-Xlog:gc` (it prints *Using G1*).

Confirmed on JDK 21: G1 is selected, the default pause goal is 200 ms, and `-XX:+UseConcMarkSweepGC` is rejected.

Source: JEP 248, JEP 377, JEP 439, JEP 363, JEP 379, JEP 474, JEP 490, JEP 523

Also checked against: Oracle GC tuning guide: available collectors

4) What replaced PermGen?

Before Java 8, class metadata lived in the **permanent generation (PermGen)**, a fixed-size part of the heap sized with `-XX:MaxPermSize`. It caused `OutOfMemoryError: PermGen space`, especially in application servers that reloaded classes.

- **Java 8 (JEP 122)** removed it. Class metadata moved to **native memory** (called **Metaspace**), which grows as needed instead of having a fixed limit; interned strings and static variables moved to the Java heap.
- You can still cap it with `-XX:MaxMetaspaceSize`; running out gives `OutOfMemoryError: Metaspace`, typically from class loader leaks or heavy dynamic class generation.
- `-XX:MaxPermSize` no longer exists: on JDK 21 the JVM refuses to start with it.

Source: JEP 122

Also checked against: Oracle troubleshooting guide: OutOfMemoryError

5) Java has garbage collection. Can it still leak memory? How do you find leaks?

- **Static collections** that only grow (caches without eviction, registries).
- **Listeners and callbacks** that are registered but never removed.
- **ThreadLocal** values in pool threads that are never `remove()`d.
- **Unclosed resources** (streams, connections) and inner or anonymous classes that keep a reference to their outer object.

Finding them: watch heap use after full GCs in the GC log (`-Xlog:gc`) or JFR, take a heap dump (`jcmd <pid> GC.heap_dump file.hprof`) and look for what keeps the biggest objects reachable (Eclipse MAT, VisualVM). **Fixing them:** bound caches, use weak references where fitting, deregister listeners, and close resources with try-with-resources. `-XX:+HeapDumpOnOutOfMemoryError` writes a dump automatically when the heap runs out.

Also checked against: Oracle troubleshooting guide: memory leaks and heap dumps

6) How do you choose and tune a garbage collector?

- **Start with the default (G1)** and a sensible heap (`-Xmx`); in containers use `-XX:MaxRAMPercentage` (the default is 25% of the available memory; confirmed on JDK 21).
- **Turn on logging** with `-Xlog:gc` and look at pause times, frequency and how much the heap shrinks after a collection.
- **Latency matters most** (many tiny requests, strict response times): try **ZGC**. **Throughput matters most** (batch jobs): try **Parallel**.
- **Tune one thing at a time**, for example the pause goal `-XX:MaxGCPauseMillis` for G1, and compare against a load test.
- Reduce **allocation** (fewer short-lived objects) before tuning flags. Do not call `System.gc()`.

Also checked against: Oracle GC tuning guide: available collectors, System.gc() Javadoc: no guarantee

JDBC Best Practices (Modern)

Questions in this topic (5)

7) How do you use try-with-resources with JDBC?

8) How does PreparedStatement protect against SQL injection, and what does it not protect?

9) What is connection pooling, and why should you use a DataSource?

10) What are the transaction isolation levels in JDBC?

11) Is Class.forName() still needed to load a JDBC driver?

7) How do you use try-with-resources with JDBC?

```
String sql = "select id, name from employee where dept = ?";
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, "IT");
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        }
    }
} catch (SQLException e) {
    // handle or rethrow
}
```

- `Connection`, `Statement` and `ResultSet` are all `AutoCloseable`, and they are closed in **reverse** order even when an exception is thrown.
- Closing a `Statement` also closes its current `ResultSet`; a nested try for the `ResultSet` is still good practice when you run several queries.
- With a connection pool, `close()` does not close the physical connection; it returns it to the pool.

The old way (close everything in a `finally` block, with its own try-catch) was long and often leaked connections when one close failed.

Also checked against: java.sql.Statement, JLS 14.20: try-with-resources

8) How does PreparedStatement protect against SQL injection, and what does it not protect?

With a `PreparedStatement`, the database receives the query with `?` placeholders first, and the values afterwards as data. Input such as `x' OR '1'='1` is just a strange string value.

```
PreparedStatement ps = con.prepareStatement("select * from users where name = ? and active = ?");
ps.setString(1, userInput);
ps.setBoolean(2, true);
```

What it does not cover:

- **Names that cannot be parameters:** table names, column names and `ORDER BY` direction. Check them against an allow-list.
- **Dynamic SQL inside stored procedures** that concatenates its arguments.
- `LIKE` **wildcards** in user input (`%`, `_`) still need escaping if you do not want them to act as wildcards.

OWASP lists parameterized queries as the first defence against SQL injection; also give the database account only the privileges it needs.

Source: OWASP SQL Injection Prevention Cheat Sheet

9) What is connection pooling, and why should you use a DataSource?

Opening a database connection is slow (network, authentication, session setup). A **connection pool** opens a number of connections once and lends them out; `close()` on a pooled connection gives it back instead of closing it.

- The `DriverManager` Javadoc says a `DataSource` is the **preferred** way to get a connection: your code asks the `DataSource` and does not care whether it is pooled.
- Common pools: **HikariCP** (Spring Boot always chooses it first when it is on the classpath, and the JDBC starters include it), Tomcat JDBC, Apache DBCP.
- **Pool size:** a small pool is usually better than a large one, because the database can only run so many queries at once. Size it by measuring, and release connections quickly (use try-with-resources).
- **Common problems:** connection leaks (not closing), a pool smaller than the number of threads waiting for connections, and long transactions that hold a connection.

Also checked against: java.sql.DriverManager, Oracle JDBC tutorial: DataSource, Spring Boot reference: connection pools

10) What are the transaction isolation levels in JDBC?

Level (`Connection.TRANSACTION_...`)	Dirty read	Non-repeatable read	Phantom read
`READ_UNCOMMITTED`	possible	possible	possible
`READ_COMMITTED`	prevented	possible	possible
`REPEATABLE_READ`	prevented	prevented	possible
`SERIALIZABLE`	prevented	prevented	prevented

- **Dirty read:** reading data another transaction has changed but not committed.
- **Non-repeatable read:** reading the same row twice and getting different values because another transaction committed in between.
- **Phantom read:** running the same query twice and getting different rows.

```
con.setAutoCommit(false);
con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
```

The default depends on the database: **Oracle** defaults to read committed (it offers read committed, serializable and a read-only mode); **PostgreSQL** defaults to read committed, and its read uncommitted level behaves like read committed; **MySQL InnoDB** defaults to repeatable read. A database may not support every level.

Also checked against: java.sql.Connection, Oracle Database Concepts: data concurrency, PostgreSQL: transaction isolation, MySQL InnoDB isolation levels

11) Is Class.forName() still needed to load a JDBC driver?

`DriverManager` finds drivers on the classpath through the **service-provider mechanism** (a `META-INF/services/java.sql.Driver` file inside the driver JAR), so this line is usually unnecessary:

```
Class.forName("com.mysql.cj.jdbc.Driver");   // old style, normally not needed
```

- Just call `DriverManager.getConnection(url, user, password)`, or use a `DataSource`.
- **It is still needed** for drivers older than JDBC 4.0: Oracle's JDBC tutorial says those must be loaded manually with `Class.forName`.

Also checked against: java.sql.DriverManager, Oracle JDBC tutorial: connecting